

Squirrel

Generated by Doxygen 1.9.8

1 Data Structure Index	1
1.1 Data Structures	1
2 File Index	3
2.1 File List	3
3 Data Structure Documentation	5
3.1 SQCompiler Class Reference	5
3.1.1 Detailed Description	8
3.1.2 Constructor & Destructor Documentation	9
3.1.2.1 SQCompiler()	9
3.1.3 Member Function Documentation	9
3.1.3.1 BIN_EXP()	9
3.1.3.2 BitwiseAndExp()	11
3.1.3.3 BitwiseOrExp()	12
3.1.3.4 BitwiseXorExp()	13
3.1.3.5 ChooseArithOpByToken()	14
3.1.3.6 ChooseCompArithCharByToken()	15
3.1.3.7 ClassExp()	16
3.1.3.8 ClassStatement()	17
3.1.3.9 CommaExpr()	18
3.1.3.10 CompExp()	19
3.1.3.11 Compile()	20
3.1.3.12 CreateFunction()	22
3.1.3.13 DeleteExpr()	24
3.1.3.14 DoWhileStatement()	25
3.1.3.15 Emit2ArgsOP()	26
3.1.3.16 EmitCompoundArith()	27
3.1.3.17 EmitDerefOp()	28
3.1.3.18 EmitLoadConstFloat()	29
3.1.3.19 EmitLoadConstInt()	30
3.1.3.20 EnumStatement()	30
3.1.3.21 EqExp()	31
3.1.3.22 Error()	33
3.1.3.23 Expect()	33
3.1.3.24 ExpectScalar()	35
3.1.3.25 Expression()	37
3.1.3.26 Factor()	39
3.1.3.27 ForEachStatement()	42
3.1.3.28 ForStatement()	43
3.1.3.29 FunctionCallArgs()	45
3.1.3.30 FunctionExp()	46
3.1.3.31 FunctionStatement()	48

3.1.3.32 IfBlock()	49
3.1.3.33 IfStatement()	50
3.1.3.34 INVOKE_EXP()	52
3.1.3.35 IsEndOfStatement()	52
3.1.3.36 Lex()	53
3.1.3.37 LocalDeclStatement()	54
3.1.3.38 LogicalAndExp()	55
3.1.3.39 LogicalOrExp()	57
3.1.3.40 MoveIfCurrentTargetIsLocal()	58
3.1.3.41 MultExp()	59
3.1.3.42 NeedGet()	60
3.1.3.43 OptionalSemicolon()	61
3.1.3.44 ParseBindEnv()	62
3.1.3.45 ParseTableOrClass()	63
3.1.3.46 PlusExp()	64
3.1.3.47 PrefixedExpr()	66
3.1.3.48 PrefixIncDec()	68
3.1.3.49 ResolveBreaks()	69
3.1.3.50 ResolveContinues()	70
3.1.3.51 ShiftExp()	70
3.1.3.52 Statement()	72
3.1.3.53 Statements()	74
3.1.3.54 SwitchStatement()	75
3.1.3.55 ThrowError()	77
3.1.3.56 TryCatchStatement()	78
3.1.3.57 UnaryOP()	79
3.1.3.58 WhileStatement()	80
3.1.4 Field Documentation	82
3.1.4.1 _compilererror	82
3.1.4.2 _debugline	82
3.1.4.3 _debugop	82
3.1.4.4 _errorjmp	82
3.1.4.5 _es	82
3.1.4.6 _fs	83
3.1.4.7 _lex	83
3.1.4.8 _lineinfo	83
3.1.4.9 _raiseerror	83
3.1.4.10 _scope	83
3.1.4.11 _sourcename	84
3.1.4.12 _token	84
3.1.4.13 _vm	84
3.2 SQExpState Struct Reference	84

3.2.1 Detailed Description	85
3.2.2 Field Documentation	85
3.2.2.1 donot_get	85
3.2.2.2 epos	85
3.2.2.3 etype	85
3.3 SQScope Struct Reference	86
3.3.1 Detailed Description	86
3.3.2 Field Documentation	86
3.3.2.1 outers	86
3.3.2.2 stacksize	86
4 File Documentation	87
4.1 squirrel3_squirrel_sqcompiler_v1.cpp File Reference	87
4.1.1 Macro Definition Documentation	88
4.1.1.1 BASE	88
4.1.1.2 BEGIN_BREAKBLE_BLOCK	88
4.1.1.3 BEGIN_SCOPE	107
4.1.1.4 END_BREAKBLE_BLOCK	107
4.1.1.5 END_SCOPE	108
4.1.1.6 END_SCOPE_NO_CLOSE	108
4.1.1.7 EXPR	109
4.1.1.8 LOCAL	109
4.1.1.9 MAX_COMPILER_ERROR_LEN	109
4.1.1.10 OBJECT	109
4.1.1.11 OUTER	109
4.1.1.12 RESOLVE_OUTERS	109
4.1.2 Function Documentation	110
4.1.2.1 Compile()	110
4.2 squirrel3_squirrel_sqcompiler_v1.cpp	111

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

SQCompiler	Squirrel スクリプトをバイトコードにコンパイルするクラス。	5
SQExpState	式の状態を管理する構造体。	84
SQScope	コンパイル時のスコープ情報を保持する構造体。	86

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

squirrel3_squirrel_sqcompiler_v1.cpp		87
--	-------	----

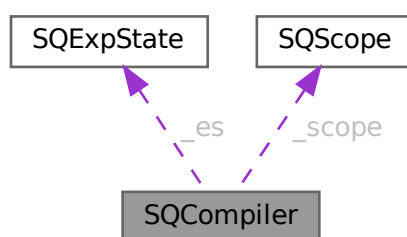
Chapter 3

Data Structure Documentation

3.1 SQCompiler Class Reference

Squirrel スクリプトをバイトコードにコンパイルするクラス。

Collaboration diagram for SQCompiler:



Public Member Functions

- [SQCompiler](#) (SQVM *v, SQLEXREADFUNC rg, SQUserPointer up, const SQChar *sourcename, bool raiseerror, bool lineinfo)
SQCompiler のコンストラクタ。
- void [Error](#) (const SQChar *s,...)
コンパイルエラーを報告し、ロングジャンプでコンパイル処理を中断する。
- void [Lex](#) ()
字句解析器を呼び出し、次のトークンを取得する。
- SQObject [Expect](#) (SQInteger tok)
特定のトークンを期待し、消費する。
- bool [IsEndOfStatement](#) ()
現在のトークンが文の終わりを示しているか判定する。
- void [OptionalSemicolon](#) ()
文末の任意で記述されるセミコロンを処理する。

- void [MovelfCurrentTargetIsLocal](#) ()
現在のターゲットがローカル変数の場合に *MOVE* 命令を生成する。
- bool [Compile](#) (SQObjectPtr &o)
コンパイル処理のメインエントリポイント。
- void [Statements](#) ()
一連の文を解析する。
- void [Statement](#) (bool closeframe=true)
単一の文を解析し、対応するバイトコードを生成する。
- void [EmitDerefOp](#) (SQOpcode op)
デリファレンス操作のオペコードを生成する。
- void [Emit2ArgsOP](#) (SQOpcode op, SQInteger p3=0)
2つの引数を取るオペコードを生成する。
- void [EmitCompoundArith](#) (SQInteger tok, SQInteger etype, SQInteger pos)
複合代入演算子 (*+=*, *-=* など) のコードを生成する。
- void [CommaExpr](#) ()
カンマ区切りの式を解析する。
- void [Expression](#) ()
式を解析します。
- template<typename T >
void [INVOKE_EXP](#) (T f)
式のパーサ関数を呼び出すためのテンプレートヘルパー。
- template<typename T >
void [BIN_EXP](#) (SQOpcode op, T f, SQInteger op3=0)
二項演算子式を解析するためのテンプレートヘルパー。
- void [LogicalOrExp](#) ()
論理 *OR* (*|*) 式を解析します。
- void [LogicalAndExp](#) ()
論理 *AND* (*&&*) 式を解析します。
- void [BitwiseOrExp](#) ()
ビットごとの *OR* (*|*) 式を解析します。
- void [BitwiseXorExp](#) ()
ビットごとの *XOR* (*^*) 式を解析します。
- void [BitwiseAndExp](#) ()
ビットごとの *AND* (*&*) 式を解析します。
- void [EqExp](#) ()
等価比較 (*==*, *!=*, *<=>*) 式を解析します。
- void [CompExp](#) ()
大小比較 (*>*, *<*, *>=*, *<=*, *in*, *instanceof*) 式を解析します。
- void [ShiftExp](#) ()
ビットシフト (*<<*, *>>*, *>>>*) 式を解析します。
- SQOpcode [ChooseArithOpByToken](#) (SQInteger tok)
トークンに基づいて算術演算のオペコードを選択します。
- SQInteger [ChooseCompArithCharByToken](#) (SQInteger tok)
複合代入演算子のトークンに基づいて演算文字を選択します。
- void [PlusExp](#) ()
加算・減算 (*+*, *-*) 式を解析します。
- void [MultExp](#) ()
乗算・除算・剰余 (***, */*, *%*) 式を解析します。
- void [PrefixedExpr](#) ()
接頭辞式および後置式 (メンバアクセス、関数呼び出し等) を解析します。
- SQInteger [Factor](#) ()

- 式の構成要素（因子）を解析します。
- void [EmitLoadConstInt](#) (SQInteger value, SQInteger target)
整数定数をロードする命令を生成します。
- void [EmitLoadConstFloat](#) (SQFloat value, SQInteger target)
浮動小数点数定数をロードする命令を生成します。
- void [UnaryOP](#) (SQOpcode op)
単項演算子の命令を生成します。
- bool [NeedGet](#) ()
オブジェクトのメンバアクセス時に値を取得 (*GET*) する必要があるかを判断します。
- void [FunctionCallArgs](#) (bool rawcall=false)
関数呼び出しの引数リストを解析します。
- void [ParseTableOrClass](#) (SQInteger separator, SQInteger terminator)
テーブルまたはクラスの本体 (*{}*内) を解析します。
- void [LocalDeclStatement](#) ()
ローカル変数またはローカル関数の宣言文を解析します。
- void [IfBlock](#) ()
if 文または *else* 文の本体ブロックを解析します。
- void [IfStatement](#) ()
if-else 文を解析します。
- void [WhileStatement](#) ()
while ループを解析します。
- void [DoWhileStatement](#) ()
do-while ループを解析します。
- void [ForStatement](#) ()
C スタイルの *for* ループ (*for*(*init*; *cond*; *inc*)) を解析します。
- void [ForEachStatement](#) ()
foreach ループ (*foreach*(*val*, *key* in *container*)) を解析します。
- void [SwitchStatement](#) ()
switch 文を解析します。
- void [FunctionStatement](#) ()
function 文による関数宣言を解析します。
- void [ClassStatement](#) ()
class 文によるクラス宣言を解析します。
- SQObject [ExpectScalar](#) ()
スカラー値 (整数、浮動小数点数、文字列、真偽値) を期待し、解析します。
- void [EnumStatement](#) ()
enum (列挙型) の宣言を解析します。
- void [TryCatchStatement](#) ()
try-catch 文を解析します。
- SQInteger [ParseBindEnv](#) ()
関数の環境束縛構文 [...] を解析します。
- void [FunctionExp](#) (bool lambda=false)
関数式 (*function*() {...}) またはラムダ式 (*@*() {...}) を解析します。
- void [ClassExp](#) ()
クラス式 (*class extends ... {}*) を解析します。
- void [DeleteExpr](#) ()
delete 式を解析します。
- void [PrefixIncDec](#) (SQInteger token)
前置インクリメント/デクリメント (*++var*, *--var*) を解析します。
- void [CreateFunction](#) (SQObject &name, SQInteger boundtarget, bool lambda=false)
関数プロトタイプを生成します。

- void [ResolveBreaks](#) (SQFuncState *funcstate, SQInteger ntoreresolve)
未解決の *break* 文のジャンプ先を解決します。
- void [ResolveContinues](#) (SQFuncState *funcstate, SQInteger ntoreresolve, SQInteger targetpos)
未解決の *continue* 文のジャンプ先を解決します。

Static Public Member Functions

- static void [ThrowError](#) (void *ud, const SQChar *s)
字句解析器からのエラーを中継する静的コールバック関数。

Private Attributes

- SQInteger [_token](#)
現在のトークン
- SQFuncState * [_fs](#)
現在の関数状態へのポインタ
- SQObjectPtr [_sourcename](#)
ソース名
- SQLexer [_lex](#)
字句解析器インスタンス
- bool [_lineinfo](#)
行情報を生成するかどうかのフラグ
- bool [_raiseerror](#)
エラーハンドラを呼び出すかどうかのフラグ
- SQInteger [_debugline](#)
デバッグ用の行番号
- SQInteger [_debugop](#)
デバッグ用のオペコード
- SQExpState [_es](#)
現在の式の状態
- SQScope [_scope](#)
現在のスコープ情報
- SQChar [_compilererror](#) [MAX_COMPILER_ERROR_LEN]
コンパイラエラーメッセージ用バッファ
- jmp_buf [_errorjmp](#)
エラー発生時のジャンプ先バッファ
- SQVM * [_vm](#)
Squirrel VM へのポインタ

3.1.1 Detailed Description

Squirrel スクリプトをバイトコードにコンパイルするクラス。

字句解析、構文解析、中間コード生成の全プロセスを管理します。

Definition at line 131 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

3.1.2 Constructor & Destructor Documentation

3.1.2.1 SQCompiler()

```
SQCompiler::SQCompiler (
    SQVM * v,
    SQLEXREADFUNC rg,
    SQUserPointer up,
    const SQChar * sourcename,
    bool raiseerror,
    bool lineinfo ) [inline]
```

SQCompiler のコンストラクタ。

コンパイラの初期化を行います。

Parameters

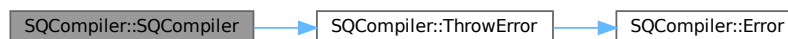
<i>v</i>	関連付けられるSquirrel VM へのポインタ。
<i>rg</i>	字句解析器がソースコードを読み込むための関数ポインタ。
<i>up</i>	rg 関数に渡されるユーザーポインタ。
<i>sourcename</i>	コンパイル対象のソース名。
<i>raiseerror</i>	エラー発生時にエラーハンドラを呼び出すかどうか。
<i>lineinfo</i>	バイトコードに行情報を含めるかどうか。

Definition at line 144 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00145     {
00146         _vm=v;
00147         _lex.Init(_ss(v), rg, up,ThrowError,this);
00148         _sourcename = SQString::Create(_ss(v), sourcename);
00149         _lineinfo = lineinfo;_raiseerror = raiseerror;
00150         _scope.outers = 0;
00151         _scope.stacksize = 0;
00152         _compilererror[0] = _SC('\0');
00153     }
```

References [_compilererror](#), [_lex](#), [_lineinfo](#), [_raiseerror](#), [_scope](#), [_sourcename](#), [_vm](#), [SQScope::outers](#), [SQScope::stacksize](#), and [ThrowError\(\)](#).

Here is the call graph for this function:



3.1.3 Member Function Documentation

3.1.3.1 BIN_EXP()

```
template<typename T >
void SQCompiler::BIN_EXP (
```

```
SQOpcode op,
T f,
SQInteger op3 = 0 ) [inline]
```

二項演算子式を解析するためのテンプレートヘルパー。

現在のトークンを消費し、より高い優先順位の式を解析するためにパーサー関数 `f` を呼び出します。その後、スタックから2つのオペランドをポップし、指定されたオペコード `op` で二項演算命令を生成します。

Template Parameters

<code>T</code>	呼び出すメンバ関数の型。
----------------	--------------

Parameters

<code>op</code>	生成する命令のオペコード。
<code>f</code>	右辺を解析するために呼び出す、より高い優先順位のパーサー関数。
<code>op3</code>	命令に渡す追加の引数（ビット演算の種類など）。

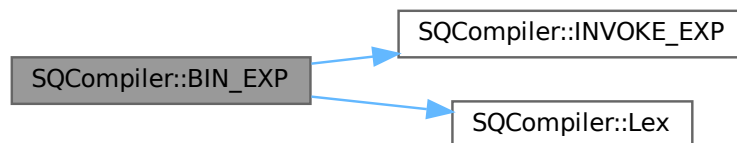
Definition at line 632 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00633 {
00634     Lex();
00635     INVOKE_EXP(f);
00636     SQInteger op1 = _fs->PopTarget(); SQInteger op2 = _fs->PopTarget();
00637     _fs->AddInstruction(op, _fs->PushTarget(), op1, op2, op3);
00638     _es.etype = EXPR;
00639 }
```

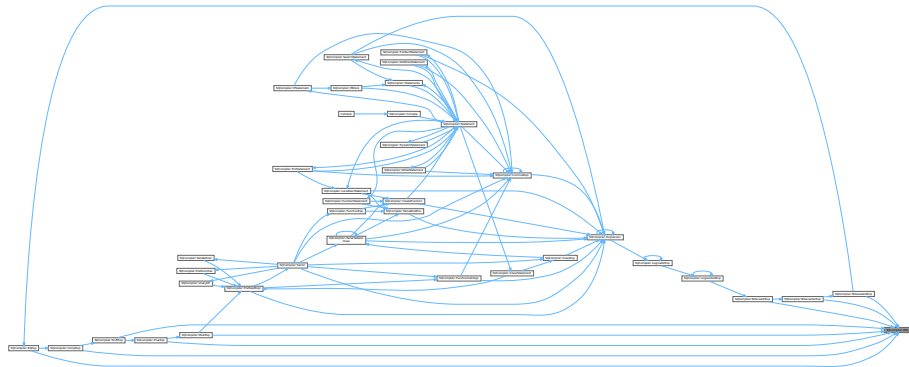
References [_es](#), [_fs](#), [SQExpState::etype](#), [EXPR](#), [INVOKE_EXP\(\)](#), and [Lex\(\)](#).

Referenced by [BitwiseAndExp\(\)](#), [BitwiseOrExp\(\)](#), [BitwiseXorExp\(\)](#), [CompExp\(\)](#), [EqExp\(\)](#), [MultExp\(\)](#), [PlusExp\(\)](#), and [ShiftExp\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.2 BitwiseAndExp()

```
void SQCompiler::BitwiseAndExp ( ) [inline]
```

ビットごとのAND (&) 式を解析します。

BIN_EXP ヘルパーを利用して、ビットごとのAND 演算命令を生成します。

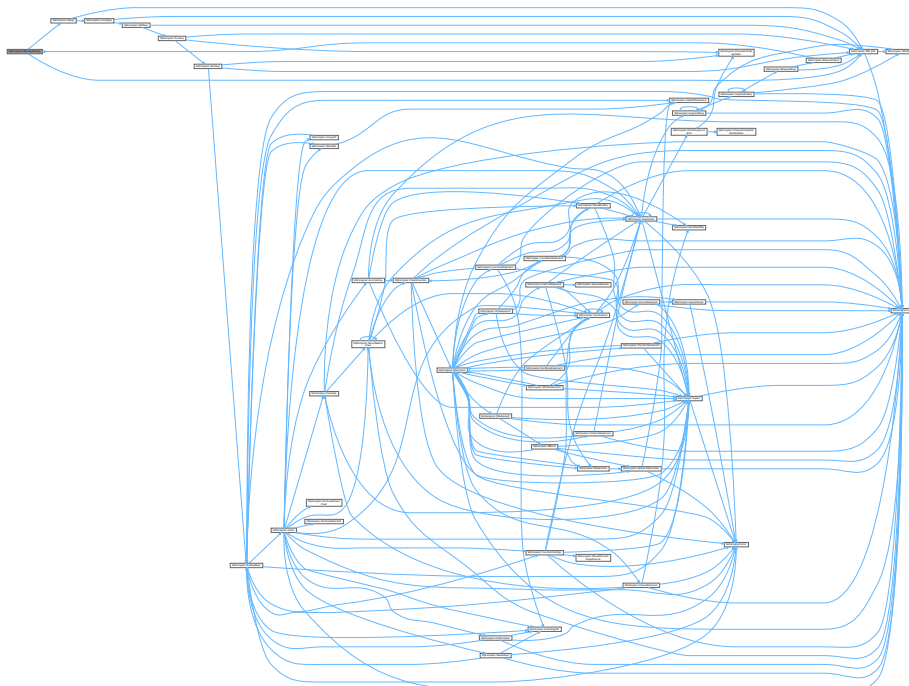
Definition at line 717 of file `squirrel3_squirrel_sqcompiler_v1.cpp`.

```
00718     {
00719         EqExp();
00720         for(;;) if(_token == _SC('&'))
00721             {BIN_EXP(_OP_BITW, &SQCompiler::EqExp, BW_AND);
00722              }else return;
00723     }
```

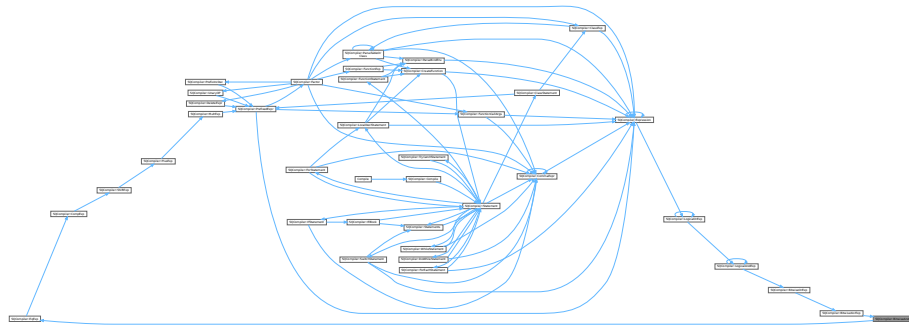
References `_token`, `BIN_EXP()`, and `EqExp()`.

Referenced by `BitwiseXorExp()`.

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.3 BitwiseOrExp()

```
void SQCompiler::BitwiseOrExp ( ) [inline]
```

ビットごとのOR (|) 式を解析します。

BIN_EXP ヘルパーを利用して、ビットごとのOR 演算命令を生成します。

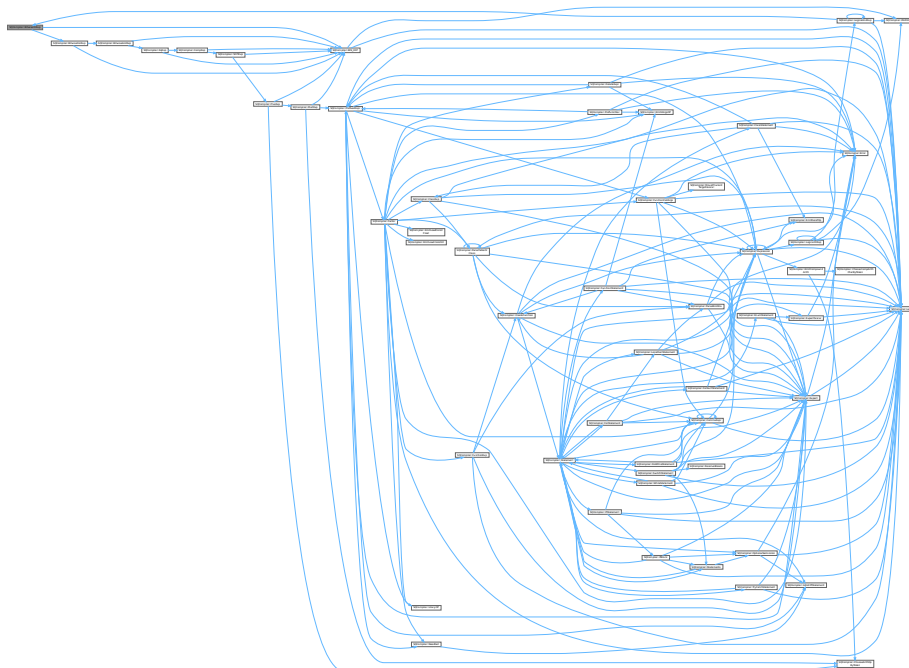
Definition at line 695 of file `squirrel3_squirrel_sqcompiler_v1.cpp`.

```
00696     {
00697         BitwiseXorExp();
00698         for(;;) if(_token == _SC('|'))
00699             {BIN_EXP(_OP_BITW, &SQCompiler::BitwiseXorExp, BW_OR);
00700              }else return;
00701     }
```

References `_token`, `BIN_EXP()`, and `BitwiseXorExp()`.

Referenced by `LogicalAndExp()`.

Here is the call graph for this function:



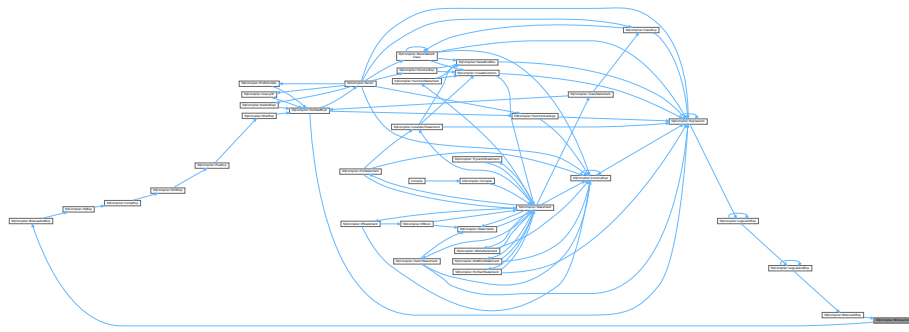
```
void SQCompiler::BitwiseXorExp ( ) [inline]
```

BIN EXP ヘルパーを利用して、ビットごとのXOR 演算命令を生成します。

```
00707 {
00708     BitwiseAndExp();
00709     for(;;) if(_token == _SC('^'))
00710     {BIN_EXP(_OP_BITW, &SQCompiler::BitwiseAndExp, BW_XOR);
00711     }else return;
00712 }
```

Referenced by [BitwiseOrExp\(\)](#).

Here is the caller graph for this function:



3.1.3.5 ChooseArithOpByToken()

```
SQOpcode SQCompiler::ChooseArithOpByToken (
    SQInteger tok ) [inline]
```

トークンに基づいて算術演算のオペコードを選択します。

+, -, *, /, % や +=, -= などのトークンを、対応するVM のオペコード (_OP_ADD, _OP_SUB など) に変換します。

Parameters

<i>tok</i>	変換するトークン。
------------	-----------

Returns

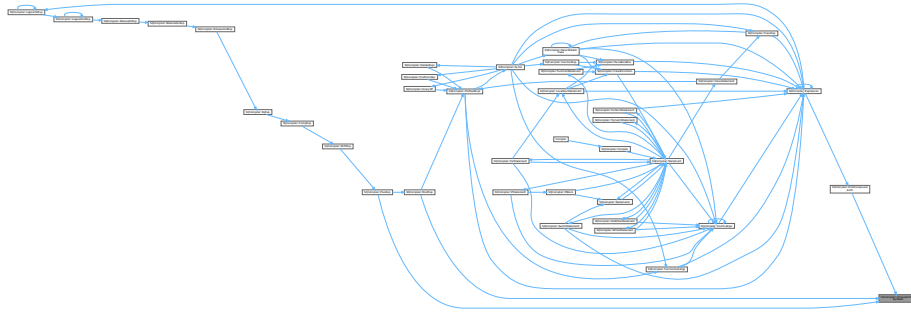
対応する算術演算オペコード。

Definition at line 775 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00776     {
00777         switch(tok) {
00778             case TK_PLUSEQ: case '+': return _OP_ADD;
00779             case TK_MINUSEQ: case '-': return _OP_SUB;
00780             case TK_MULEQ: case '*': return _OP_MUL;
00781             case TK_DIVEQ: case '/': return _OP_DIV;
00782             case TK_MODEQ: case '%': return _OP_MOD;
00783             default: assert(0);
00784         }
00785         return _OP_ADD;
00786     }
```

Referenced by [EmitCompoundArith\(\)](#), [MultExp\(\)](#), and [PlusExp\(\)](#).

Here is the caller graph for this function:



3.1.3.6 ChooseCompArithCharByToken()

```
SQInteger SQCompiler::ChooseCompArithCharByToken (
    SQInteger tok ) [inline]
```

複合代入演算子のトークンに基づいて演算文字を選択します。

`+=`, `-=` などのトークンを、`_OP_COMPARITH` 命令で使用する演算子文字 (`+`, `-` など) に変換します。

Parameters

<i>tok</i>	変換する複合代入演算子のトークン。
------------	-------------------

Returns

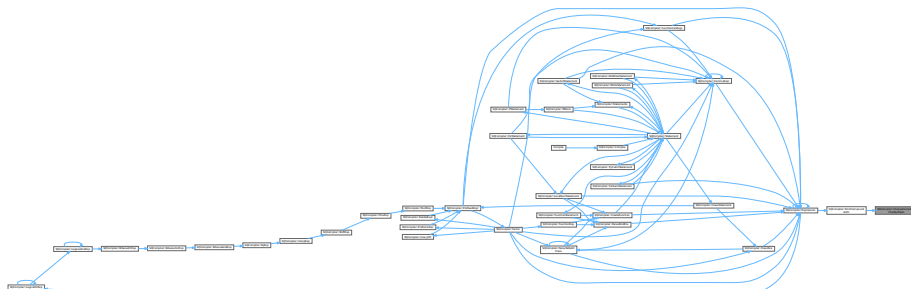
対応する演算子文字を表す整数。

Definition at line 793 of file `squirrel3_squirrel_sqcompiler_v1.cpp`.

```
00794 {
00795     SQInteger oper;
00796     switch(tok){
00797         case TK_MINUSEQ: oper = '-'; break;
00798         case TK_PLUSEQ: oper = '+'; break;
00799         case TK_MULEQ: oper = '*'; break;
00800         case TK_DIVEQ: oper = '/'; break;
00801         case TK_MODEQ: oper = '%'; break;
00802         default: oper = 0; //shut up compiler
00803         assert(0); break;
00804     };
00805     return oper;
00806 }
```

Referenced by [EmitCompoundArith\(\)](#).

Here is the caller graph for this function:



3.1.3.7 ClassExp()

```
void SQCompiler::ClassExp ( ) [inline]
```

クラス式 (class extends ... {}) を解析します。

extends による継承や、<<...>> による属性の指定を処理し、新しいクラスオブジェクトを生成するための `_OP_NEWOBJ` 命令を追加します。その後、クラス本体を解析します。

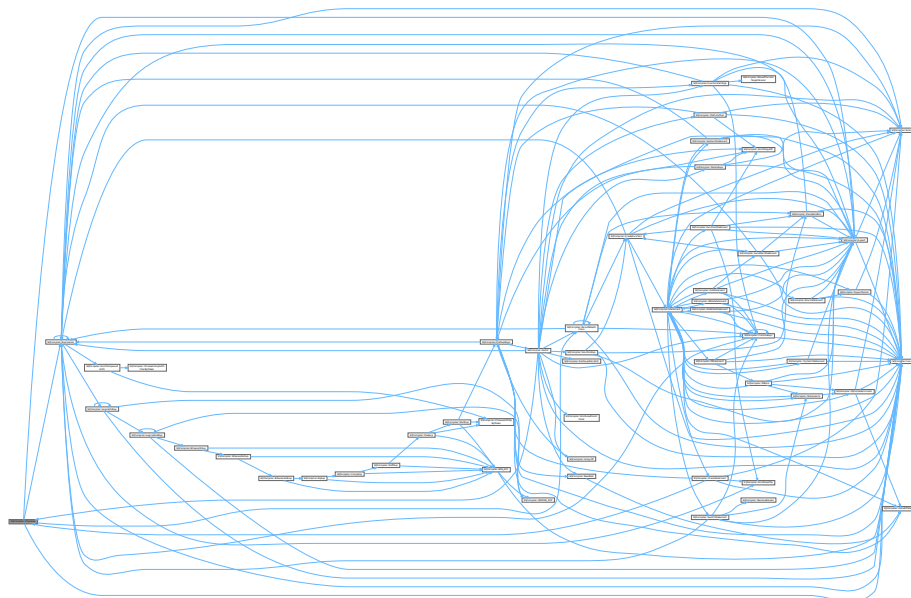
Definition at line 1826 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
01827 {
01828     SQInteger base = -1;
01829     SQInteger attrs = -1;
01830     if(_token == TK_EXTENDS) {
01831         Lex(); Expression();
01832         base = _fs->TopTarget();
01833     }
01834     if(_token == TK_ATTR_OPEN) {
01835         Lex();
01836         _fs->AddInstruction(_OP_NEWOBJ, _fs->PushTarget(), 0, NOT_TABLE);
01837         ParseTableOrClass(_SC(' '), TK_ATTR_CLOSE);
01838         attrs = _fs->TopTarget();
01839     }
01840     Expect(_SC('{'));
01841     if(attrs != -1) _fs->PopTarget();
01842     if(base != -1) _fs->PopTarget();
01843     _fs->AddInstruction(_OP_NEWOBJ, _fs->PushTarget(), base, attrs, NOT_CLASS);
01844     ParseTableOrClass(_SC(';'), _SC('}'));
01845 }
```

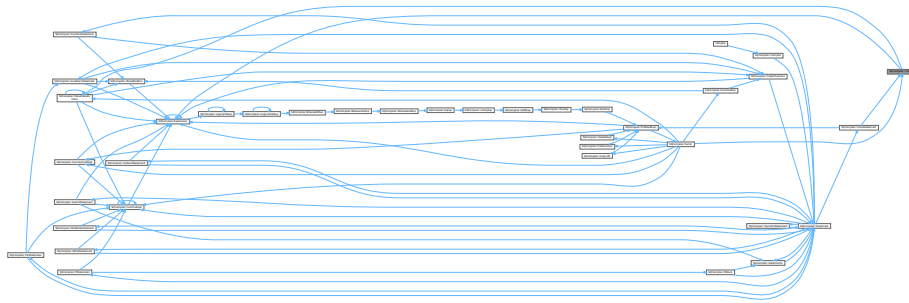
References [_fs](#), [_token](#), [Expect\(\)](#), [Expression\(\)](#), [Lex\(\)](#), and [ParseTableOrClass\(\)](#).

Referenced by [ClassStatement\(\)](#), and [Factor\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.8 ClassStatement()

```
void SQCompiler::ClassStatement ( ) [inline]
```

class 文によるクラス宣言を解析します。

class MyClass ... や class MyNamespace.MyClass ... といった構文を解析します。クラス式を評価し、結果を指定されたスロットに新しいスロットとして設定します。

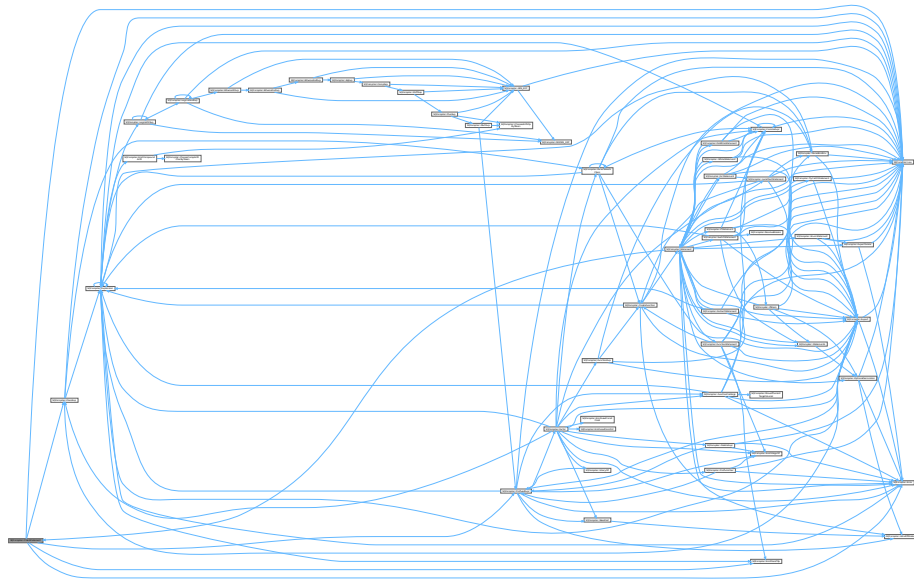
Definition at line 1651 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
01652     {
01653         SQExpState es;
01654         Lex();
01655         es = _es;
01656         _es.donot_get = true;
01657         PrefixedExpr();
01658         if(_es.etype == EXPR) {
01659             Error(_SC("invalid class name"));
01660         }
01661         else if(_es.etype == OBJECT || _es.etype == BASE) {
01662             ClassExp();
01663             EmitDerefOp(_OP_NEWSLOT);
01664             _fs->PopTarget();
01665         }
01666         else {
01667             Error(_SC("cannot create a class in a local with the syntax(class <local>)"));
01668         }
01669         _es = es;
01670     }
```

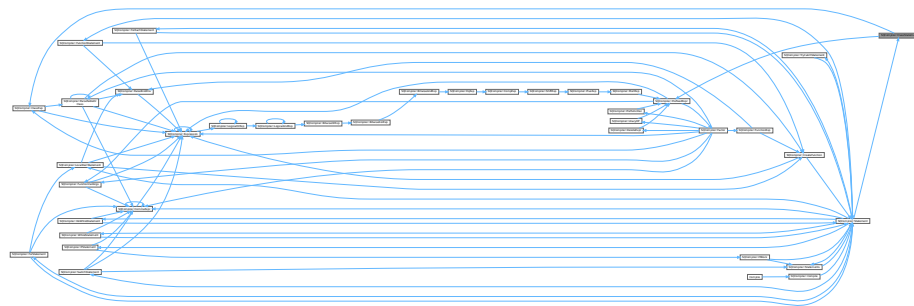
References [_es](#), [_fs](#), [BASE](#), [ClassExp\(\)](#), [SQExpState::donot_get](#), [EmitDerefOp\(\)](#), [Error\(\)](#), [SQExpState::etype](#), [EXPR](#), [Lex\(\)](#), [OBJECT](#), and [PrefixedExpr\(\)](#).

Referenced by [Statement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.9 CommaExpr()

```
void SQCompiler::CommaExpr ( ) [inline]
```

カンマ区切りの式を解析する。

カンマが続く限り、式の解析を再帰的に行います。各式の評価後、結果は破棄されます（最後の式の結果のみが残る）。

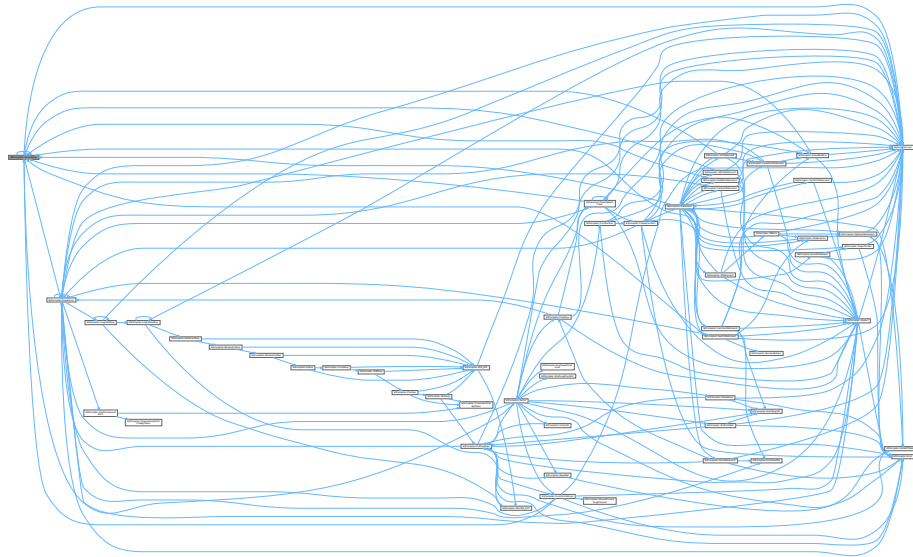
Definition at line 515 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00516     {
00517         for(Expression();_token == ',';_fs->PopTarget(), Lex(), CommaExpr());
00518     }
```

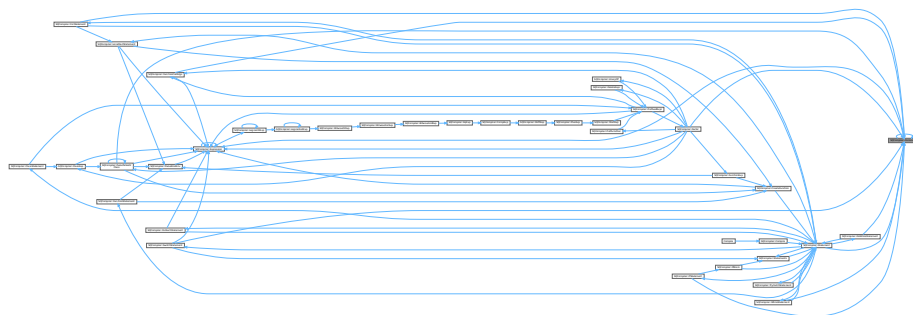
References [_fs](#), [_token](#), [CommaExpr\(\)](#), [Expression\(\)](#), and [Lex\(\)](#).

Referenced by [CommaExpr\(\)](#), [DoWhileStatement\(\)](#), [Factor\(\)](#), [ForStatement\(\)](#), [FunctionCallArgs\(\)](#), [IfStatement\(\)](#), [ParseTableOrClass\(\)](#), [Statement\(\)](#), [SwitchStatement\(\)](#), and [WhileStatement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.10 CompExp()

```
void SQCompiler::CompExp ( ) [inline]
```

大小比較 (>, <, >=, <=, in, instanceof) 式を解析します。

BIN_EXP ヘルパーを利用して、大小比較や存在確認、インスタンス確認の命令を生成します。

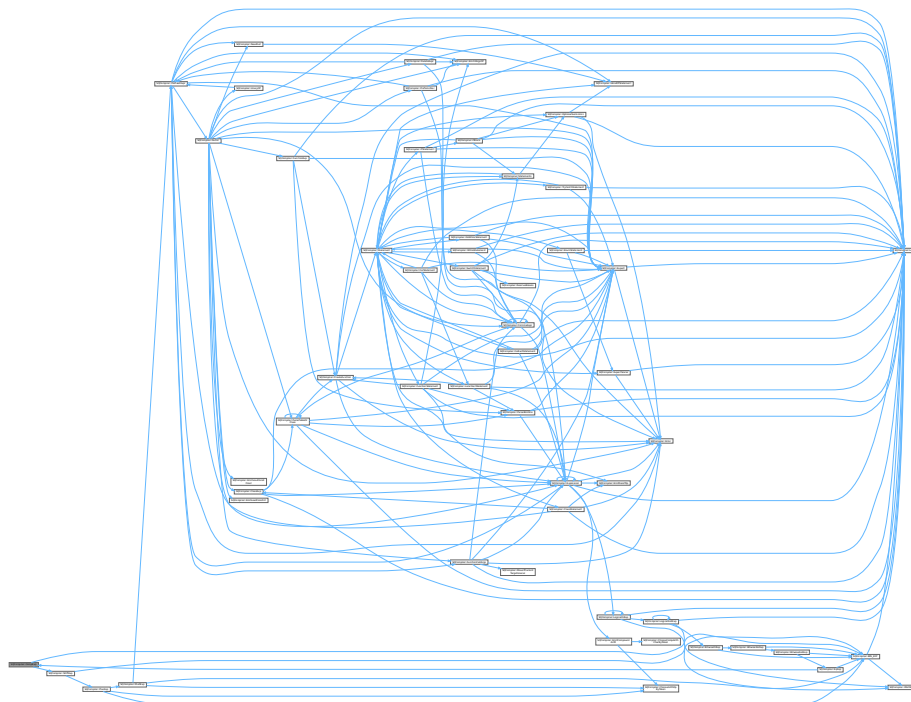
Definition at line 742 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00743     {
00744         ShiftExp();
00745         for(;;) switch(_token) {
00746             case _SC('>'): BIN_EXP(_OP_CMP, &SQCompiler::ShiftExp, CMP_G); break;
00747             case _SC('<'): BIN_EXP(_OP_CMP, &SQCompiler::ShiftExp, CMP_L); break;
00748             case TK_GE: BIN_EXP(_OP_CMP, &SQCompiler::ShiftExp, CMP_GE); break;
00749             case TK_LE: BIN_EXP(_OP_CMP, &SQCompiler::ShiftExp, CMP_LE); break;
00750             case TK_IN: BIN_EXP(_OP_EXISTS, &SQCompiler::ShiftExp); break;
00751             case TK_INSTANCEOF: BIN_EXP(_OP_INSTANCEOF, &SQCompiler::ShiftExp); break;
00752             default: return;
00753         }
00754     }
```

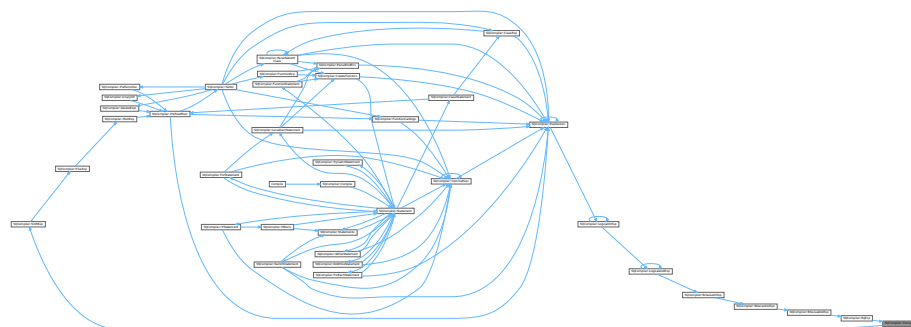
References [_token](#), [BIN_EXP\(\)](#), and [ShiftExp\(\)](#).

Referenced by [EqExp\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.11 Compile()

```
bool SQCompiler::Compile (
    SQObjectPtr & o ) [inline]
```

コンパイル処理のメインエントリーポイント。

ソースコードを読み込み、解析し、最終的な関数プロトタイプオブジェクトを生成します。

Parameters

out	<i>o</i>	コンパイル結果として生成された関数プロトタイプの格納先。
-----	----------	------------------------------

Returns

コンパイルが成功した場合は true、失敗した場合は false。

Definition at line 280 of file squirrel3_squirrel_sqcompiler_v1.cpp.

```

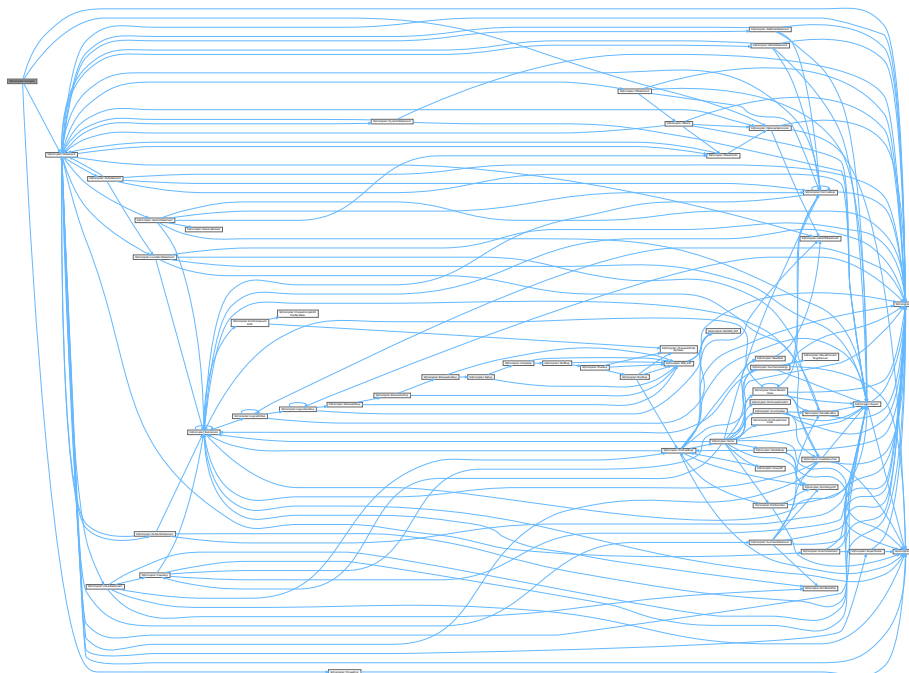
00281     {
00282         _debugline = 1;
00283         _debugop = 0;
00284
00285         SQFuncState funcstate(_ss(_vm), NULL, ThrowError, this);
00286         funcstate._name = SQString::Create(_ss(_vm), _SC("main"));
00287         _fs = &funcstate;
00288         _fs->AddParameter(_fs->CreateString(_SC("this")));
00289         _fs->AddParameter(_fs->CreateString(_SC("vargv")));
00290         _fs->_varparams = true;
00291         _fs->_sourcename = _sourcename;
00292         SQInteger stacksize = _fs->GetStackSize();
00293         if(setjmp(_errorjmp) == 0) {
00294             Lex();
00295             while(_token > 0){
00296                 Statement();
00297                 if(_lex._prevtoken != _SC('}') && _lex._prevtoken != _SC(';')) OptionalSemicolon();
00298             }
00299             _fs->SetStackSize(stacksize);
00300             _fs->AddLineInfos(_lex._currentline, _lineinfo, true);
00301             _fs->AddInstruction(_OP_RETURN, 0xFF);
00302             _fs->SetStackSize(0);
00303             o = _fs->BuildProto();
00304 #ifdef _DEBUG_DUMP
00305             _fs->Dump(_funcproto(o));
00306 #endif
00307         }
00308         else {
00309             if(_raiseerror && _ss(_vm)->_compilererrorhandler) {
00310                 _ss(_vm)->_compilererrorhandler(_vm, _compilererror, sq_type(_sourcename) ==
00311                 OT_STRING?_stringval(_sourcename):_SC("unknown"),
00312                 _lex._currentline, _lex._currentcolumn);
00313             }
00314             _vm->_lasterror = SQString::Create(_ss(_vm), _compilererror, -1);
00315             return false;
00316         }
00317         return true;
00318     }

```

References [_compilererror](#), [_debugline](#), [_debugop](#), [_errorjmp](#), [_fs](#), [_lex](#), [_lineinfo](#), [_raiseerror](#), [_sourcename](#), [_token](#), [_vm](#), [Lex\(\)](#), [OptionalSemicolon\(\)](#), [Statement\(\)](#), and [ThrowError\(\)](#).

Referenced by [Compile\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.12 CreateFunction()

```
void SQCompiler::CreateFunction (
    SQObject & name,
    SQInteger boundtarget,
    bool lambda = false ) [inline]
```

関数プロトタイプを生成します。

新しい関数状態 (SQFuncState) を作成し、仮引数、デフォルト引数、可変長引数を解析します。その後、関数本体を解析し、最終的に SQFunctionProto オブジェクトを構築して親の関数状態に登録します。

Parameters

<i>name</i>	関数の名前。
<i>boundtarget</i>	環境束縛のターゲットとなるスタック位置。束縛がない場合は 0xFF。
<i>lambda</i>	ラムダ式かどうかを示すフラグ。

Definition at line 1907 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```

1908     {
1909         SQFuncState *funcstate = _fs->PushChildState(_ss(_vm));
1910         funcstate->_name = name;
1911         SQObject paramname;
1912         funcstate->AddParameter(_fs->CreateString(_SC("this")));
1913         funcstate->_sourcename = _sourcename;
1914         SQInteger defparams = 0;
1915         while(_token != _SC('')) {
1916             if(_token == TK_VARPARAMS) {
1917                 if(defparams > 0) Error(_SC("function with default parameters cannot have variable
number of parameters"));
1918                 funcstate->AddParameter(_fs->CreateString(_SC("vargv")));
1919                 funcstate->_varparams = true;
1920                 Lex();
1921                 if(_token != _SC('')) Error(_SC("expected ' '"));
1922                 break;
1923             }
1924             else {
1925                 paramname = Expect(TK_IDENTIFIER);
1926                 funcstate->AddParameter(paramname);
1927                 if(_token == _SC('=')) {
1928                     Lex();
1929                     Expression();
1930                     funcstate->AddDefaultParam(_fs->TopTarget());
1931                     defparams++;
1932                 }
1933                 else {
1934                     if(defparams > 0) Error(_SC("expected '='"));
1935                 }
1936                 if(_token == _SC(',',')) Lex();
1937                 else if(_token != _SC('')) Error(_SC("expected ' ' or ','"));
1938             }
1939         }
  
```

```

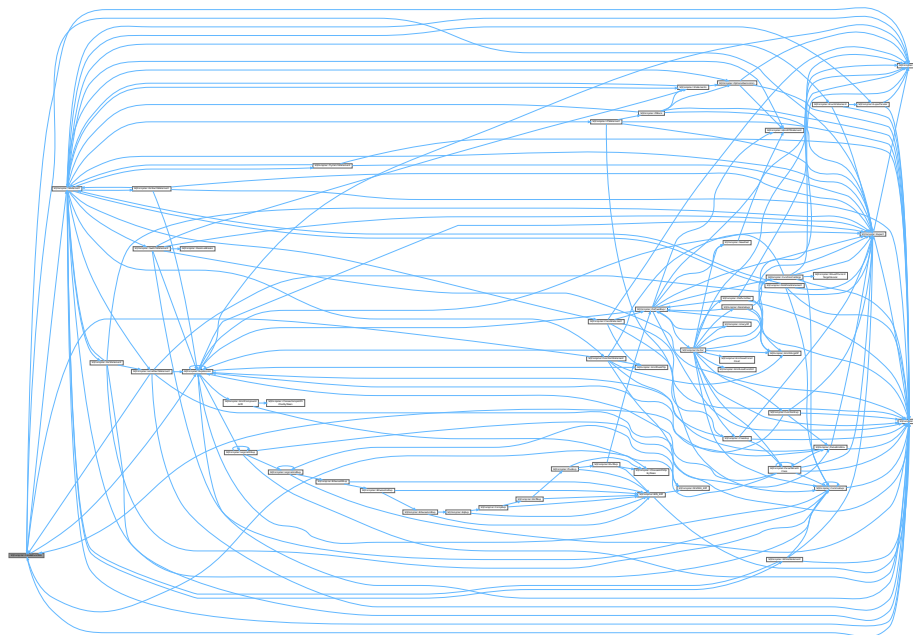
01940     Expect(_SC('')');
01941     if (boundtarget != 0xFF) {
01942         _fs->PopTarget();
01943     }
01944     for(SQInteger n = 0; n < defparams; n++) {
01945         _fs->PopTarget();
01946     }
01947
01948     SQFuncState *currchunk = _fs;
01949     _fs = funcstate;
01950     if(lambda) {
01951         Expression();
01952         _fs->AddInstruction(_OP_RETURN, 1, _fs->PopTarget());
01953     } else {
01954         Statement(false);
01955     }
01956     funcstate->AddLineInfos(_lex._prevtoken == _SC('\n')?_lex._lasttokenline:_lex._currentline,
_lineinfo, true);
01957     funcstate->AddInstruction(_OP_RETURN, -1);
01958     funcstate->SetStackSize(0);
01959
01960     SQFunctionProto *func = funcstate->BuildProto();
01961     #ifdef _DEBUG_DUMP
01962     funcstate->Dump(func);
01963     #endif
01964     _fs = currchunk;
01965     _fs->_functions.push_back(func);
01966     _fs->PopChildState();
01967 }

```

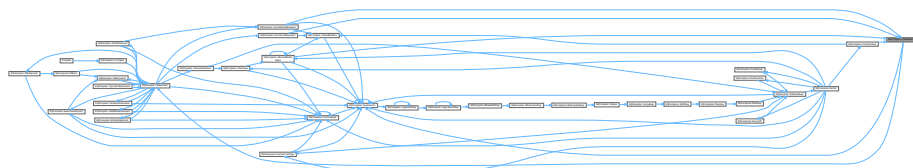
References [_fs](#), [_lex](#), [_lineinfo](#), [_sourcename](#), [_token](#), [_vm](#), [Error\(\)](#), [Expect\(\)](#), [Expression\(\)](#), [Lex\(\)](#), and [Statement\(\)](#).

Referenced by [FunctionExp\(\)](#), [FunctionStatement\(\)](#), [LocalDeclStatement\(\)](#), and [ParseTableOrClass\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.13 DeleteExpr()

```
void SQCompiler::DeleteExpr ( ) [inline]
```

delete 式を解析します。

delete table.slot のような構文を解析し、_OP_DELETE 命令を生成します。ローカル変数や式自体は削除できないため、エラーチェックも行います。

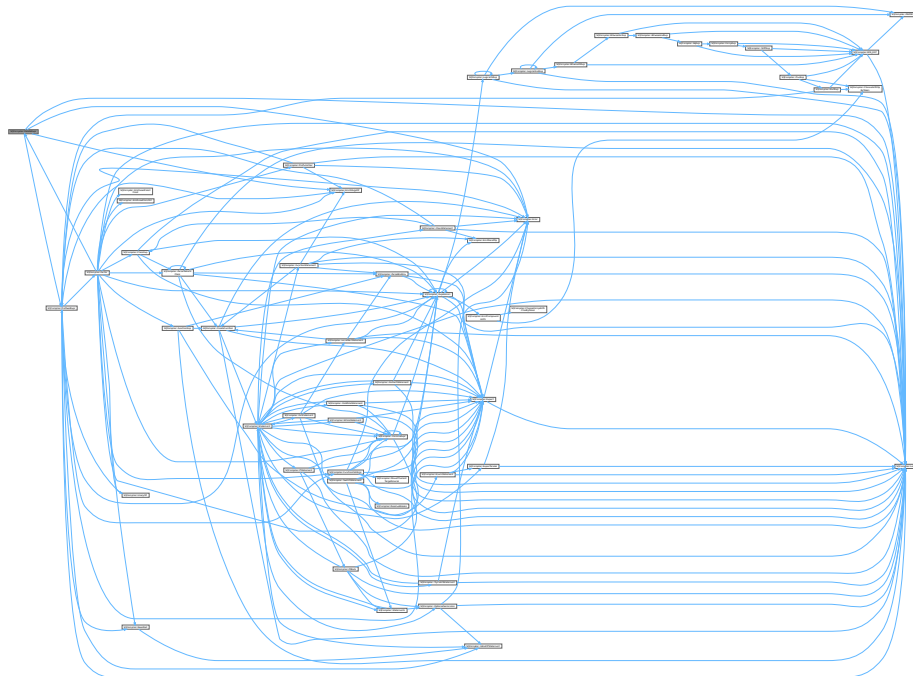
Definition at line 1851 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
01852     {
01853         SQExpState es;
01854         Lex();
01855         es = _es;
01856         _es.donot_get = true;
01857         PrefixedExpr();
01858         if(_es.etype==EXPR) Error(_SC("can't delete an expression"));
01859         if(_es.etype==OBJECT || _es.etype==BASE) {
01860             Emit2ArgsOP(_OP_DELETE);
01861         }
01862         else {
01863             Error(_SC("cannot delete an (outer) local"));
01864         }
01865         _es = es;
01866     }
```

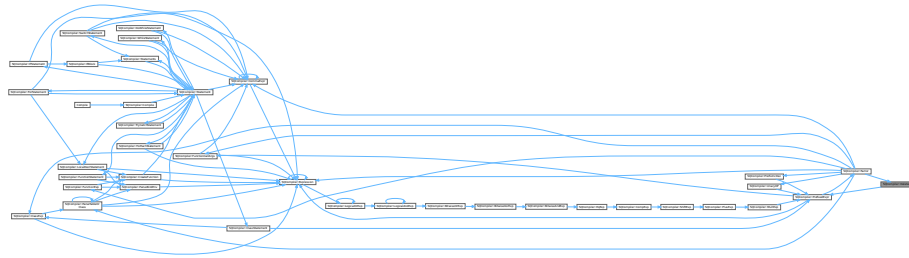
References [_es](#), [BASE](#), [SQExpState::donot_get](#), [Emit2ArgsOP\(\)](#), [Error\(\)](#), [SQExpState::etype](#), [EXPR](#), [Lex\(\)](#), [OBJECT](#), and [PrefixedExpr\(\)](#).

Referenced by [Factor\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.14 DoWhileStatement()

```
void SQCompiler::DoWhileStatement ( ) [inline]
```

do-while ループを解析します。

最初にループ本体を実行し、その後に条件を評価してループの継続を判断します。break/continue にも対応します。

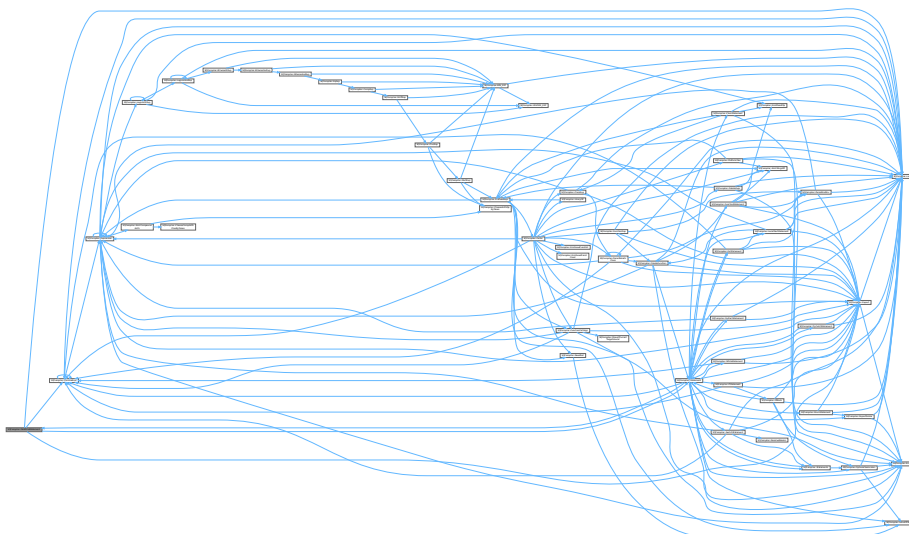
Definition at line 1446 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
01447 {
01448     Lex();
01449     SQInteger jmptrg = _fs->GetCurrentPos();
01450     BEGIN_BREAKABLE_BLOCK()
01451     BEGIN_SCOPE();
01452     Statement();
01453     END_SCOPE();
01454     Expect(TK_WHILE);
01455     SQInteger continuetr = _fs->GetCurrentPos();
01456     Expect(_SC('(')); CommaExpr(); Expect(_SC(')'));
01457     _fs->AddInstruction(_OP_JZ, _fs->PopTarget(), 1);
01458     _fs->AddInstruction(_OP_JMP, 0, jmptrg - _fs->GetCurrentPos() - 1);
01459     END_BREAKABLE_BLOCK(continuetr);
01460 }
```

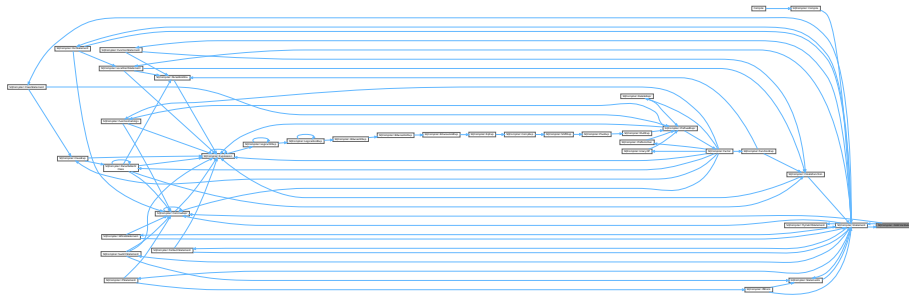
References [_fs](#), [BEGIN_BREAKABLE_BLOCK](#), [BEGIN_SCOPE](#), [CommaExpr\(\)](#), [END_BREAKABLE_BLOCK](#), [END_SCOPE](#), [Expect\(\)](#), [Lex\(\)](#), and [Statement\(\)](#).

Referenced by [Statement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.15 Emit2ArgsOP()

```
void SQCompiler::Emit2ArgsOP (
    SQOpcode op,
    SQInteger p3 = 0 ) [inline]
```

2つの引数を取るオペコードを生成する。

スタックから2つの引数 (p1, p2) をポップし、指定されたオペコードを持つ命令を追加します。

Parameters

<i>op</i>	生成する命令のオペコード。
<i>p3</i>	命令の第3引数 (オプション)。

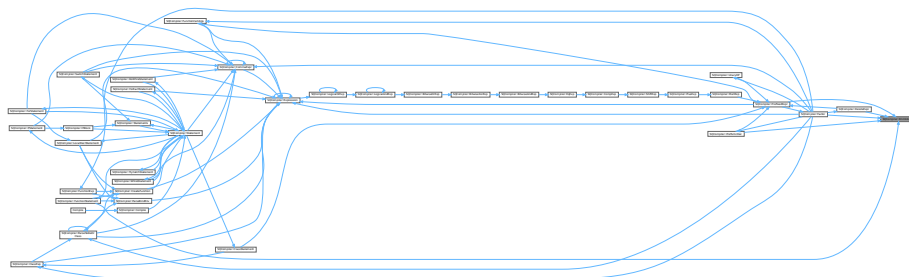
Definition at line 461 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00462 {
00463     SQInteger p2 = _fs->PopTarget(); //src in OP_GET
00464     SQInteger p1 = _fs->PopTarget(); //key in OP_GET
00465     _fs->AddInstruction(op, _fs->PushTarget(), p1, p2, p3);
00466 }
```

References [_fs](#).

Referenced by [DeleteExpr\(\)](#), [Factor\(\)](#), [FunctionStatement\(\)](#), [PrefixedExpr\(\)](#), and [PrefixIncDec\(\)](#).

Here is the caller graph for this function:



3.1.3.16 EmitCompoundArith()

```
void SQCompiler::EmitCompoundArith (
    SQInteger tok,
    SQInteger etype,
    SQInteger pos ) [inline]
```

複合代入演算子 (+, -, など) のコードを生成する。

左辺値の式タイプ（ローカル、オブジェクト、アウター）に応じて、適切な一連の命令を生成します。

Parameters

<i>tok</i>	複合代入演算子のトークン。
<i>etype</i>	左辺値の式タイプ。
<i>pos</i>	ローカルまたはアウター変数の場合の位置。

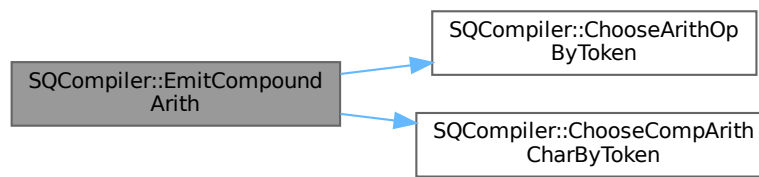
Definition at line 474 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00475     {
00476         /* Generate code depending on the expression type */
00477         switch(etype) {
00478             case LOCAL:{
00479                 SQInteger p2 = _fs->PopTarget(); //src in OP_GET
00480                 SQInteger p1 = _fs->PopTarget(); //key in OP_GET
00481                 _fs->PushTarget(p1);
00482                 //EmitCompArithLocal(tok, p1, p1, p2);
00483                 _fs->AddInstruction(ChooseArithOpByToken(tok), p1, p2, p1, 0);
00484                 _fs->SnoozeOpt();
00485             }
00486             break;
00487             case OBJECT:
00488             case BASE:
00489                 {
00490                     SQInteger val = _fs->PopTarget();
00491                     SQInteger key = _fs->PopTarget();
00492                     SQInteger src = _fs->PopTarget();
00493                     /* _OP_COMPARITH mixes dest obj and source val in the arg1 */
00494                     _fs->AddInstruction(_OP_COMPARITH, _fs->PushTarget(), (src<<16)|val, key,
ChooseCompArithCharByToken(tok));
00495                 }
00496             break;
00497             case OUTER:
00498                 {
00499                     SQInteger val = _fs->TopTarget();
00500                     SQInteger tmp = _fs->PushTarget();
00501                     _fs->AddInstruction(_OP_GETOUTER, tmp, pos);
00502                     _fs->AddInstruction(ChooseArithOpByToken(tok), tmp, val, tmp, 0);
00503                     _fs->PopTarget();
00504                     _fs->PopTarget();
00505                     _fs->AddInstruction(_OP_SETOUTER, _fs->PushTarget(), pos, tmp);
00506                 }
00507             break;
00508         }
00509     }
```

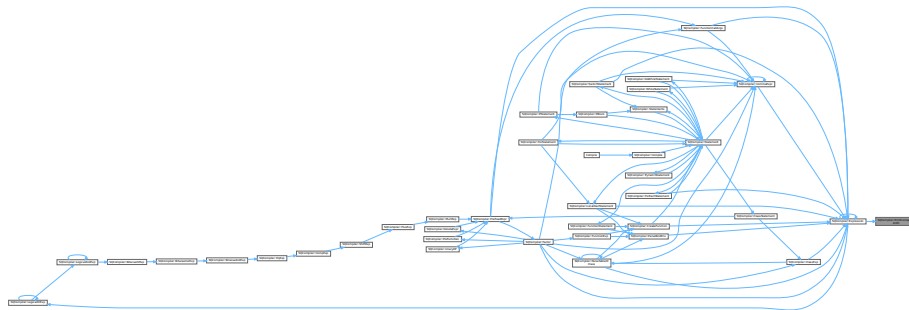
References [_fs](#), [BASE](#), [ChooseArithOpByToken\(\)](#), [ChooseCompArithCharByToken\(\)](#), [LOCAL](#), [OBJECT](#), and [OUTER](#).

Referenced by [Expression\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.17 EmitDerefOp()

```
void SQCompiler::EmitDerefOp (
    SQOpcode op ) [inline]
```

デリファレンス操作のオペコードを生成する。

スタックから値、キー、ソースオブジェクトを取得し、指定されたオペコード (`_OP_NEWSLOT`, `_OP_SET` など) を持つ命令を追加します。

Parameters

<i>op</i>	生成する命令のオペコード。
-----------	---------------

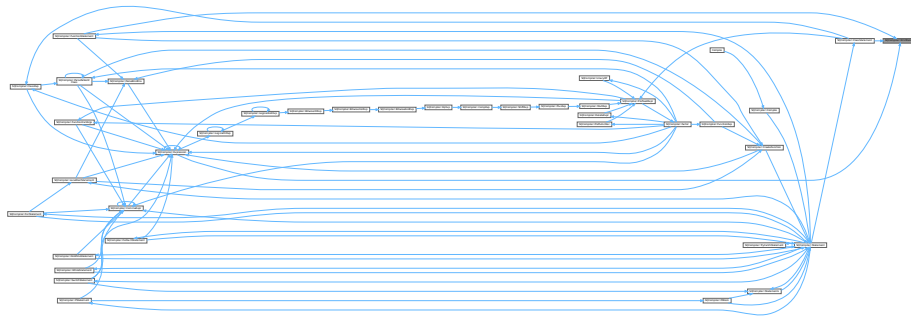
Definition at line 448 of file `squirrel3_squirrel_sqcompiler_v1.cpp`.

```
00449     {
00450         SQInteger val = _fs->PopTarget();
00451         SQInteger key = _fs->PopTarget();
00452         SQInteger src = _fs->PopTarget();
00453         _fs->AddInstruction(op, _fs->PushTarget(), src, key, val);
00454     }
```

References [_fs](#).

Referenced by [ClassStatement\(\)](#), [Expression\(\)](#), and [FunctionStatement\(\)](#).

Here is the caller graph for this function:



3.1.3.18 EmitLoadConstFloat()

```
void SQCompiler::EmitLoadConstFloat (
    SQFloat value,
    SQInteger target ) [inline]
```

浮動小数点数定数をロードする命令を生成します。

SQFloat が 32 ビットの場合は、_OP_LOADFLOAT 命令を使用します。それ以外の場合は、定数テーブル経由で _OP_LOAD 命令を使用します。

Parameters

<i>value</i>	ロードする浮動小数点数の値。
<i>target</i>	格納先のスタック位置。-1 の場合、新しいターゲットをプッシュします。

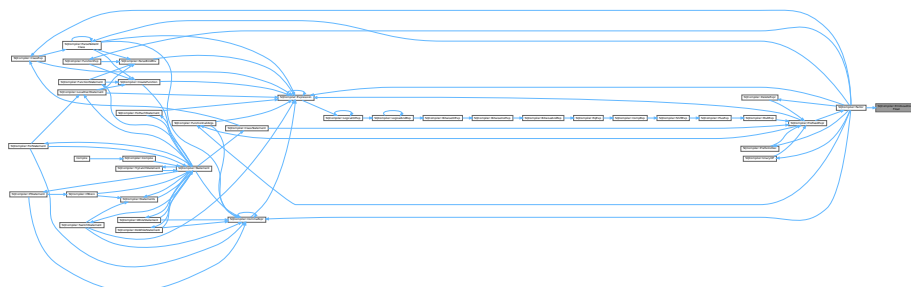
Definition at line 1135 of file `squirrel3_squirrel_sqcompiler_v1.cpp`.

```
01136     {
01137         if(target < 0) {
01138             target = _fs->PushTarget();
01139         }
01140         if(sizeof(SQFloat) == sizeof(SQInt32)) {
01141             _fs->AddInstruction(_OP_LOADFLOAT, target, *((SQInt32 *)&value));
01142         }
01143         else {
01144             _fs->AddInstruction(_OP_LOAD, target, _fs->GetNumericConstant(value));
01145         }
01146     }
```

References [_fs](#).

Referenced by [Factor\(\)](#).

Here is the caller graph for this function:



3.1.3.19 EmitLoadConstInt()

```
void SQCompiler::EmitLoadConstInt (
    SQInteger value,
    SQInteger target ) [inline]
```

整数定数をロードする命令を生成します。

値が 32 ビットに収まる場合は、より効率的な `_OP_LOADINT` 命令を使用します。収まらない場合は、定数テーブル経由で `_OP_LOAD` 命令を使用します。

Parameters

<i>value</i>	ロードする整数の値。
<i>target</i>	格納先のスタック位置。-1 の場合、新しいターゲットをプッシュします。

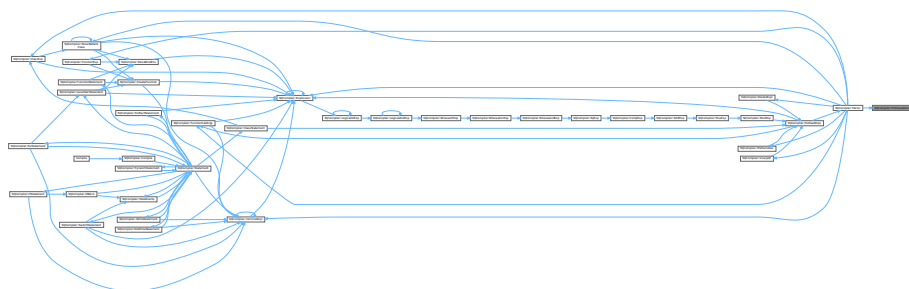
Definition at line 1116 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
01117 {
01118     if(target < 0) {
01119         target = _fs->PushTarget();
01120     }
01121     if(value <= INT_MAX && value > INT_MIN) { //does it fit in 32 bits?
01122         _fs->AddInstruction(_OP_LOADINT, target,value);
01123     }
01124     else {
01125         _fs->AddInstruction(_OP_LOAD, target, _fs->GetNumericConstant(value));
01126     }
01127 }
```

References [_fs](#).

Referenced by [Factor\(\)](#).

Here is the caller graph for this function:



3.1.3.20 EnumStatement()

```
void SQCompiler::EnumStatement ( ) [inline]
```

enum (列挙型) の宣言を解析します。

enum ブロック内の識別子と値を解析し、定数テーブルに新しいテーブルとして登録します。値が指定されない場合は、0 から始まる連番が自動的に割り当てられます。

Definition at line 1724 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
01725 {
```

```

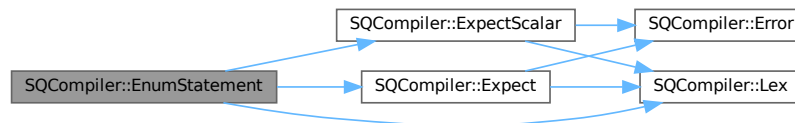
01726     Lex();
01727     SQObject id = Expect(TK_IDENTIFIER);
01728     Expect(_SC('{'));
01729
01730     SQObject table = _fs->CreateTable();
01731     SQInteger nval = 0;
01732     while(_token != _SC('}')) {
01733         SQObject key = Expect(TK_IDENTIFIER);
01734         SQObject val;
01735         if(_token == _SC('=')) {
01736             Lex();
01737             val = ExpectScalar();
01738         }
01739         else {
01740             val._type = OT_INTEGER;
01741             val._unVal.nInteger = nval++;
01742         }
01743         _table(table)->NewSlot(SQObjectPtr(key), SQObjectPtr(val));
01744         if(_token == ',') Lex();
01745     }
01746     SQTable *enums = _table(_ss(_vm)->_consts);
01747     SQObjectPtr strongid = id;
01748     enums->NewSlot(SQObjectPtr(strongid), SQObjectPtr(table));
01749     strongid.Null();
01750     Lex();
01751 }

```

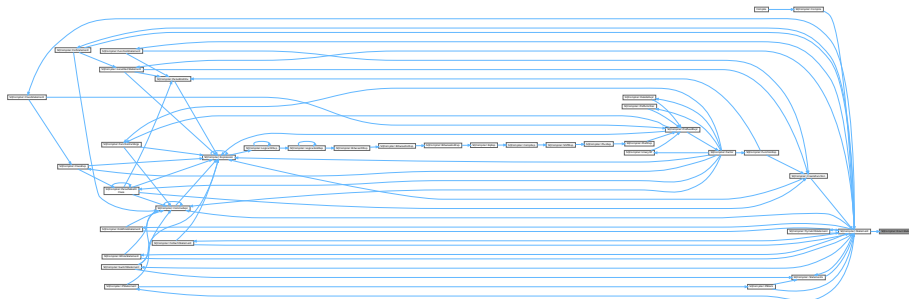
References [_fs](#), [_token](#), [_vm](#), [Expect\(\)](#), [ExpectScalar\(\)](#), and [Lex\(\)](#).

Referenced by [Statement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.21 EqExp()

```
void SQCompiler::EqExp ( ) [inline]
```

等価比較 (==, !=, <=>) 式を解析します。

BIN_EXP ヘルパーを利用して、等価比較の命令を生成します。

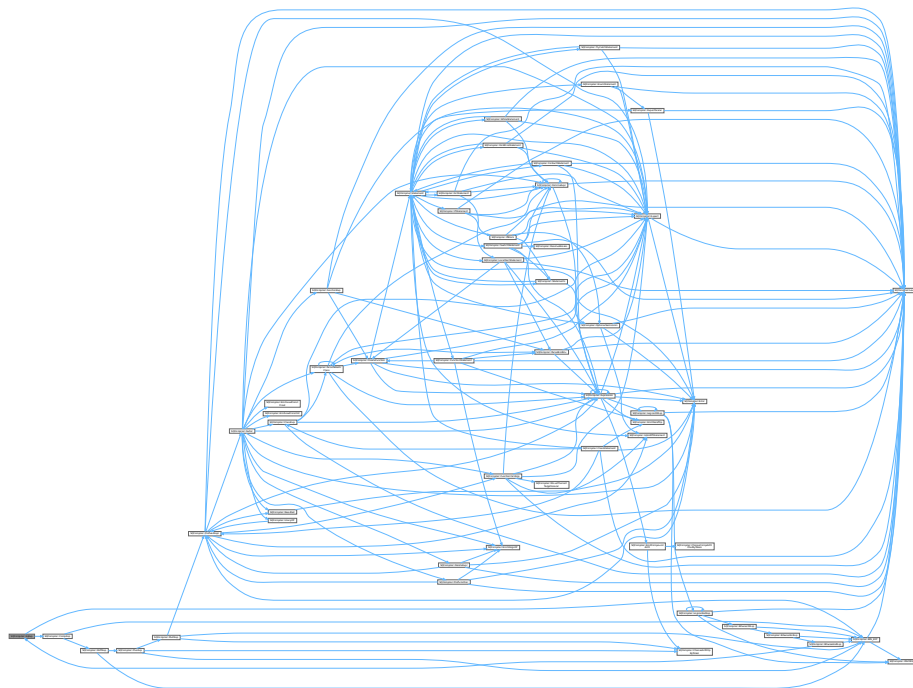
Definition at line 728 of file squirrel3_squirrel_sqcompiler_v1.cpp.

```
00729 {
00730     CompExp();
00731     for(;;) switch(_token) {
00732     case TK_EQ: BIN_EXP(_OP_EQ, &SQCompiler::CompExp); break;
00733     case TK_NE: BIN_EXP(_OP_NE, &SQCompiler::CompExp); break;
00734     case TK_3WAYSCMP: BIN_EXP(_OP_CMP, &SQCompiler::CompExp, CMP_3W); break;
00735     default: return;
00736     }
00737 }
```

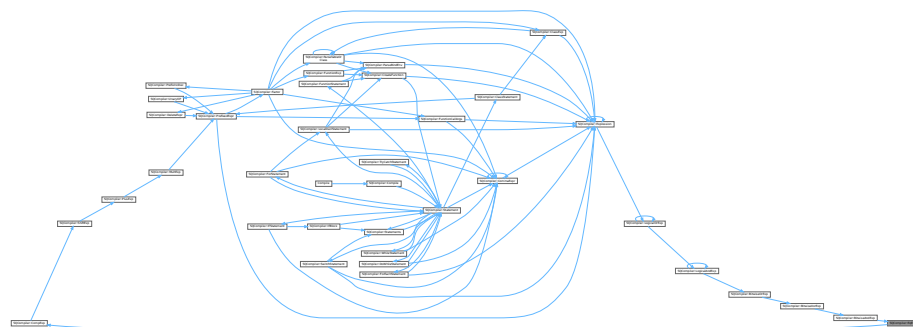
References [_token](#), [BIN_EXP\(\)](#), and [CompExp\(\)](#).

Referenced by [BitwiseAndExp\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.22 Error()

```
void SQCompiler::Error (
    const SQChar * s,
    ... ) [inline]
```

コンパイルエラーを報告し、ロングジャンプでコンパイル処理を中断する。

書式指定文字列と可変長引数を受け取り、エラーメッセージを構築します。その後、`setjmp` で設定されたエラーハンドリング箇所へジャンプします。

Parameters

<code>s</code>	書式指定文字列。
<code>...</code>	書式指定文字列に対応する可変長引数。

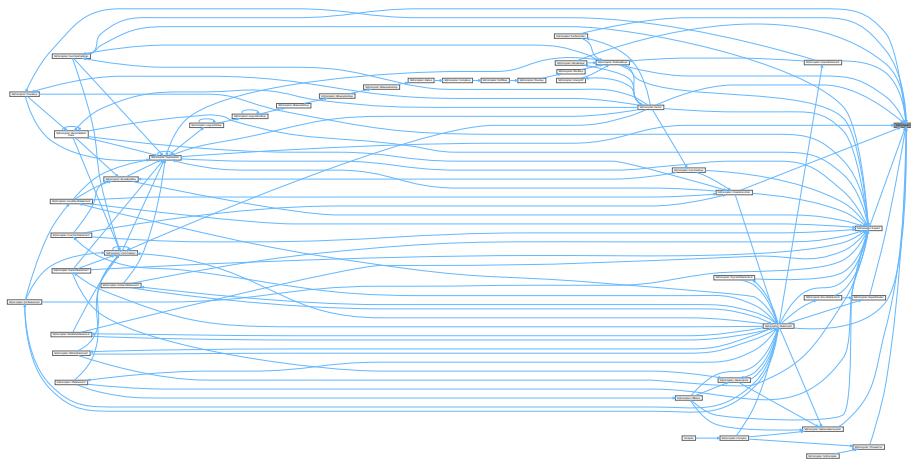
Definition at line 171 of file `squirrel3_squirrel_sqcompiler_v1.cpp`.

```
00172     {
00173         va_list vl;
00174         va_start(vl, s);
00175         scvsprintf(_compilererror, MAX_COMPILER_ERROR_LEN, s, vl);
00176         va_end(vl);
00177         longjmp(_errorjmp,1);
00178     }
```

References `_compilererror`, `_errorjmp`, and `MAX_COMPILER_ERROR_LEN`.

Referenced by `ClassStatement()`, `CreateFunction()`, `DeleteExpr()`, `Expect()`, `ExpectScalar()`, `Expression()`, `Factor()`, `FunctionCallArgs()`, `OptionalSemicolon()`, `PrefixedExpr()`, `PrefixIncDec()`, `Statement()`, and `ThrowError()`.

Here is the caller graph for this function:



3.1.3.23 Expect()

```
SQObject SQCompiler::Expect (
    SQInteger tok ) [inline]
```

特定のトークンを期待し、消費する。

現在のトークンが期待するトークンと一致するか検証します。一致しない場合はコンパイルエラーを発生させます。一致した場合、トークンの種類に応じて `SQObject` を生成して返し、次のトークンに進みます。

Parameters

<i>tok</i>	期待するトークンの種類。
------------	--------------

Returns

トークンから生成されたSQObject。

Definition at line 192 of file `squirrel3_squirrel_sqcompiler_v1.cpp`.

```

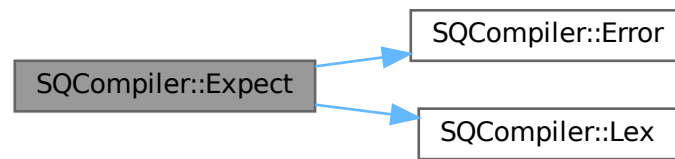
00193     {
00194
00195         if(_token != tok) {
00196             if(_token == TK_CONSTRUCTOR && tok == TK_IDENTIFIER) {
00197                 //do nothing
00198             }
00199             else {
00200                 const SQChar *etypename;
00201                 if(tok > 255) {
00202                     switch(tok)
00203                     {
00204                         case TK_IDENTIFIER:
00205                             etypename = _SC("IDENTIFIER");
00206                             break;
00207                         case TK_STRING_LITERAL:
00208                             etypename = _SC("STRING_LITERAL");
00209                             break;
00210                         case TK_INTEGER:
00211                             etypename = _SC("INTEGER");
00212                             break;
00213                         case TK_FLOAT:
00214                             etypename = _SC("FLOAT");
00215                             break;
00216                         default:
00217                             etypename = _lex.Tok2Str(tok);
00218                     }
00219                     Error(_SC("expected '%s'", etypename);
00220                 }
00221                 Error(_SC("expected '%c'", tok);
00222             }
00223         }
00224         SQObjectPtr ret;
00225         switch(tok)
00226         {
00227             case TK_IDENTIFIER:
00228                 ret = _fs->CreateString(_lex._svalue);
00229                 break;
00230             case TK_STRING_LITERAL:
00231                 ret = _fs->CreateString(_lex._svalue, _lex._longstr.size()-1);
00232                 break;
00233             case TK_INTEGER:
00234                 ret = SQObjectPtr(_lex._nvalue);
00235                 break;
00236             case TK_FLOAT:
00237                 ret = SQObjectPtr(_lex._fvalue);
00238                 break;
00239         }
00240         Lex();
00241         return ret;
00242     }

```

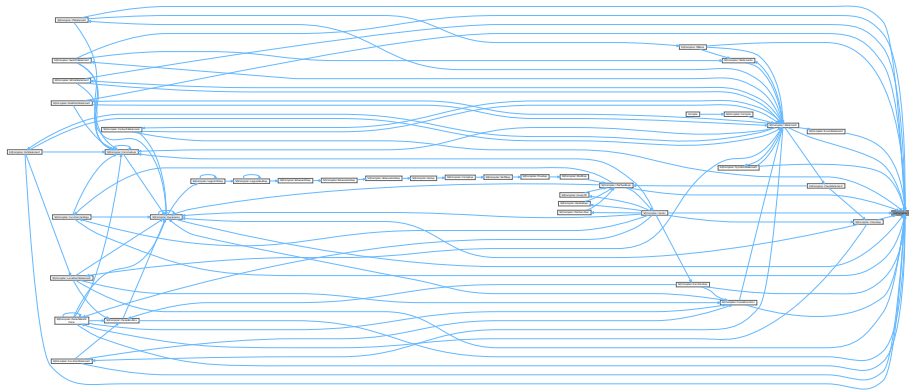
References [_fs](#), [_lex](#), [_token](#), [Error\(\)](#), and [Lex\(\)](#).

Referenced by [ClassExp\(\)](#), [CreateFunction\(\)](#), [DoWhileStatement\(\)](#), [EnumStatement\(\)](#), [Expression\(\)](#), [Factor\(\)](#), [ForEachStatement\(\)](#), [ForStatement\(\)](#), [FunctionCallArgs\(\)](#), [FunctionExp\(\)](#), [FunctionStatement\(\)](#), [IfBlock\(\)](#), [IfStatement\(\)](#), [LocalDeclStatement\(\)](#), [ParseBindEnv\(\)](#), [ParseTableOrClass\(\)](#), [PrefixedExpr\(\)](#), [Statement\(\)](#), [SwitchStatement\(\)](#), [TryCatchStatement\(\)](#), and [WhileStatement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.24 ExpectScalar()

```
SQObject SQCompiler::ExpectScalar ( ) [inline]
```

スカラー値 (整数、浮動小数点数、文字列、真偽値) を期待し、解析します。

`const` 宣言や `enum` 宣言の右辺値など、コンパイル時に値が確定するスカラー値を解析するために使用されます。

Returns

解析されたスカラー値を格納した `SQObject`。

Definition at line 1676 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```

1677     {
1678         SQObject val;
1679         val._type = OT_NULL; val._unVal.nInteger = 0; //shut up GCC 4.x
1680         switch(_token) {
1681             case TK_INTEGER:
1682                 val._type = OT_INTEGER;
1683                 val._unVal.nInteger = _lex._nvalue;
1684                 break;
1685             case TK_FLOAT:
1686                 val._type = OT_FLOAT;
1687                 val._unVal.fFloat = _lex._fvalue;
1688                 break;
  
```

```

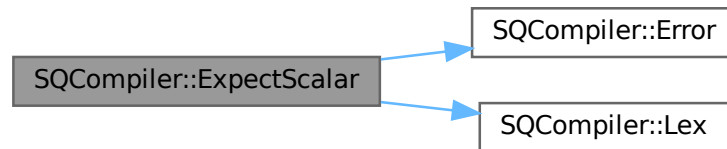
01689         case TK_STRING_LITERAL:
01690             val = _fs->CreateString(_lex._svalue, _lex._longstr.size()-1);
01691             break;
01692         case TK_TRUE:
01693         case TK_FALSE:
01694             val._type = OT_BOOL;
01695             val._unVal.nInteger = _token == TK_TRUE ? 1 : 0;
01696             break;
01697         case '-':
01698             Lex();
01699             switch(_token)
01700             {
01701             case TK_INTEGER:
01702                 val._type = OT_INTEGER;
01703                 val._unVal.nInteger = -_lex._nvalue;
01704                 break;
01705             case TK_FLOAT:
01706                 val._type = OT_FLOAT;
01707                 val._unVal.fFloat = -_lex._fvalue;
01708                 break;
01709             default:
01710                 Error(_SC("scalar expected : integer, float"));
01711             }
01712             break;
01713         default:
01714             Error(_SC("scalar expected : integer, float, or string"));
01715     }
01716     Lex();
01717     return val;
01718 }

```

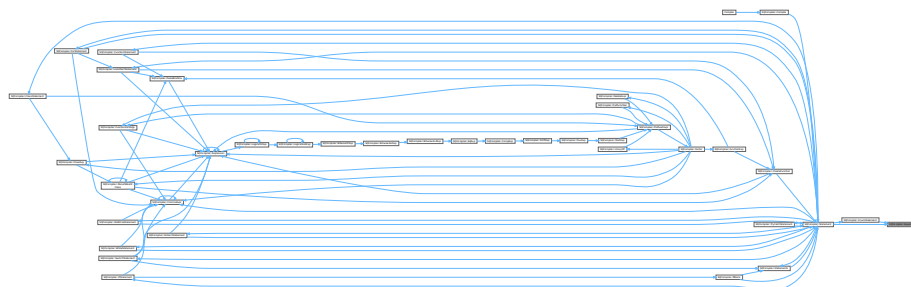
References [_fs](#), [_lex](#), [_token](#), [Error\(\)](#), and [Lex\(\)](#).

Referenced by [EnumStatement\(\)](#), and [Statement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.25 Expression()

```
void SQCompiler::Expression ( ) [inline]
```

式を解析します。

代入演算子 ('=', '+=', '-=' など) や三項演算子 ('?:') を含む、最も低い優先順位の式を解析します。これは式解析の主要なエントリーポイントです。内部で LogicalOrExp を呼び出し、より高い優先順位の式の解析を開始します。

Definition at line 524 of file squirrel3_squirrel_sqcompiler_v1.cpp.

```
00525     {
00526         SQExpState es = _es;
00527         _es.etype      = EXPR;
00528         _es.epos       = -1;
00529         _es.donot_get  = false;
00530         LogicalOrExp();
00531         switch(_token) {
00532             case _SC('='):
00533             case TK_NEWSLOT:
00534             case TK_MINUSEQ:
00535             case TK_PLUSEQ:
00536             case TK_MULEQ:
00537             case TK_DIVEQ:
00538             case TK_MODEQ:{
00539                 SQInteger op = _token;
00540                 SQInteger ds = _es.etype;
00541                 SQInteger pos = _es.epos;
00542                 if(ds == EXPR) Error(_SC("can't assign expression"));
00543                 else if(ds == BASE) Error(_SC("'base' cannot be modified"));
00544                 Lex(); Expression();
00545
00546                 switch(op){
00547                     case TK_NEWSLOT:
00548                         if(ds == OBJECT || ds == BASE)
00549                             EmitDerefOp(_OP_NEWSLOT);
00550                         else //if _derefstate != Deref_NO_DEREF && Deref_FIELD so is the index of a local
00551                             Error(_SC("can't 'create' a local slot"));
00552                         break;
00553                     case _SC('='): //ASSIGN
00554                         switch(ds) {
00555                             case LOCAL:
00556                                 {
00557                                     SQInteger src = _fs->PopTarget();
00558                                     SQInteger dst = _fs->TopTarget();
00559                                     _fs->AddInstruction(_OP_MOVE, dst, src);
00560                                 }
00561                                 break;
00562                             case OBJECT:
00563                             case BASE:
00564                                 EmitDerefOp(_OP_SET);
00565                                 break;
00566                             case OUTER:
00567                                 {
00568                                     SQInteger src = _fs->PopTarget();
00569                                     SQInteger dst = _fs->PushTarget();
00570                                     _fs->AddInstruction(_OP_SETOUTER, dst, pos, src);
00571                                 }
00572                                 break;
00573                             case TK_MINUSEQ:
00574                             case TK_PLUSEQ:
00575                             case TK_MULEQ:
00576                             case TK_DIVEQ:
00577                             case TK_MODEQ:
00578                                 EmitCompoundArith(op, ds, pos);
00579                                 break;
00580                             }
00581                         }
00582                     break;
00583                 case _SC('?:'): {
00584                     Lex();
00585                     _fs->AddInstruction(_OP_JZ, _fs->PopTarget());
00586                     SQInteger jzpos = _fs->GetCurrentPos();
00587                     SQInteger trg = _fs->PushTarget();
00588                     Expression();
00589                     SQInteger first_exp = _fs->PopTarget();
00590                     if(trg != first_exp) _fs->AddInstruction(_OP_MOVE, trg, first_exp);
00591                     SQInteger endfirstexp = _fs->GetCurrentPos();
00592                     _fs->AddInstruction(_OP_JMP, 0, 0);
00593                     Expect(_SC(':'));
00594                 }
```

```

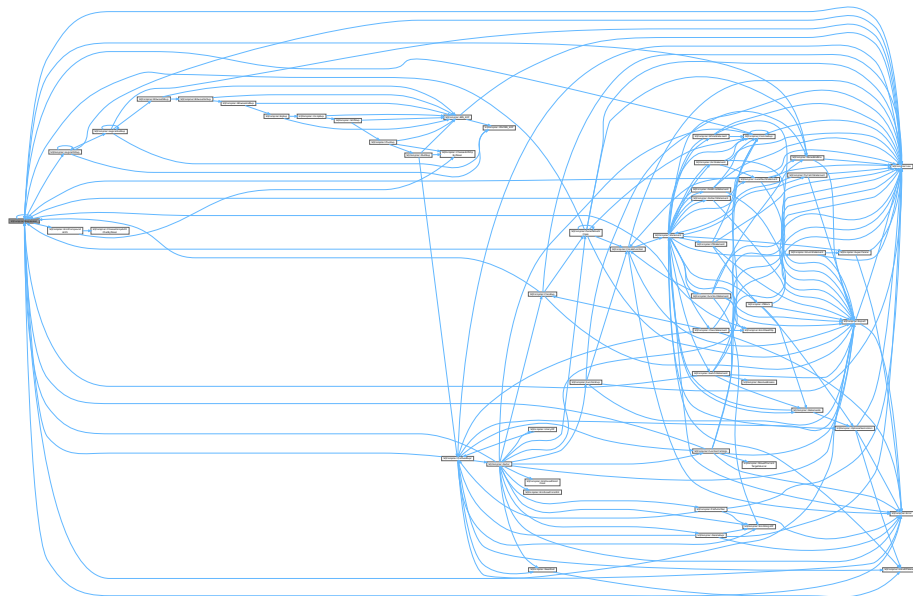
00595         SQInteger jmppos = _fs->GetCurrentPos();
00596         Expression();
00597         SQInteger second_exp = _fs->PopTarget();
00598         if(trg != second_exp) _fs->AddInstruction(_OP_MOVE, trg, second_exp);
00599         _fs->SetInstructionParam(jmppos, 1, _fs->GetCurrentPos() - jmppos);
00600         _fs->SetInstructionParam(jzpos, 1, endfirstexp - jzpos + 1);
00601         _fs->SnoozeOpt();
00602     }
00603     break;
00604 }
00605 _es = es;
00606 }

```

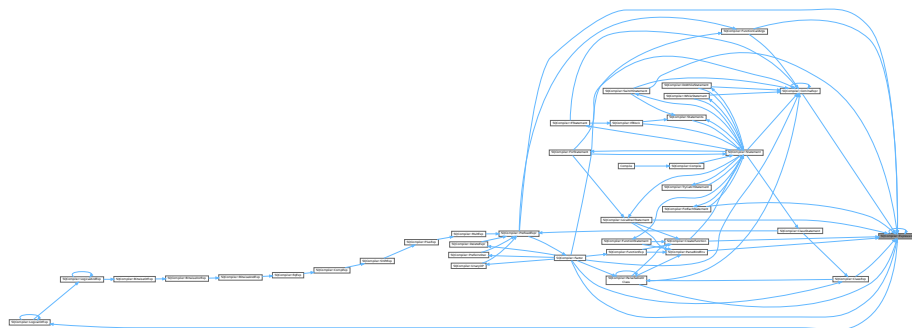
References [_es](#), [_fs](#), [_token](#), [BASE](#), [SQExpState::donot_get](#), [EmitCompoundArith\(\)](#), [EmitDerefOp\(\)](#), [SQExpState::epos](#), [Error\(\)](#), [SQExpState::etype](#), [Expect\(\)](#), [EXPR](#), [Expression\(\)](#), [Lex\(\)](#), [LOCAL](#), [LogicalOrExp\(\)](#), [OBJECT](#), and [OUTER](#).

Referenced by [ClassExp\(\)](#), [CommaExpr\(\)](#), [CreateFunction\(\)](#), [Expression\(\)](#), [Factor\(\)](#), [ForEachStatement\(\)](#), [FunctionCallArgs\(\)](#), [LocalDeclStatement\(\)](#), [ParseBindEnv\(\)](#), [ParseTableOrClass\(\)](#), [PrefixedExpr\(\)](#), and [SwitchStatement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.26 Factor()

SQInteger SQCompiler::Factor () [inline]

式の構成要素（因子）を解析します。

演算子の優先順位の最下層に位置する要素を解析します。これには、数値や文字列リテラル、変数識別子、this、base、null、テーブル/配列リテラル、括弧で囲まれた式、および単項演算子（-, !, ~, typeof 等）が含まれます。

Returns

解析された変数がローカル変数の場合、そのスタック位置。それ以外の場合は-1。

Definition at line 945 of file squirrel3_squirrel_sqcompiler_v1.cpp.

```

00946     {
00947         // _es.etype = EXPR;
00948         switch(_token)
00949         {
00950             case TK_STRING_LITERAL:
00951                 _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
00952                 _fs->GetConstant(_fs->CreateString(_lex._svalue, _lex._longstr.size()-1));
00953                 Lex();
00954                 break;
00955             case TK_BASE:
00956                 Lex();
00957                 _fs->AddInstruction(_OP_GETBASE, _fs->PushTarget());
00958                 _es.etype = BASE;
00959                 _es.epos = _fs->TopTarget();
00960                 return _es.epos;
00961             case TK_IDENTIFIER:
00962             case TK_CONSTRUCTOR:
00963             case TK_THIS: {
00964                 SQObject id;
00965                 SQObject constant;
00966
00967                 switch(_token) {
00968                     case TK_IDENTIFIER: id = _fs->CreateString(_lex._svalue); break;
00969                     case TK_THIS: id = _fs->CreateString(_SC("this"), 4); break;
00970                     case TK_CONSTRUCTOR: id = _fs->CreateString(_SC("constructor"), 11); break;
00971                 }
00972
00973                 SQInteger pos = -1;
00974                 Lex();
00975                 if((pos = _fs->GetLocalVariable(id)) != -1) {
00976                     /* Handle a local variable (includes 'this') */
00977                     _fs->PushTarget(pos);
00978                     _es.etype = LOCAL;
00979                     _es.epos = pos;
00980                 }
00981
00982                 else if((pos = _fs->GetOuterVariable(id)) != -1) {
00983                     /* Handle a free var */
00984                     if(NeedGet()) {
00985                         _es.epos = _fs->PushTarget();
00986                         _fs->AddInstruction(_OP_GETOUTER, _es.epos, pos);
00987                         /* _es.etype = EXPR; already default value */
00988                     }
00989                     else {
00990                         _es.etype = OUTER;
00991                         _es.epos = pos;
00992                     }
00993                 }
00994
00995                 else if(_fs->IsConstant(id, constant)) {
00996                     /* Handle named constant */
00997                     SQObjectPtr constval;
00998                     SQObject constid;
00999                     if(sq_type(constant) == OT_TABLE) {
01000                         Expect(' ');
01001                         constid = Expect(TK_IDENTIFIER);
01002                         if(!_table(constant)->Get(constid, constval)) {
01003                             constval.Null();
01004                             Error(_SC("invalid constant [%s.%s]"), _stringval(id),
01005                             _stringval(constid));
01006                         }
01007                     }

```

```

01007         else {
01008             constval = constant;
01009         }
01010         _es.epos = _fs->PushTarget();
01011
01012         /* generate direct or literal function depending on size */
01013         SQObjectType ctype = sq_type(constval);
01014         switch(ctype) {
01015             case OT_INTEGER: EmitLoadConstInt(_integer(constval), _es.epos); break;
01016             case OT_FLOAT:   EmitLoadConstFloat(_float(constval), _es.epos); break;
01017             case OT_BOOL:    _fs->AddInstruction(_OP_LOADBOOL, _es.epos, _integer(constval));
01018             break;
01019             default: _fs->AddInstruction(_OP_LOAD, _es.epos, _fs->GetConstant(constval));
01020             break;
01021         }
01022         _es.etype = EXPR;
01023     }
01024     else {
01025         /* Handle a non-local variable, aka a field. Push the 'this' pointer on
01026         * the virtual stack (always found in offset 0, so no instruction needs to
01027         * be generated), and push the key next. Generate an _OP_LOAD instruction
01028         * for the latter. If we are not using the variable as a dref expr, generate
01029         * the _OP_GET instruction.
01030         */
01031         _fs->PushTarget(0);
01032         _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(), _fs->GetConstant(id));
01033         if(NeedGet()) {
01034             Emit2ArgsOP(_OP_GET);
01035         }
01036         _es.etype = OBJECT;
01037     }
01038     return _es.epos;
01039 }
01040 break;
01041 case TK_DOUBLE_COLON: // "::"
01042     _fs->AddInstruction(_OP_LOADROOT, _fs->PushTarget());
01043     _es.etype = OBJECT;
01044     _token = _SC('.'); /* hack: drop into PrefixExpr, case '.' */
01045     _es.epos = -1;
01046     return _es.epos;
01047 break;
01048 case TK_NULL:
01049     _fs->AddInstruction(_OP_LOADNULLS, _fs->PushTarget(), 1);
01050     Lex();
01051     break;
01052 case TK_INTEGER: EmitLoadConstInt(_lex._nvalue, -1); Lex(); break;
01053 case TK_FLOAT:   EmitLoadConstFloat(_lex._fvalue, -1); Lex(); break;
01054 case TK_TRUE:   case TK_FALSE:
01055     _fs->AddInstruction(_OP_LOADBOOL, _fs->PushTarget(), _token == TK_TRUE?1:0);
01056     Lex();
01057     break;
01058 case _SC('{'): {
01059     _fs->AddInstruction(_OP_NEWOBJ, _fs->PushTarget(), 0, 0, NOT_ARRAY);
01060     SQInteger apos = _fs->GetCurrentPos(), key = 0;
01061     Lex();
01062     while(_token != _SC('}')) {
01063         Expression();
01064         if(_token == _SC(',')) Lex();
01065         SQInteger val = _fs->PopTarget();
01066         SQInteger array = _fs->TopTarget();
01067         _fs->AddInstruction(_OP_APPENDARRAY, array, val, AAT_STACK);
01068         key++;
01069     }
01070     _fs->SetInstructionParam(apos, 1, key);
01071     Lex();
01072 }
01073 break;
01074 case _SC('('):
01075     _fs->AddInstruction(_OP_NEWOBJ, _fs->PushTarget(), 0, 0, NOT_TABLE);
01076     Lex(); ParseTableOrClass(_SC('('), _SC(')'));
01077     break;
01078 case TK_FUNCTION: FunctionExp(); break;
01079 case _SC('@'): FunctionExp(true); break;
01080 case TK_CLASS: Lex(); ClassExp(); break;
01081 case _SC('~'):
01082     Lex();
01083     switch(_token) {
01084         case TK_INTEGER: EmitLoadConstInt(~_lex._nvalue, -1); Lex(); break;
01085         case TK_FLOAT:   EmitLoadConstFloat(~_lex._fvalue, -1); Lex(); break;
01086         default: UnaryOP(_OP_NEG);
01087     }
01088     break;
01089 case _SC('!'): Lex(); UnaryOP(_OP_NOT); break;
01090 case _SC('~'):
01091     Lex();
01092     if(_token == TK_INTEGER) { EmitLoadConstInt(~_lex._nvalue, -1); Lex(); break; }
01093     UnaryOP(_OP_BWNOT);

```

```

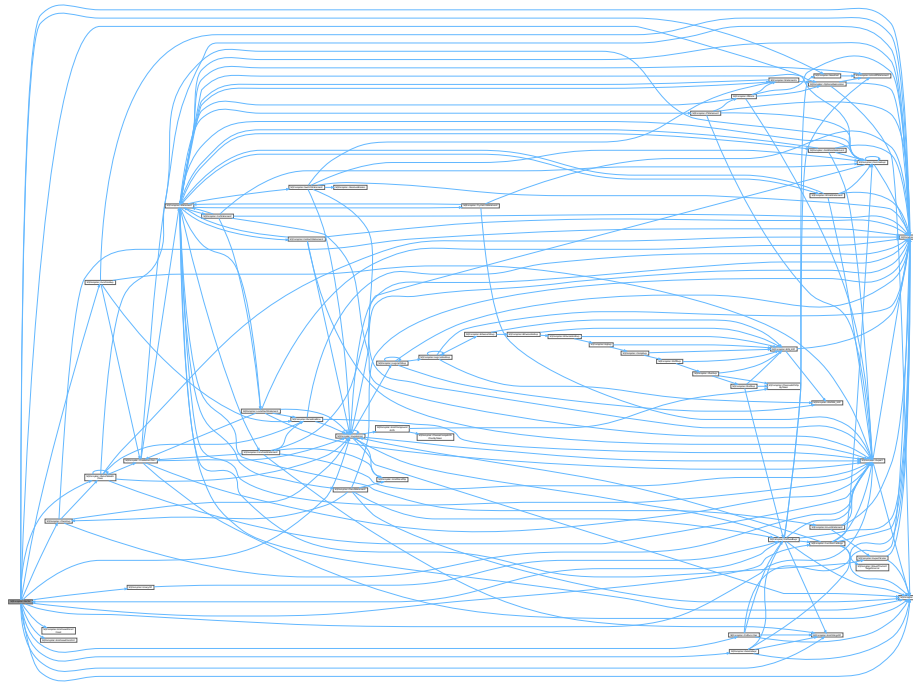
01092         break;
01093     case TK_TYPEOF : Lex() ;UnaryOP(_OP_TYPEOF); break;
01094     case TK_RESUME : Lex() ; UnaryOP(_OP_RESUME); break;
01095     case TK_CLONE : Lex() ; UnaryOP(_OP_CLONE); break;
01096     case TK_RAWCALL: Lex() ; Expect('('); FunctionCallArgs(true); break;
01097     case TK_MINUSMINUS :
01098     case TK_PLUSPLUS :PrefixIncDec(_token); break;
01099     case TK_DELETE : DeleteExpr(); break;
01100     case _SC('('): Lex() ; CommaExpr(); Expect(_SC(')'));
01101         break;
01102     case TK__LINE__ : EmitLoadConstInt(_lex._currentline,-1); Lex(); break;
01103     case TK__FILE__ : _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
01104         _fs->GetConstant(_sourcename)); Lex(); break;
01105     default: Error(_SC("expression expected"));
01106     }
01107     _es.etype = EXPR;
01108     return -1;
01109 }

```

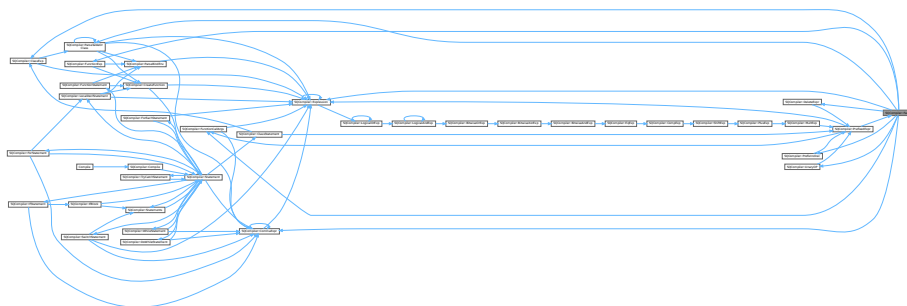
References [_es](#), [_fs](#), [_lex](#), [_sourcename](#), [_token](#), [BASE](#), [ClassExp\(\)](#), [CommaExpr\(\)](#), [DeleteExpr\(\)](#), [Emit2ArgsOP\(\)](#), [EmitLoadConstFloat\(\)](#), [EmitLoadConstInt\(\)](#), [SQExpState::epos](#), [Error\(\)](#), [SQExpState::etype](#), [Expect\(\)](#), [EXPR](#), [Expression\(\)](#), [FunctionCallArgs\(\)](#), [FunctionExp\(\)](#), [Lex\(\)](#), [LOCAL](#), [NeedGet\(\)](#), [OBJECT](#), [OUTER](#), [ParseTableOrClass\(\)](#), [PrefixIncDec\(\)](#), and [UnaryOP\(\)](#).

Referenced by [PrefixedExpr\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.27 ForEachStatement()

```
void SQCompiler::ForEachStatement ( ) [inline]
```

foreach ループ (foreach(val, key in container)) を解析します。

イテレータ用の内部ローカル変数を確保し、_OP_FOREACH 命令と_OP_POSTFOREACH 命令を使ってイテレーションを実装します。

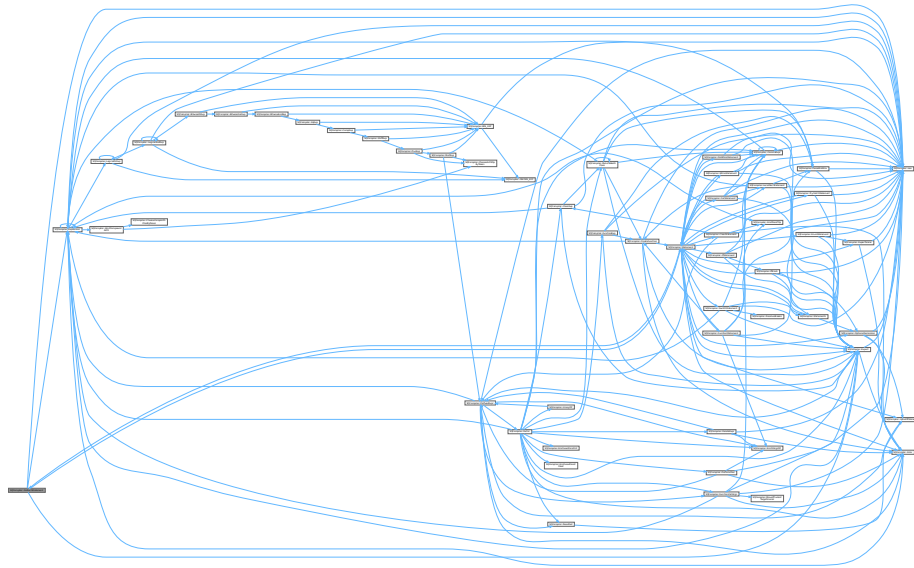
Definition at line 1516 of file squirrel3_squirrel_sqcompiler_v1.cpp.

```
01517 {
01518     SQObject idxname, valname;
01519     Lex(); Expect(_SC('(')); valname = Expect(TK_IDENTIFIER);
01520     if(_token == _SC(',',')) {
01521         idxname = valname;
01522         Lex(); valname = Expect(TK_IDENTIFIER);
01523     }
01524     else{
01525         idxname = _fs->CreateString(_SC("@INDEX@"));
01526     }
01527     Expect(TK_IN);
01528
01529     //save the stack size
01530     BEGIN_SCOPE();
01531     //put the table in the stack(evaluate the table expression)
01532     Expression(); Expect(_SC('('));
01533     SQInteger container = _fs->TopTarget();
01534     //push the index local var
01535     SQInteger indexpos = _fs->PushLocalVariable(idxname);
01536     _fs->AddInstruction(_OP_LOADNULLS, indexpos,1);
01537     //push the value local var
01538     SQInteger valuepos = _fs->PushLocalVariable(valname);
01539     _fs->AddInstruction(_OP_LOADNULLS, valuepos,1);
01540     //push reference index
01541     SQInteger itrpos = _fs->PushLocalVariable(_fs->CreateString(_SC("@ITERATOR@"))); //use invalid
    id to make it inaccessible
01542     _fs->AddInstruction(_OP_LOADNULLS, itrpos,1);
01543     SQInteger jmppos = _fs->GetCurrentPos();
01544     _fs->AddInstruction(_OP_FOREACH, container, 0, indexpos);
01545     SQInteger foreachpos = _fs->GetCurrentPos();
01546     _fs->AddInstruction(_OP_POSTFOREACH, container, 0, indexpos);
01547     //generate the statement code
01548     BEGIN_BREAKBLE_BLOCK()
01549     Statement();
01550     _fs->AddInstruction(_OP_JMP, 0, jmppos - _fs->GetCurrentPos() - 1);
01551     _fs->SetInstructionParam(foreachpos, 1, _fs->GetCurrentPos() - foreachpos);
01552     _fs->SetInstructionParam(foreachpos + 1, 1, _fs->GetCurrentPos() - foreachpos);
01553     END_BREAKBLE_BLOCK(foreachpos - 1);
01554     //restore the local variable stack(remove index,val and ref idx)
01555     _fs->PopTarget();
01556     END_SCOPE();
01557 }
```

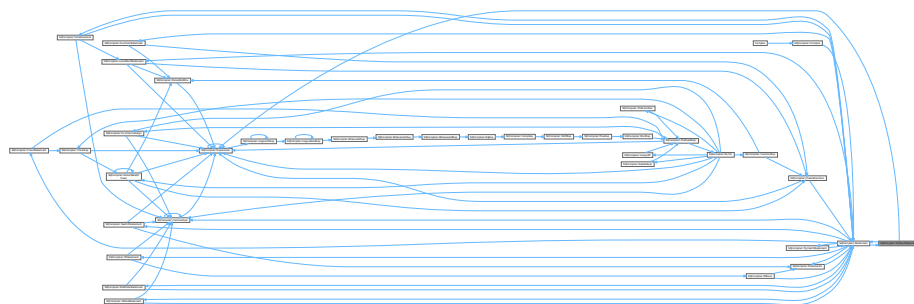
References [_fs](#), [_token](#), [BEGIN_BREAKBLE_BLOCK](#), [BEGIN_SCOPE](#), [END_BREAKBLE_BLOCK](#), [END_SCOPE](#), [Expect\(\)](#), [Expression\(\)](#), [Lex\(\)](#), and [Statement\(\)](#).

Referenced by [Statement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.28 ForStatement()

```
void SQCompiler::ForStatement ( ) [inline]
```

C スタイルの for ループ (for (init; cond; inc)) を解析します。

初期化、条件、更新の各部分を解析し、ループ構造を JMP 命令と JZ 命令で構築します。更新部分はループ本体の実行後に評価されるよう、命令列を一時的に退避・再配置します。

Definition at line 1466 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
01467 {
01468     Lex();
01469     BEGIN_SCOPE();
01470     Expect(_SC('('));
01471     if(_token == TK_LOCAL) LocalDeclStatement();
01472     else if(_token != _SC(';')){
01473         CommaExpr();
01474         _fs->PopTarget();
01475     }
01476     Expect(_SC(';'));
01477     _fs->SnoozeOpt();
01478     SQInteger jmppos = _fs->GetCurrentPos();
01479     SQInteger jzpos = -1;
```

```

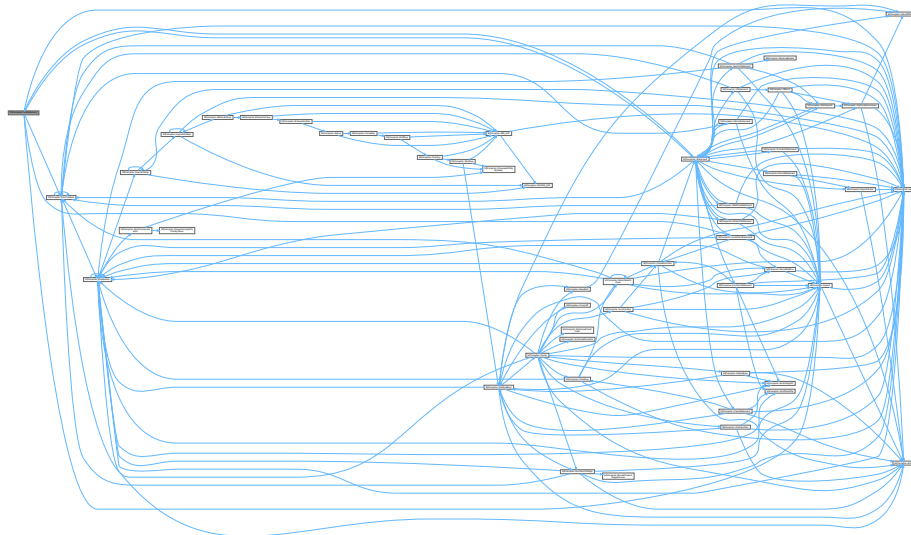
01480         if(_token != _SC(';')) { CommaExpr(); _fs->AddInstruction(_OP_JZ, _fs->PopTarget()); jzpos =
_fs->GetCurrentPos(); }
01481         Expect(_SC(';'));
01482         _fs->SnoozeOpt();
01483         SQInteger expstart = _fs->GetCurrentPos() + 1;
01484         if(_token != _SC('')) {
01485             CommaExpr();
01486             _fs->PopTarget();
01487         }
01488         Expect(_SC(''));
01489         _fs->SnoozeOpt();
01490         SQInteger expend = _fs->GetCurrentPos();
01491         SQInteger expsize = (expend - expstart) + 1;
01492         SQInstructionVec exp;
01493         if(expsize > 0) {
01494             for(SQInteger i = 0; i < expsize; i++)
01495                 exp.push_back(_fs->GetInstruction(expstart + i));
01496             _fs->PopInstructions(expsize);
01497         }
01498         BEGIN_BREAKBLE_BLOCK()
01499         Statement();
01500         SQInteger continuetr = _fs->GetCurrentPos();
01501         if(expsize > 0) {
01502             for(SQInteger i = 0; i < expsize; i++)
01503                 _fs->AddInstruction(exp[i]);
01504         }
01505         _fs->AddInstruction(_OP_JMP, 0, jmppos - _fs->GetCurrentPos() - 1, 0);
01506         if(jzpos > 0) _fs->SetInstructionParam(jzpos, 1, _fs->GetCurrentPos() - jzpos);
01507
01508         END_BREAKBLE_BLOCK(continuetr);
01509
01510         END_SCOPE();
01511     }

```

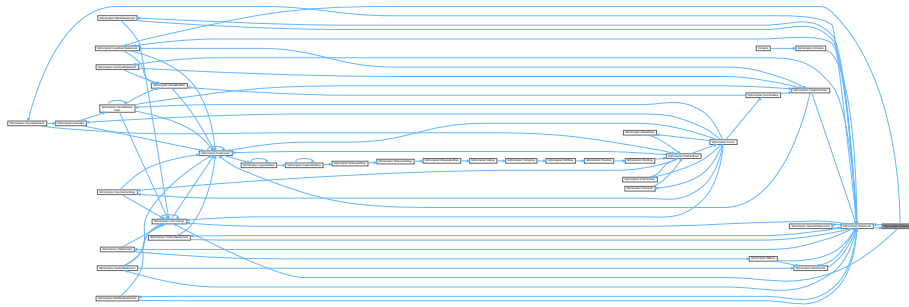
References [_fs](#), [_token](#), [BEGIN_BREAKBLE_BLOCK](#), [BEGIN_SCOPE](#), [CommaExpr\(\)](#), [END_BREAKBLE_BLOCK](#), [END_SCOPE](#), [Expect\(\)](#), [Lex\(\)](#), [LocalDeclStatement\(\)](#), and [Statement\(\)](#).

Referenced by [Statement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.29 FunctionCallArgs()

```
void SQCompiler::FunctionCallArgs (
    bool rawcall = false ) [inline]
```

関数呼び出しの引数リストを解析します。

() 内のカンマ区切りの引数式を一つずつ評価し、_OP_CALL 命令を生成します。rawcall の場合は特別な引数処理を行います。また、{}が続く場合は、関数呼び出しの結果のテーブルに対してスロットを設定する構文を処理します。

Parameters

<i>rawcall</i>	rawcall キーワードによる呼び出しかどうかを示すフラグ。
----------------	---------------------------------

Definition at line 1186 of file squirrel3_squirrel_sqcompiler_v1.cpp.

```
01187     {
01188         SQInteger nargs = 1; //this
01189         while(_token != _SC('')) {
01190             Expression();
01191             MoveIfCurrentTargetIsLocal();
01192             nargs++;
01193             if(_token == _SC(',')) {
01194                 Lex();
01195                 if(_token == ')') Error(_SC("expression expected, found ')')"));
01196             }
01197         }
01198         Lex();
01199         if (rawcall) {
01200             if (nargs < 3) Error(_SC("rawcall requires at least 2 parameters (callee and this)"));
01201             nargs -= 2; //removes callee and this from count
01202         }
01203         for(SQInteger i = 0; i < (nargs - 1); i++) _fs->PopTarget();
01204         SQInteger stackbase = _fs->PopTarget();
01205         SQInteger closure = _fs->PopTarget();
01206         _fs->AddInstruction(_OP_CALL, _fs->PushTarget(), closure, stackbase, nargs);
01207         if (_token == '{')
01208         {
01209             SQInteger retval = _fs->TopTarget();
01210             SQInteger nkeys = 0;
01211             Lex();
01212             while (_token != '}') {
01213                 switch (_token) {
01214                     case _SC('['):
01215                         Lex(); CommaExpr(); Expect(_SC(']'));
01216                         Expect(_SC('=')); Expression();
01217                         break;
01218                     default:
01219                         _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
01220 _fs->GetConstant(Expect(TK_IDENTIFIER)));
01221                         Expect(_SC('=')); Expression();
01222                         break;
```

```

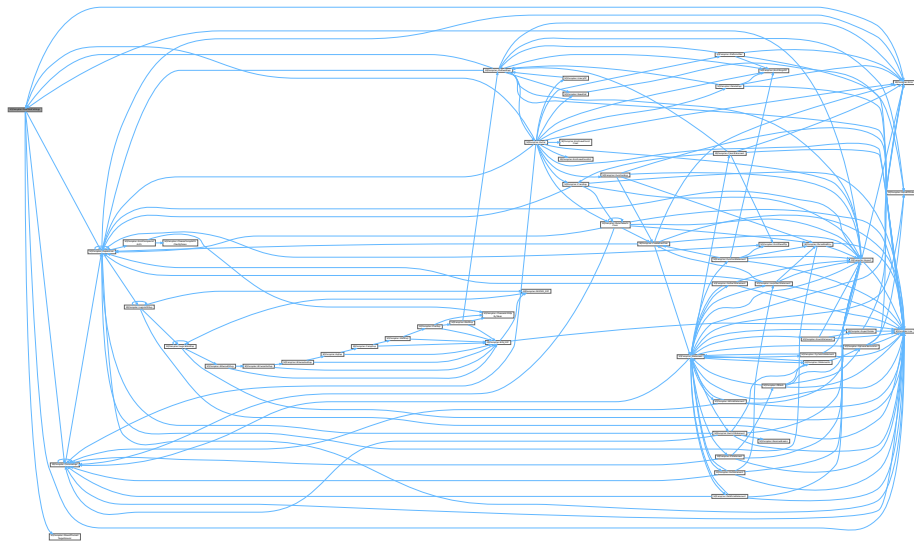
01222         }
01223         if (_token == ',') Lex();
01224         nkeys++;
01225         SQInteger val = _fs->PopTarget();
01226         SQInteger key = _fs->PopTarget();
01227         _fs->AddInstruction(_OP_SET, 0xFF, retval, key, val);
01228     }
01229     Lex();
01230 }
01231 }

```

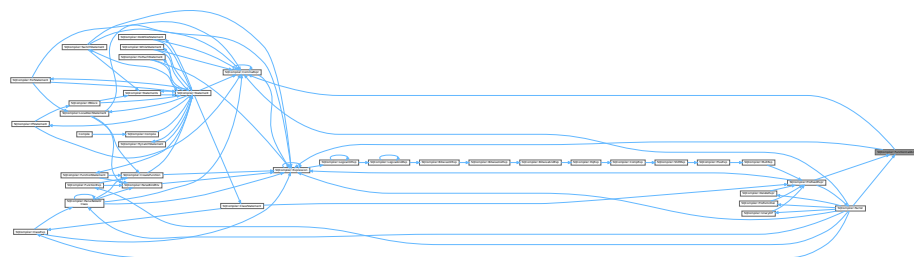
References [_fs](#), [_token](#), [CommaExpr\(\)](#), [Error\(\)](#), [Expect\(\)](#), [Expression\(\)](#), [Lex\(\)](#), and [MovelfCurrentTargetIsLocal\(\)](#).

Referenced by [Factor\(\)](#), and [PrefixedExpr\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.30 FunctionExp()

```

void SQCompiler::FunctionExp (
    bool lambda = false ) [inline]

```

関数式 (function() {...}) またはラムダ式 (@() {...}) を解析します。

無名関数またはラムダ式を解析し、クロージャを生成してスタックにプッシュします。

Parameters

<i>lambda</i>	ラムダ式 (@) かどうかを示すフラグ。
---------------	----------------------

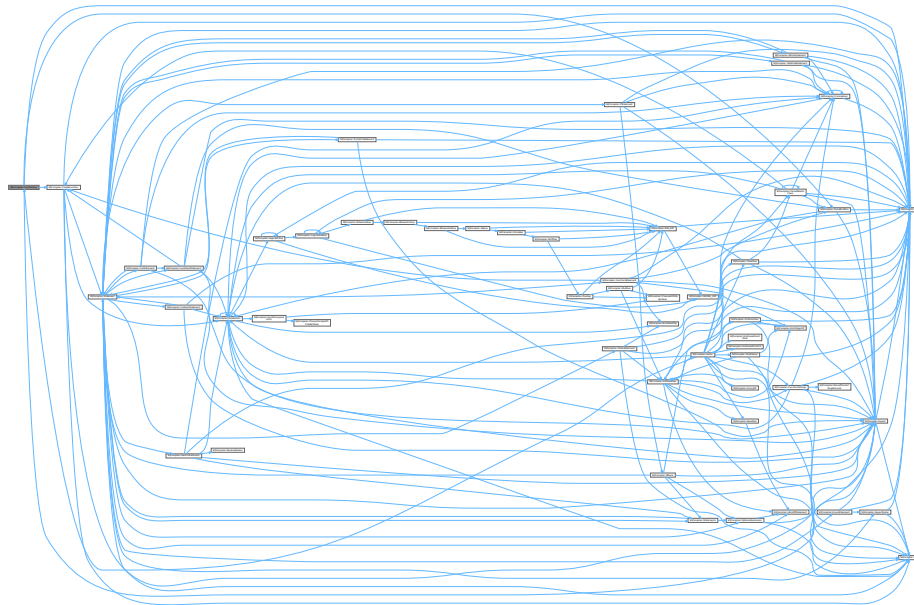
Definition at line 1808 of file squirrel3_squirrel_sqcompiler_v1.cpp.

```
01809 {  
01810     Lex();  
01811     SQInteger boundtarget = 0xFF;  
01812     if (_token == _SC('[')) {  
01813         boundtarget = ParseBindEnv();  
01814     }  
01815     Expect(_SC('('));  
01816     SQObjectPtr dummy;  
01817     CreateFunction(dummy, boundtarget, lambda);  
01818     _fs->AddInstruction(_OP_CLOSURE, _fs->PushTarget(), _fs->_functions.size() - 1, boundtarget);  
01819 }
```

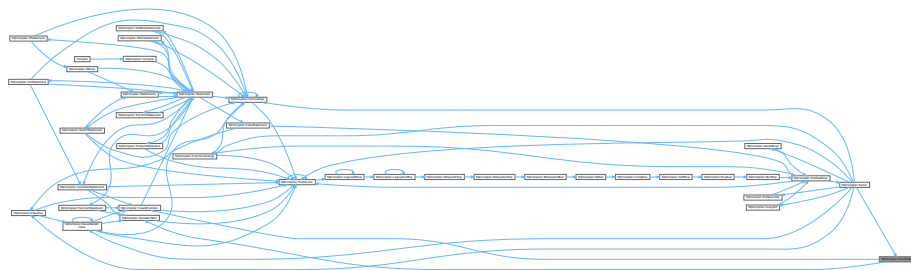
References [_fs](#), [_token](#), [CreateFunction\(\)](#), [Expect\(\)](#), [Lex\(\)](#), and [ParseBindEnv\(\)](#).

Referenced by [Factor\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.31 FunctionStatement()

```
void SQCompiler::FunctionStatement ( ) [inline]
```

function 文による関数宣言を解析します。

function MyFunc() ... や function MyTable.MyFunc() ... といった構文を解析し、関数クロージャを生成して、指定されたスロットに新しいスロットとして設定します。

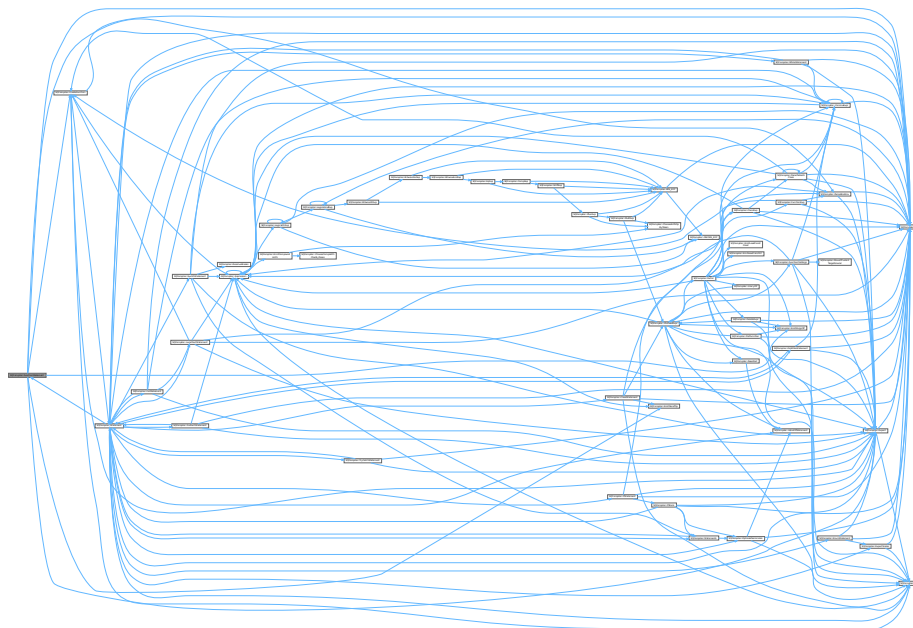
Definition at line 1622 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
01623 {
01624     SQObject id;
01625     Lex(); id = Expect(TK_IDENTIFIER);
01626     _fs->PushTarget(0);
01627     _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(), _fs->GetConstant(id));
01628     if(_token == TK_DOUBLE_COLON) Emit2ArgsOP(_OP_GET);
01629
01630     while(_token == TK_DOUBLE_COLON) {
01631         Lex();
01632         id = Expect(TK_IDENTIFIER);
01633         _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(), _fs->GetConstant(id));
01634         if(_token == TK_DOUBLE_COLON) Emit2ArgsOP(_OP_GET);
01635     }
01636     SQInteger boundtarget = 0xFF;
01637     if (_token == _SC('[')) {
01638         boundtarget = ParseBindEnv();
01639     }
01640     Expect(_SC('('));
01641     CreateFunction(id, boundtarget);
01642     _fs->AddInstruction(_OP_CLOSURE, _fs->PushTarget(), _fs->_functions.size() - 1, boundtarget);
01643     EmitDerefOp(_OP_NEWSLOT);
01644     _fs->PopTarget();
01645 }
```

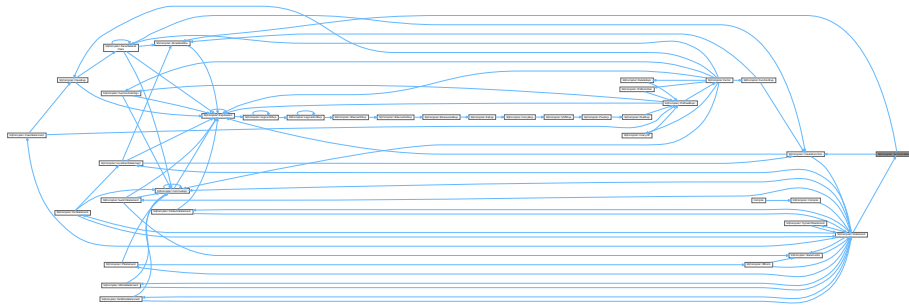
References [_fs](#), [_token](#), [CreateFunction\(\)](#), [Emit2ArgsOP\(\)](#), [EmitDerefOp\(\)](#), [Expect\(\)](#), [Lex\(\)](#), and [ParseBindEnv\(\)](#).

Referenced by [Statement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.32 IfBlock()

```
void SQCompiler::IfBlock ( ) [inline]
```

if 文または else 文の本体ブロックを解析します。

ブロックが{}で囲まれているか、単一の文であるかを判別し、適切に解析処理を呼び出します。

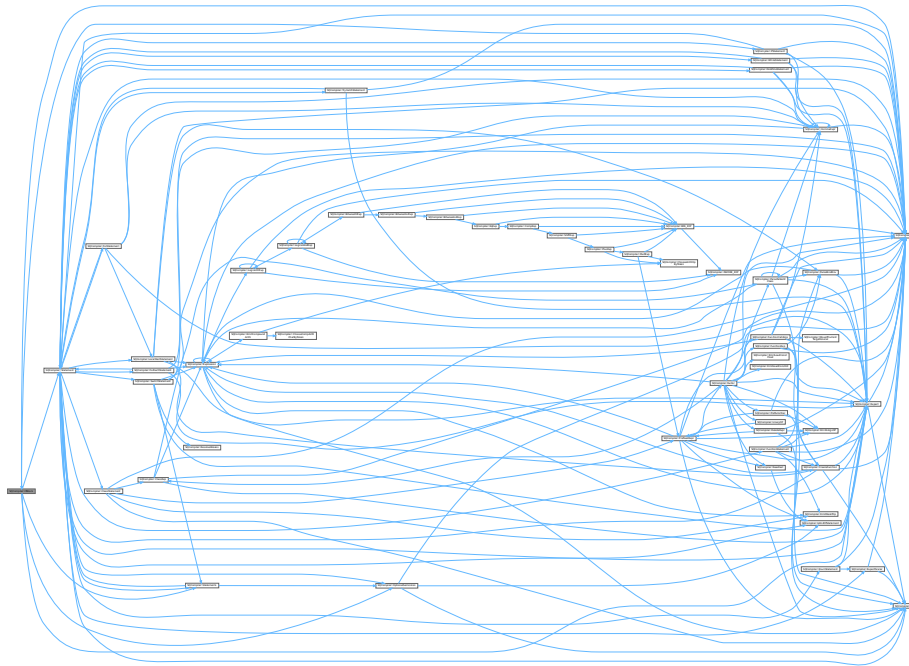
Definition at line 1353 of file squirrel3_squirrel_sqcompiler_v1.cpp.

```
01354     {
01355         if (_token == _SC('{'))
01356         {
01357             BEGIN_SCOPE();
01358             Lex();
01359             Statements();
01360             Expect(_SC('}'));
01361             if (true) {
01362                 END_SCOPE();
01363             }
01364             else {
01365                 END_SCOPE_NO_CLOSE();
01366             }
01367         }
01368         else {
01369             //BEGIN_SCOPE();
01370             Statement();
01371             if (_lex._prevtoken != _SC(';') && _lex._prevtoken != _SC(';')) OptionalSemicolon();
01372             //END_SCOPE();
01373         }
01374     }
```

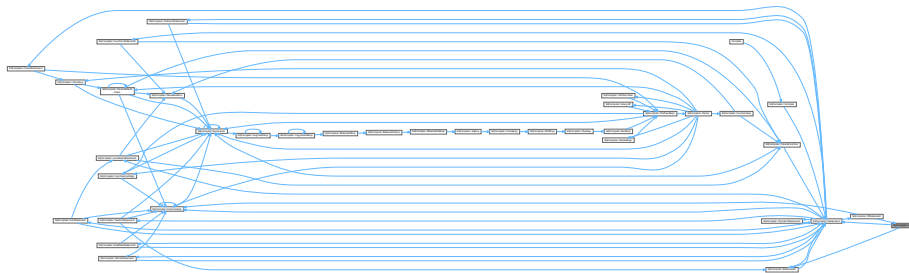
References [_lex](#), [_token](#), [BEGIN_SCOPE](#), [END_SCOPE](#), [END_SCOPE_NO_CLOSE](#), [Expect\(\)](#), [Lex\(\)](#), [OptionalSemicolon\(\)](#), [Statement\(\)](#), and [Statements\(\)](#).

Referenced by [IfStatement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.33 IfStatement()

```
void SQCompiler::IfStatement ( ) [inline]
```

if-else 文を解析します。

条件式を評価し、_OP_JZ (Jump if Zero) 命令を生成して条件が偽の場合のジャンプを設定します。if ブロックと、存在すればelse ブロックを解析し、ジャンプ先アドレスを解決します。

Definition at line 1380 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
01381     {
01382         SQInteger jmppos;
01383         bool haselse = false;
01384         Lex(); Expect(_SC('(')); CommaExpr(); Expect(_SC(')'));
01385         _fs->AddInstruction(_OP_JZ, _fs->PopTarget());
01386         SQInteger jnpos = _fs->GetCurrentPos();
01387
01388
01389
01390         IfBlock();
```

```

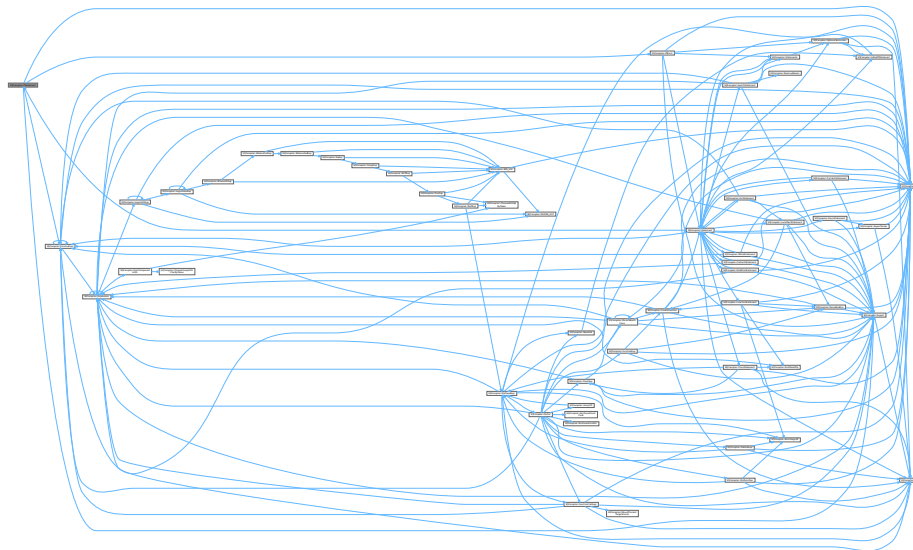
01391         //
01392         /*static int n = 0;
01393         if (_token != _SC('}') && _token != TK_ELSE) {
01394             printf("IF %d-----!!!!!!!!\n", n);
01395             if (n == 5)
01396             {
01397                 printf("asd");
01398             }
01399             n++;
01400             //OptionalSemicolon();
01401         }*/
01402
01403         SQInteger endifblock = _fs->GetCurrentPos();
01404         if (_token == TK_ELSE) {
01405             haselse = true;
01406             //BEGIN_SCOPE();
01407             _fs->AddInstruction(_OP_JMP);
01408             jmppos = _fs->GetCurrentPos();
01409             Lex();
01410             //Statement(); if(_lex._prevtoken != _SC('}')) OptionalSemicolon();
01411             IfBlock();
01412             //END_SCOPE();
01413             _fs->SetInstructionParam(jmppos, 1, _fs->GetCurrentPos() - jmppos);
01414         }
01415         _fs->SetInstructionParam(jnepos, 1, endifblock - jnepos + (haselse?1:0));
01416     }
01417 }

```

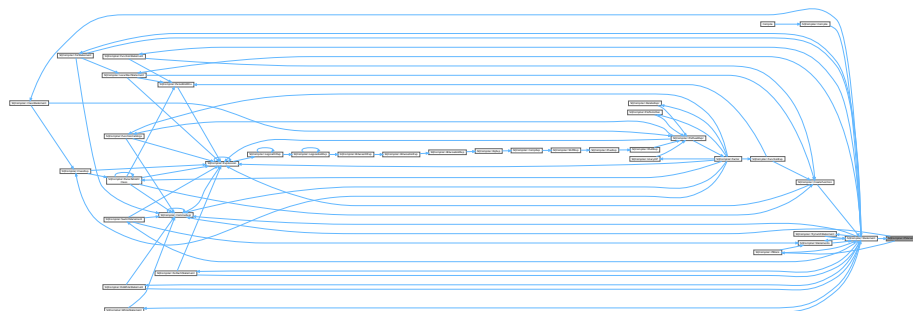
References [_fs](#), [_token](#), [CommaExpr\(\)](#), [Expect\(\)](#), [IfBlock\(\)](#), and [Lex\(\)](#).

Referenced by [Statement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.34 INVOKE_EXP()

```
template<typename T >
void SQCompiler::INVOKE_EXP (
    T f ) [inline]
```

式のパーサー関数を呼び出すためのテンプレートヘルパー。

式の状態 (`_es`) を一時的に保存し、デフォルトの式状態 (`EXPR`) にリセットしてから、引数で指定されたパーサー関数 `f` を実行します。実行後、元の式の状態を復元します。これにより、ネストした式の解析を安全に行うことができます。

Template Parameters

<code>T</code>	呼び出すメンバ関数の型。
----------------	--------------

Parameters

<code>f</code>	呼び出すコンパイラのメンバ関数ポインタ。
----------------	----------------------

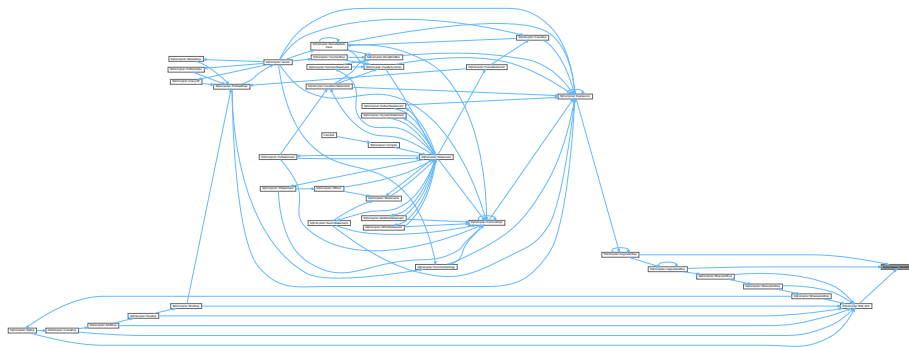
Definition at line 614 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00615     {
00616         SQExpState es = _es;
00617         _es.etype     = EXPR;
00618         _es.epos      = -1;
00619         _es.donot_get = false;
00620         (this->*f) ();
00621         _es = es;
00622     }
```

References [_es](#), [SQExpState::donot_get](#), [SQExpState::epos](#), [SQExpState::etype](#), and [EXPR](#).

Referenced by [BIN_EXP\(\)](#), [LogicalAndExp\(\)](#), and [LogicalOrExp\(\)](#).

Here is the caller graph for this function:



3.1.3.35 IsEndOfStatement()

```
bool SQCompiler::IsEndOfStatement ( ) [inline]
```

現在のトークンが文の終わりを示しているか判定する。

直前のトークンが改行であるか、現在のトークンがスクリプトの終端 (EOB)、`?`、`;` のいずれかである場合に `true` を返します。

Returns

文の終わりであれば true、そうでなければ false。

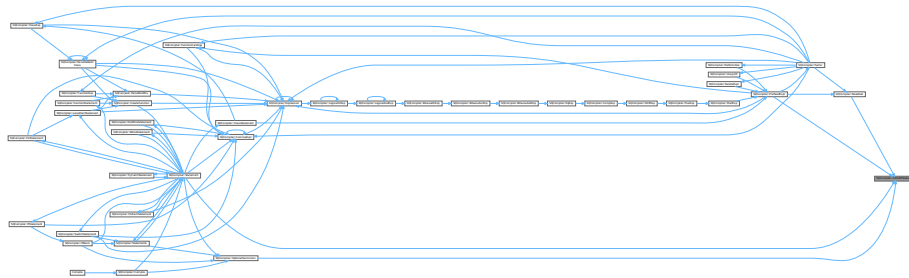
Definition at line 248 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00248 { return ((_lex._prevtoken == _SC('\n')) || (_token == SQUIRREL_EOB) || (_token == _SC('}')) ||
      (_token == _SC(';'))); }
```

References [_lex](#), and [_token](#).

Referenced by [NeedGet\(\)](#), [OptionalSemicolon\(\)](#), [PrefixedExpr\(\)](#), and [Statement\(\)](#).

Here is the caller graph for this function:



3.1.3.36 Lex()

```
void SQCompiler::Lex ( ) [inline]
```

字句解析器を呼び出し、次のトークンを取得する。

[_lex.Lex\(\)](#) を呼び出し、結果を内部メンバ [_token](#) に格納します。

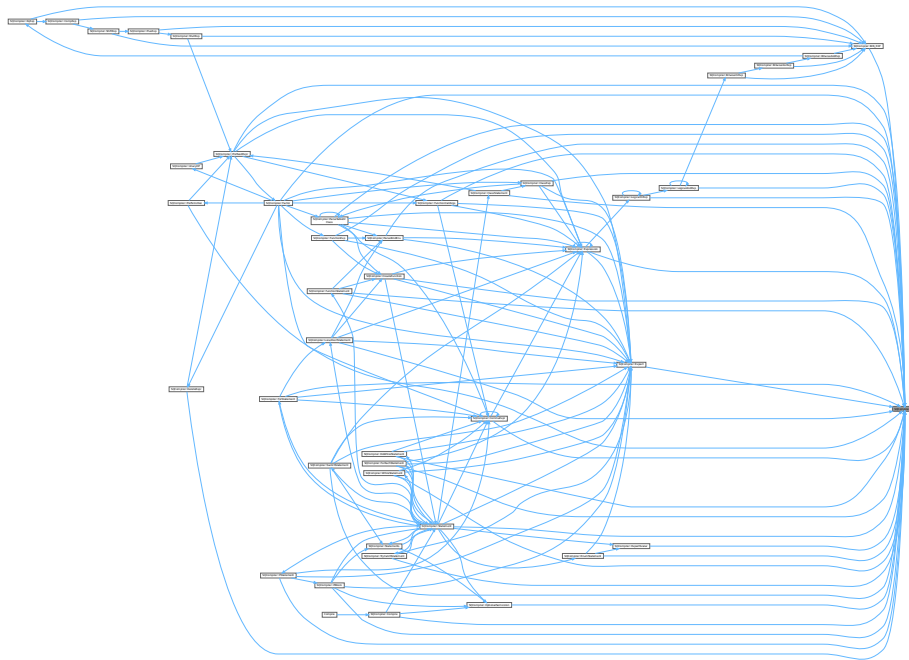
Definition at line 183 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00183 { _token = _lex.Lex(); }
```

References [_lex](#), and [_token](#).

Referenced by [BIN_EXP\(\)](#), [ClassExp\(\)](#), [ClassStatement\(\)](#), [CommaExpr\(\)](#), [Compile\(\)](#), [CreateFunction\(\)](#), [DeleteExpr\(\)](#), [DoWhileStatement\(\)](#), [EnumStatement\(\)](#), [Expect\(\)](#), [ExpectScalar\(\)](#), [Expression\(\)](#), [Factor\(\)](#), [ForEachStatement\(\)](#), [ForStatement\(\)](#), [FunctionCallArgs\(\)](#), [FunctionExp\(\)](#), [FunctionStatement\(\)](#), [IfBlock\(\)](#), [IfStatement\(\)](#), [LocalDeclStatement\(\)](#), [LogicalAndExp\(\)](#), [LogicalOrExp\(\)](#), [OptionalSemicolon\(\)](#), [ParseBindEnv\(\)](#), [ParseTableOrClass\(\)](#), [PrefixedExpr\(\)](#), [PrefixIncDec\(\)](#), [Statement\(\)](#), [SwitchStatement\(\)](#), [TryCatchStatement\(\)](#), and [WhileStatement\(\)](#).

Here is the caller graph for this function:



3.1.3.37 LocalDeclStatement()

```
void SQCompiler::LocalDeclStatement ( ) [inline]
```

ローカル変数またはローカル関数の宣言文を解析します。

local キーワードに続く識別子をローカル変数として登録します。初期化式がある場合はそれを評価します。カンマで複数の変数を一度に宣言することも可能です。local function ... 構文もここで処理されます。

Definition at line 1314 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```

1315     {
1316         SQObject varname;
1317         Lex();
1318         if( _token == TK_FUNCTION) {
1319             SQInteger boundtarget = 0xFF;
1320             Lex();
1321             varname = Expect(TK_IDENTIFIER);
1322             if (_token == _SC('(')) {
1323                 boundtarget = ParseBindEnv();
1324             }
1325             Expect(_SC('('));
1326             CreateFunction(varname,0xFF,false);
1327             _fs->AddInstruction(_OP_CLOSURE, _fs->PushTarget(), _fs->_functions.size() - 1,
boundtarget);
1328             _fs->PopTarget();
1329             _fs->PushLocalVariable(varname);
1330             return;
1331         }
1332
1333         do {
1334             varname = Expect(TK_IDENTIFIER);
1335             if (_token == _SC('=')) {
1336                 Lex(); Expression();
1337                 SQInteger src = _fs->PopTarget();
1338                 SQInteger dest = _fs->PushTarget();
1339                 if(dest != src) _fs->AddInstruction(_OP_MOVE, dest, src);
1340             }
1341             else{
1342                 _fs->AddInstruction(_OP_LOADNULLS, _fs->PushTarget(),1);

```

```

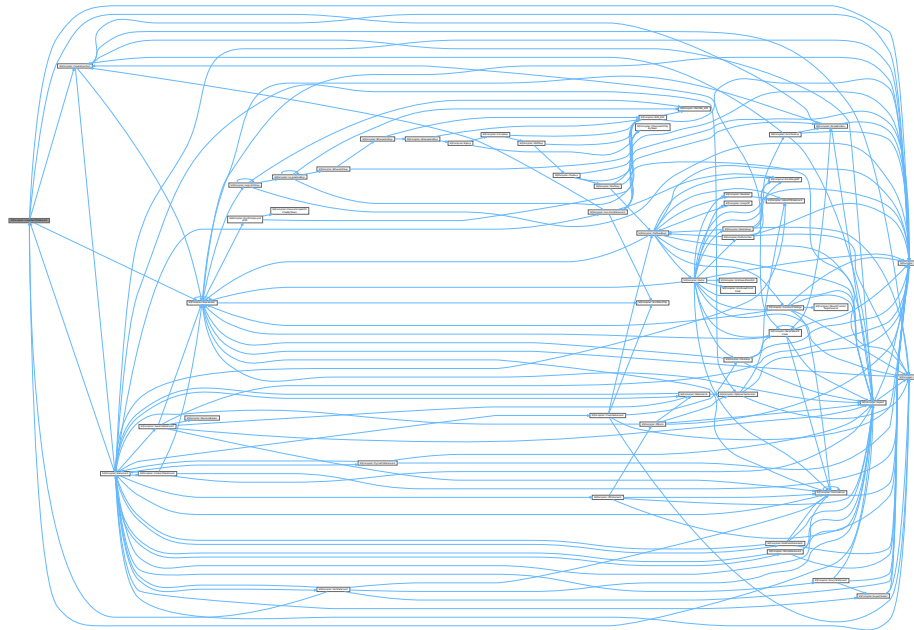
01343         }
01344         _fs->PopTarget();
01345         _fs->PushLocalVariable(varname);
01346         if(_token == _SC(',',')) Lex(); else break;
01347     } while(1);
01348 }

```

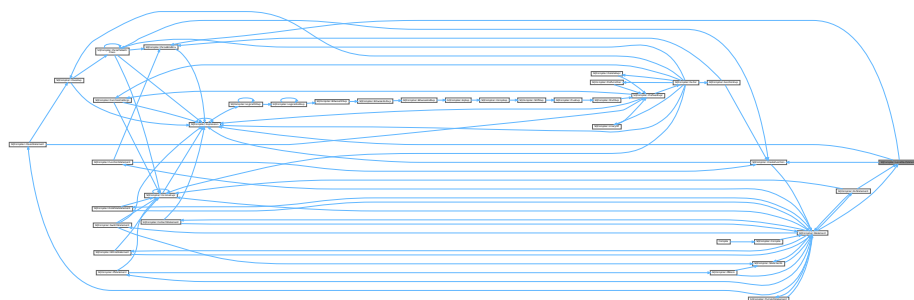
References [_fs](#), [_token](#), [CreateFunction\(\)](#), [Expect\(\)](#), [Expression\(\)](#), [Lex\(\)](#), and [ParseBindEnv\(\)](#).

Referenced by [ForStatement\(\)](#), and [Statement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.38 LogicalAndExp()

```
void SQCompiler::LogicalAndExp ( ) [inline]
```

論理AND (&&) 式を解析します。

短絡評価を実装します。左辺式が `false` の場合、右辺式は評価されません。

Definition at line 667 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```

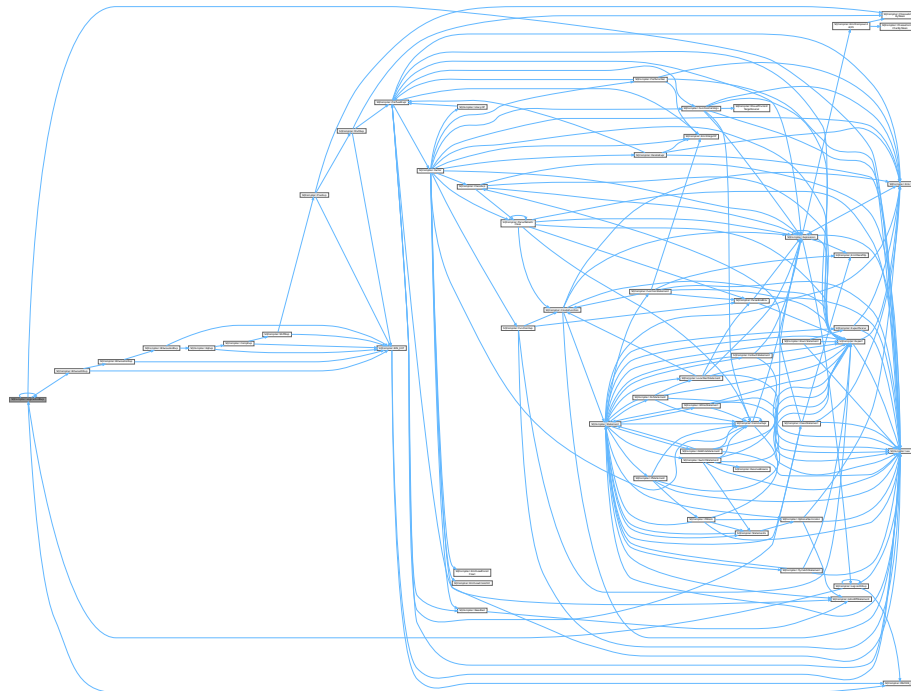
00668     {
00669         BitwiseOrExp();
00670         for(;;) switch(_token) {
00671             case TK_AND: {
00672                 SQInteger first_exp = _fs->PopTarget();
00673                 SQInteger trg = _fs->PushTarget();
00674                 _fs->AddInstruction(_OP_AND, trg, 0, first_exp, 0);
00675                 SQInteger jpos = _fs->GetCurrentPos();
00676                 if(trg != first_exp) _fs->AddInstruction(_OP_MOVE, trg, first_exp);
00677                 Lex(); INVOKE_EXP(&SQCompiler::LogicalAndExp);
00678                 _fs->SnoozeOpt();
00679                 SQInteger second_exp = _fs->PopTarget();
00680                 if(trg != second_exp) _fs->AddInstruction(_OP_MOVE, trg, second_exp);
00681                 _fs->SnoozeOpt();
00682                 _fs->SetInstructionParam(jpos, 1, (_fs->GetCurrentPos() - jpos));
00683                 _es.etype = EXPR;
00684                 break;
00685             }
00686             default:
00687                 return;
00688         }
00689     }
00690 }

```

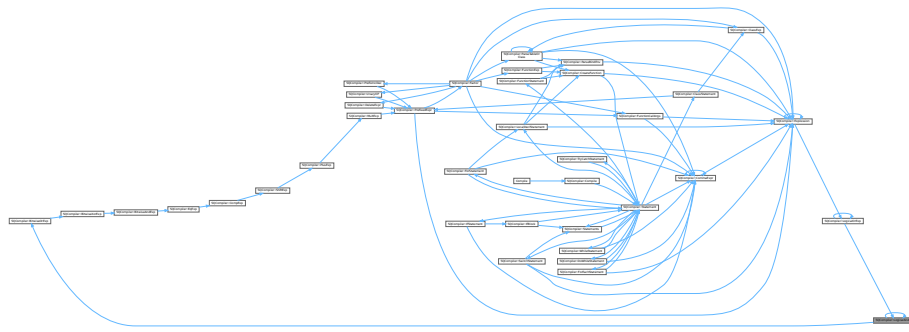
References [_es](#), [_fs](#), [_token](#), [BitwiseOrExp\(\)](#), [SQExpState::etype](#), [EXPR](#), [INVOKE_EXP\(\)](#), [Lex\(\)](#), and [LogicalAndExp\(\)](#).

Referenced by [LogicalAndExp\(\)](#), and [LogicalOrExp\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.39 LogicalOrExp()

```
void SQCompiler::LogicalOrExp ( ) [inline]
```

論理OR (||) 式を解析します。

短絡評価を実装します。左辺式が true の場合、右辺式は評価されません。

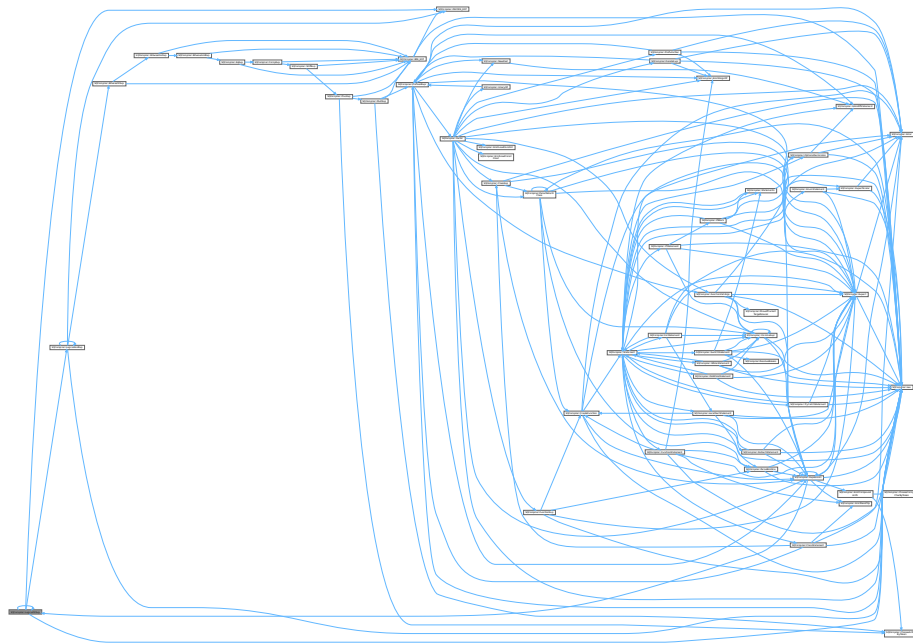
Definition at line 644 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00645     {
00646         LogicalAndExp();
00647         for(;;) if(_token == TK_OR) {
00648             SQInteger first_exp = _fs->PopTarget();
00649             SQInteger trg = _fs->PushTarget();
00650             _fs->AddInstruction(_OP_OR, trg, 0, first_exp, 0);
00651             SQInteger jpos = _fs->GetCurrentPos();
00652             if(trg != first_exp) _fs->AddInstruction(_OP_MOVE, trg, first_exp);
00653             Lex(); INVOKE_EXP(&SQCompiler::LogicalOrExp);
00654             _fs->SnoozeOpt();
00655             SQInteger second_exp = _fs->PopTarget();
00656             if(trg != second_exp) _fs->AddInstruction(_OP_MOVE, trg, second_exp);
00657             _fs->SnoozeOpt();
00658             _fs->SetInstructionParam(jpos, 1, (_fs->GetCurrentPos() - jpos));
00659             _es.etype = EXPR;
00660             break;
00661         }else return;
00662     }
```

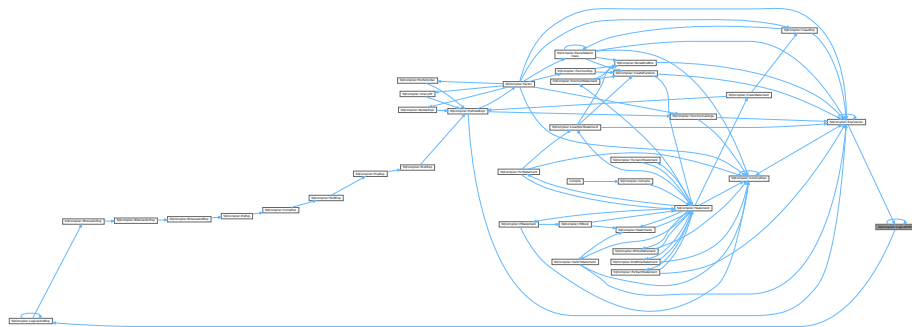
References [_es](#), [_fs](#), [_token](#), [SQExpState::etype](#), [EXPR](#), [INVOKE_EXP\(\)](#), [Lex\(\)](#), [LogicalAndExp\(\)](#), and [LogicalOrExp\(\)](#).

Referenced by [Expression\(\)](#), and [LogicalOrExp\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.40 MoveIfCurrentTargetIsLocal()

```
void SQCompiler::MoveIfCurrentTargetIsLocal ( ) [inline]
```

現在のターゲットがローカル変数の場合にMOVE 命令を生成する。

スタックトップのターゲットがローカル変数を指している場合、その値を新しいターゲット位置に移動させるための_OP_MOVE 命令を追加します。これは、式の評価結果が一時的な位置ではなく、永続的なレジスタに格納されることを保証するために使用されます。

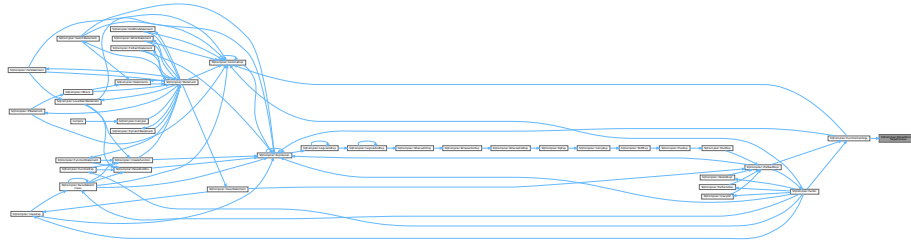
Definition at line 267 of file squirrel3_squirrel_sqcompiler_v1.cpp.

```
00267 {
00268     SQInteger trg = _fs->TopTarget();
00269     if(_fs->IsLocal(trg)) {
00270         trg = _fs->PopTarget(); //pops the target and moves it
00271         _fs->AddInstruction(_OP_MOVE, _fs->PushTarget(), trg);
00272     }
00273 }
```

References [_fs](#).

Referenced by [FunctionCallArgs\(\)](#).

Here is the caller graph for this function:



3.1.3.41 MultExp()

```
void SQCompiler::MultExp ( ) [inline]
```

乗算・除算・剰余(*,/,%) 式を解析します。

BIN_EXP ヘルパーを利用して、乗算・除算・剰余命令を生成します。

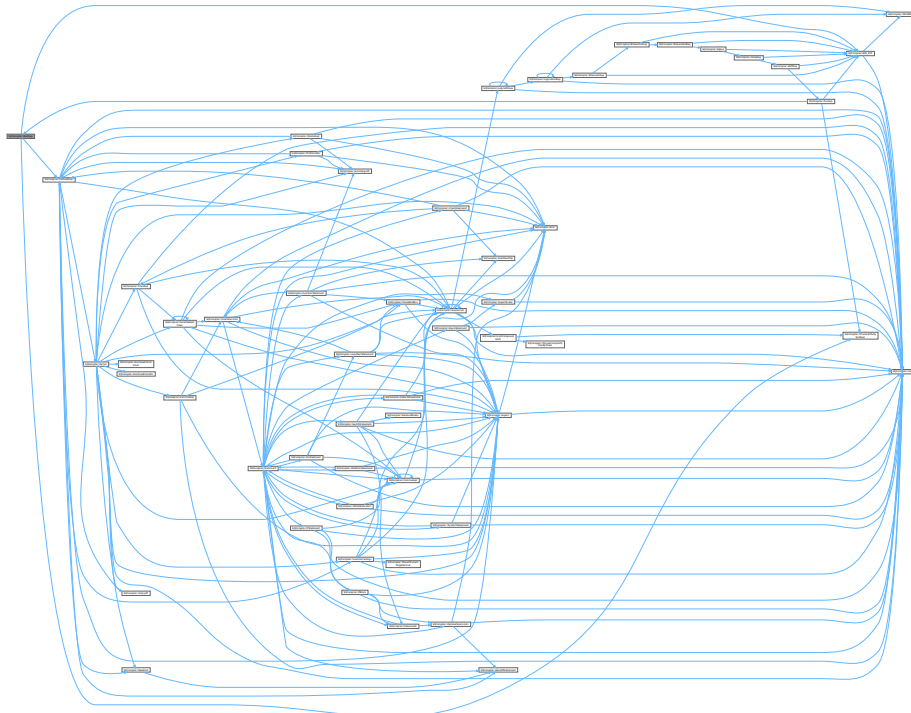
Definition at line 825 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00826     {
00827         PrefixedExpr();
00828         for(;;) switch(_token) {
00829             case _SC('*'): case _SC('/'): case _SC('%'):
00830                 BIN_EXP(ChooseArithOpByToken(_token), &SQCompiler::PrefixedExpr); break;
00831             default: return;
00832         }
00833     }
```

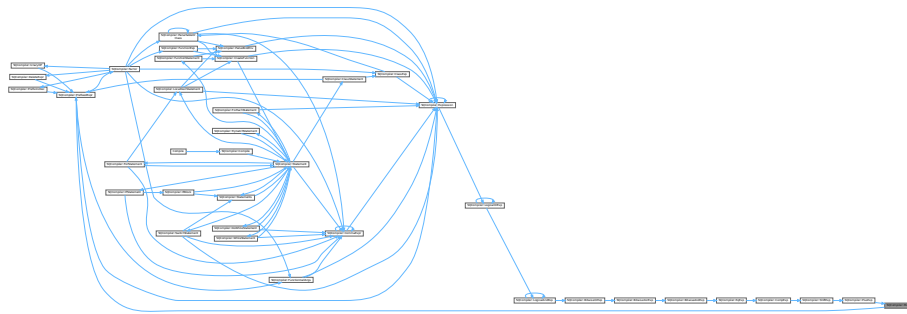
References [_token](#), [BIN_EXP\(\)](#), [ChooseArithOpByToken\(\)](#), and [PrefixedExpr\(\)](#).

Referenced by [PlusExp\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.42 NeedGet()

```
bool SQCompiler::NeedGet ( ) [inline]
```

オブジェクトのメンバアクセス時に値を取得 (GET) する必要があるかを判断します。

次のトークンが代入演算子や関数呼び出しなど、左辺値として扱われる文脈でない場合にGETが必要であると判断します。これにより、`a.b = 1` のような代入文では不要なGET命令の生成を抑制します。

Returns

GET 命令を生成する必要がある場合は `true`、不要であれば `false`。

Definition at line 1165 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
01166     {
01167         switch(_token) {
01168             case _SC('='): case _SC('('): case TK_NEWSLIST: case TK_MODEQ: case TK_MULEQ:
01169             case TK_DIVEQ: case TK_MINUSEQ: case TK_PLUSEQ:
01170                 return false;
01171             case TK_PLUSPLUS: case TK_MINUSMINUS:
01172                 if (!IsEndOfStatement()) {
01173                     return false;
01174                 }
01175             break;
01176         }
01177         return (!_es.donot_get || ( _es.donot_get && (_token == _SC('.') || _token == _SC('['))));
01178     }
```

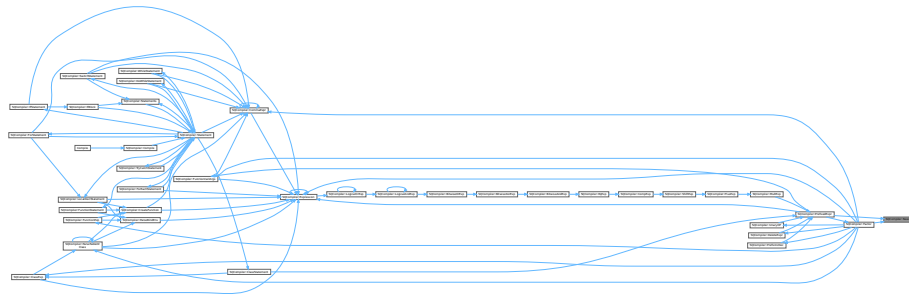
References [_es](#), [_token](#), [SQExpState::donot_get](#), and [IsEndOfStatement\(\)](#).

Referenced by [Factor\(\)](#), and [PrefixedExpr\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.43 OptionalSemicolon()

```
void SQCompiler::OptionalSemicolon ( ) [inline]
```

文末の任意で記述されるセミコロンを処理する。

現在のトークンがセミコロンであればそれを消費します。セミコロンでなく、かつ文の終わりでもない場合はエラーを発生させます。

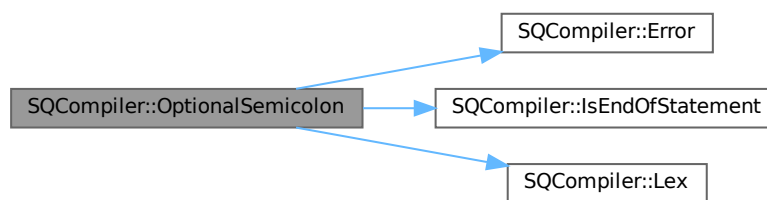
Definition at line 254 of file `squirrel3_squirrel_sqcompiler_v1.cpp`.

```
00255     {
00256         if(_token == _SC(';')) { Lex(); return; }
00257         if(!IsEndOfStatement()) {
00258             Error(_SC("end of statement expected (; or lf)"));
00259         }
00260     }
```

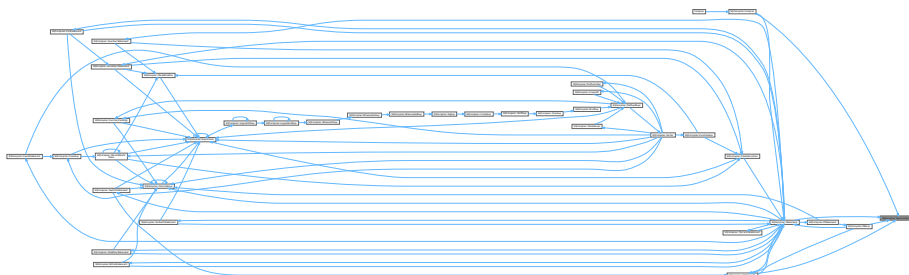
References [_token](#), [Error\(\)](#), [IsEndOfStatement\(\)](#), and [Lex\(\)](#).

Referenced by [Compile\(\)](#), [IfBlock\(\)](#), [Statement\(\)](#), and [Statements\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.44 ParseBindEnv()

SQInteger SQCompiler::ParseBindEnv () [inline]

関数の環境束縛構文 [...] を解析します。

function foo[...](){} のような構文で、[] 内の式を評価し、その結果を関数の環境として束縛するための準備をします。

Returns

束縛される環境オブジェクトが格納されているスタック上の位置。

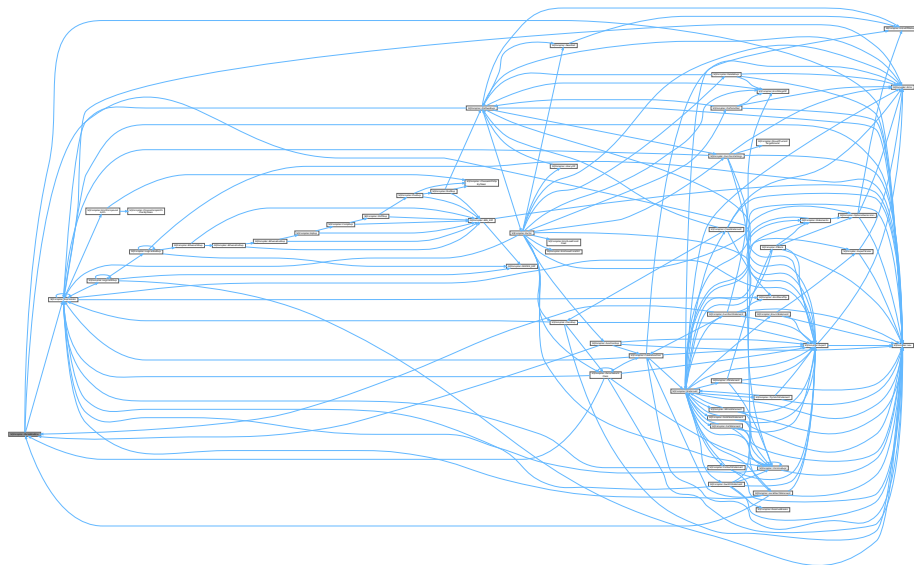
Definition at line 1794 of file squirrel3_squirrel_sqcompiler_v1.cpp.

```
01795 {
01796     SQInteger boundtarget;
01797     Lex();
01798     Expression();
01799     boundtarget = _fs->TopTarget();
01800     Expect(_SC(']'));
01801     return boundtarget;
01802 }
```

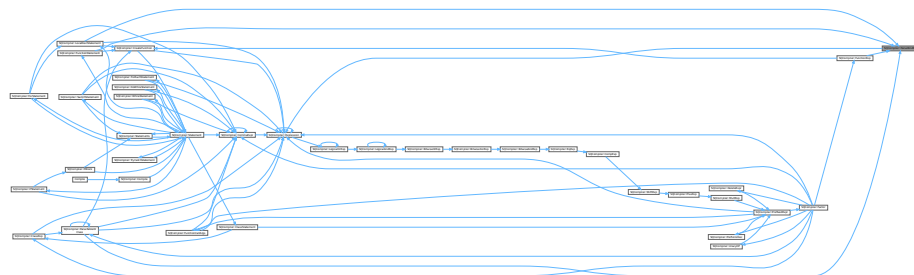
References [_fs](#), [Expect\(\)](#), [Expression\(\)](#), and [Lex\(\)](#).

Referenced by [FunctionExp\(\)](#), [FunctionStatement\(\)](#), [LocalDeclStatement\(\)](#), and [ParseTableOrClass\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.45 ParseTableOrClass()

```
void SQCompiler::ParseTableOrClass (
    SQInteger separator,
    SQInteger terminator ) [inline]
```

テーブルまたはクラスの本体 ({}内) を解析します。

キーと値のペアを、指定された **terminator** トークンが現れるまで解析します。 **separator** はエントリ間の区切り文字 (テーブルは','、クラスは';') です。 クラスの場合、**static** メンバや属性 (<< >>) も処理します。

Parameters

<i>separator</i>	エントリ間の区切り文字。
<i>terminator</i>	ブロックの終了を示すトークン。

Definition at line 1240 of file `squirrel3_squirrel_sqcompiler_v1.cpp`.

```
01241     {
01242         SQInteger tpos = _fs->GetCurrentPos(), nkeys = 0;
01243         while(_token != terminator) {
01244             bool hasattrs = false;
01245             bool isstatic = false;
01246             //check if is an attribute
01247             if(separator == ';' ) {
01248                 if(_token == TK_ATTR_OPEN) {
01249                     _fs->AddInstruction(_OP_NEWOBJ, _fs->PushTarget(), 0, NOT_TABLE); Lex();
01250                     ParseTableOrClass(' ', TK_ATTR_CLOSE);
01251                     hasattrs = true;
01252                 }
01253                 if(_token == TK_STATIC) {
01254                     isstatic = true;
01255                     Lex();
01256                 }
01257             }
01258             switch(_token) {
01259             case TK_FUNCTION:
01260             case TK_CONSTRUCTOR: {
01261                 SQInteger tk = _token;
01262                 Lex();
01263                 SQObject id = tk == TK_FUNCTION ? Expect(TK_IDENTIFIER) :
01264                     _fs->CreateString(_SC("constructor"));
01265                 _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(), _fs->GetConstant(id));
01266                 SQInteger boundtarget = 0xFF;
01267                 if (_token == _SC('{ ')) {
01268                     boundtarget = ParseBindEnv();
01269                 }
01270                 Expect(_SC(' '));
01271                 CreateFunction(id, boundtarget);
01272                 _fs->AddInstruction(_OP_CLOSURE, _fs->PushTarget(), _fs->_functions.size() - 1,
01273                     boundtarget);
01274             }
01275             case _SC('{ '):
01276                 Lex(); CommaExpr(); Expect(_SC('] '));
01277                 Expect(_SC('=')); Expression();
01278                 break;
01279             case TK_STRING_LITERAL: //JSON
01280                 if(separator == ',') { //only works for tables
01281                     _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
01282                         _fs->GetConstant(Expect(TK_STRING_LITERAL)));
01283                     Expect(_SC(':')); Expression();
01284                     break;
01285                 }
01286             default :
01287                 _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
01288                     _fs->GetConstant(Expect(TK_IDENTIFIER)));
01289                 Expect(_SC('=')); Expression();
01290             }
01291             if(_token == separator) Lex(); //optional comma/semicolon
01292             nkeys++;
01293             SQInteger val = _fs->PopTarget();
01294             SQInteger key = _fs->PopTarget();
01295             SQInteger attrs = hasattrs ? _fs->PopTarget() - 1;
01296             ((void)attrs);
01297             assert((hasattrs && (attrs == key-1)) || !hasattrs);
01298         }
01299     }
```

```

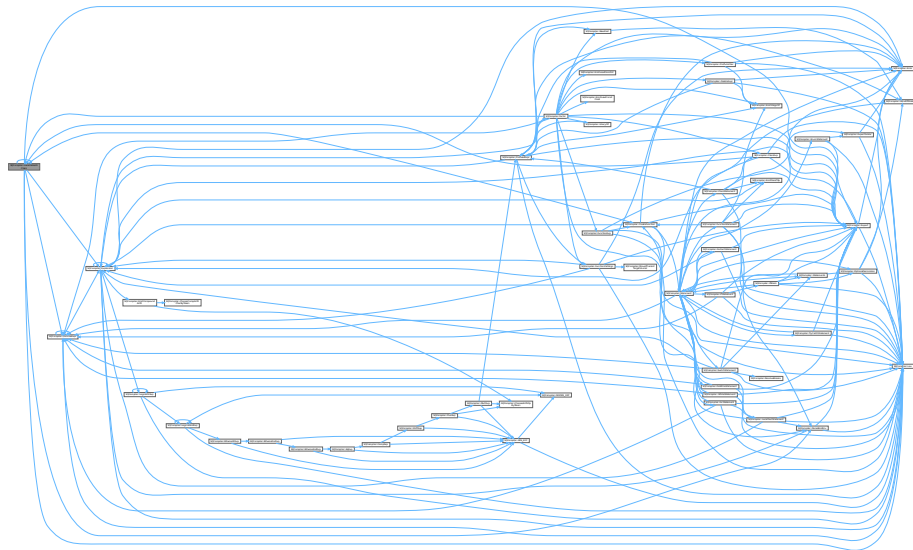
01296         unsigned char flags =
01297         (hasattrs?NEW_SLOT_ATTRIBUTES_FLAG:0)|(isstatic?NEW_SLOT_STATIC_FLAG:0);
01297         SQInteger table = _fs->TopTarget(); //«BECAUSE OF THIS NO COMMON EMIT FUNC IS POSSIBLE
01298         if(separator == _SC(',') { //hack recognizes a table from the separator
01299             _fs->AddInstruction(_OP_NEWSLOT, 0xFF, table, key, val);
01300         }
01301         else {
01302             _fs->AddInstruction(_OP_NEWSLOTA, flags, table, key, val); //this for classes only as
01302             it invokes _newmember
01303         }
01304     }
01305     if(separator == _SC(',') { //hack recognizes a table from the separator
01306         _fs->SetInstructionParam(tpos, 1, nkeys);
01307         Lex();
01308     }

```

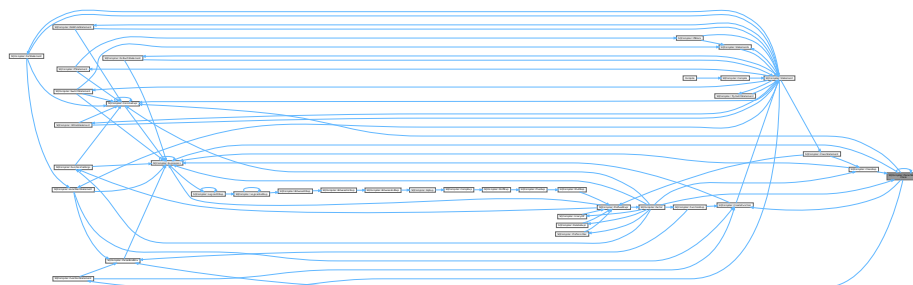
References [_fs](#), [_token](#), [CommaExpr\(\)](#), [CreateFunction\(\)](#), [Expect\(\)](#), [Expression\(\)](#), [Lex\(\)](#), [ParseBindEnv\(\)](#), and [ParseTableOrClass\(\)](#).

Referenced by [ClassExp\(\)](#), [Factor\(\)](#), and [ParseTableOrClass\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.46 PlusExp()

```
void SQCompiler::PlusExp ( ) [inline]
```

加算・減算 (+, -) 式を解析します。

BIN_EXP ヘルパーを利用して、加算・減算命令を生成します。

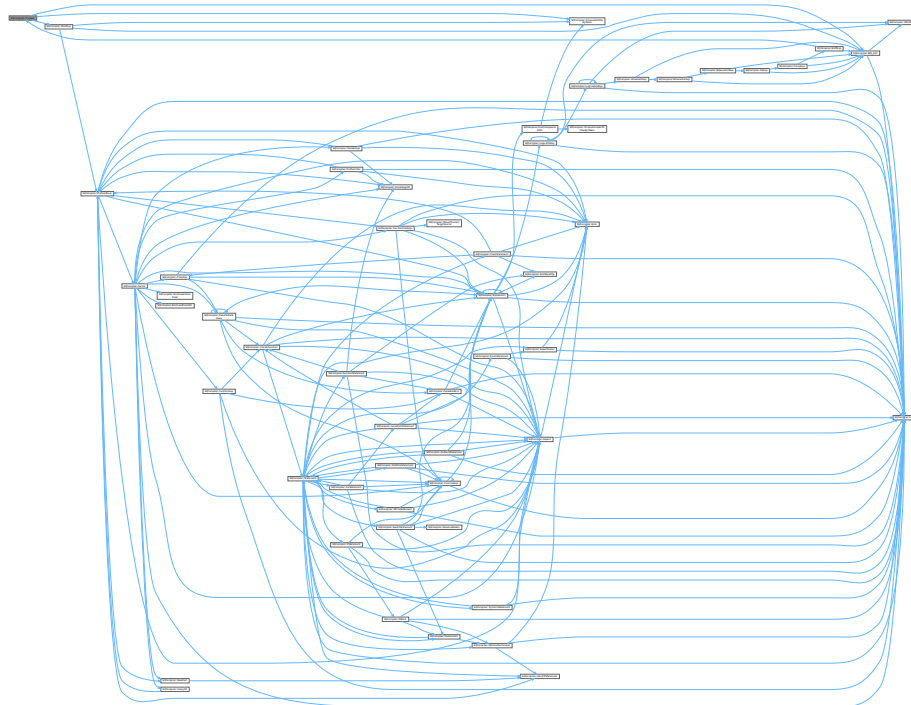
Definition at line 811 of file squirrel3_squirrel_sqcompiler_v1.cpp.

```
00812     {
00813         MultExp();
00814         for(;;) switch(_token) {
00815             case _SC('+'): case _SC('-'):
00816                 BIN_EXP(ChooseArithOpByToken(_token), &SQCompiler::MultExp); break;
00817             default: return;
00818         }
00819     }
```

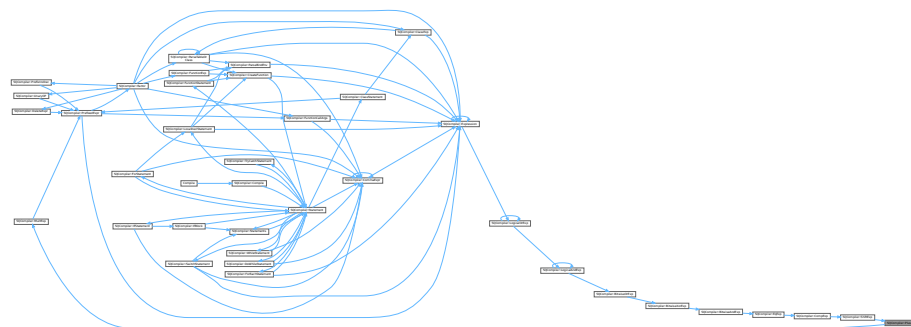
References [_token](#), [BIN_EXP\(\)](#), [ChooseArithOpByToken\(\)](#), and [MultExp\(\)](#).

Referenced by [ShiftExp\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.47 PrefixedExpr()

```
void SQCompiler::PrefixedExpr ( ) [inline]
```

接頭辞式および後置式（メンバアクセス、関数呼び出し等）を解析します。

まず `Factor()` を呼び出して式の基本部分を解析し、その後ろに続くドット(.)、ブラケット([])、関数呼び出し(()、後置インクリメント/デクリメント(++/--)などをループで処理します。

Definition at line 839 of file `squirrel3_squirrel_sqcompiler_v1.cpp`.

```
00840     {
00841         SQInteger pos = Factor();
00842         for(;;) {
00843             switch(_token) {
00844                 case _SC('.') :
00845                     pos = -1;
00846                     Lex();
00847
00848                     _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
00849 _fs->GetConstant(Expect(TK_IDENTIFIER)));
00849                     if(_es.etype==BASE) {
00850                         Emit2ArgsOP(_OP_GET);
00851                         pos = _fs->TopTarget();
00852                         _es.etype = EXPR;
00853                         _es.epos = pos;
00854                     }
00855                     else {
00856                         if(NeedGet()) {
00857                             Emit2ArgsOP(_OP_GET);
00858                         }
00859                         _es.etype = OBJECT;
00860                     }
00861                     break;
00862                 case _SC('[') :
00863                     if(_lex._prevtoken == _SC('\n')) Error(_SC("cannot break deref/or comma needed after
[exp]=exp slot declaration"));
00864                     Lex(); Expression(); Expect(_SC(']'));
00865                     pos = -1;
00866                     if(_es.etype==BASE) {
00867                         Emit2ArgsOP(_OP_GET);
00868                         pos = _fs->TopTarget();
00869                         _es.etype = EXPR;
00870                         _es.epos = pos;
00871                     }
00872                     else {
00873                         if(NeedGet()) {
00874                             Emit2ArgsOP(_OP_GET);
00875                         }
00876                         _es.etype = OBJECT;
00877                     }
00878                     break;
00879                 case TK_MINUSMINUS:
00880                 case TK_PLUSPLUS:
00881                     {
00882                         if(IsEndOfStatement()) return;
00883                         SQInteger diff = (_token==TK_MINUSMINUS) ? -1 : 1;
00884                         Lex();
00885                         switch(_es.etype)
00886                         {
00887                             case EXPR: Error(_SC("can't '++' or '--' an expression")); break;
00888                             case OBJECT:
00889                             case BASE:
00890                                 if(_es.donot_get == true) { Error(_SC("can't '++' or '--' an
expression")); break; } //mmh dor this make sense?
00891                                 Emit2ArgsOP(_OP_PINC, diff);
00892                                 break;
00893                             case LOCAL: {
00894                                 SQInteger src = _fs->PopTarget();
00895                                 _fs->AddInstruction(_OP_PINCL, _fs->PushTarget(), src, 0, diff);
00896                                 }
00897                                 break;
00898                             case OUTER: {
00899                                 SQInteger tmp1 = _fs->PushTarget();
00900                                 SQInteger tmp2 = _fs->PushTarget();
00901                                 _fs->AddInstruction(_OP_GETOUTER, tmp2, _es.epos);
00902                                 _fs->AddInstruction(_OP_PINCL, tmp1, tmp2, 0, diff);
00903                                 _fs->AddInstruction(_OP_SETOUTER, tmp2, _es.epos, tmp2);
00904                                 _fs->PopTarget();
00905                             }
00906                         }
00907                     }
00908                     return;
00909 }
```

```

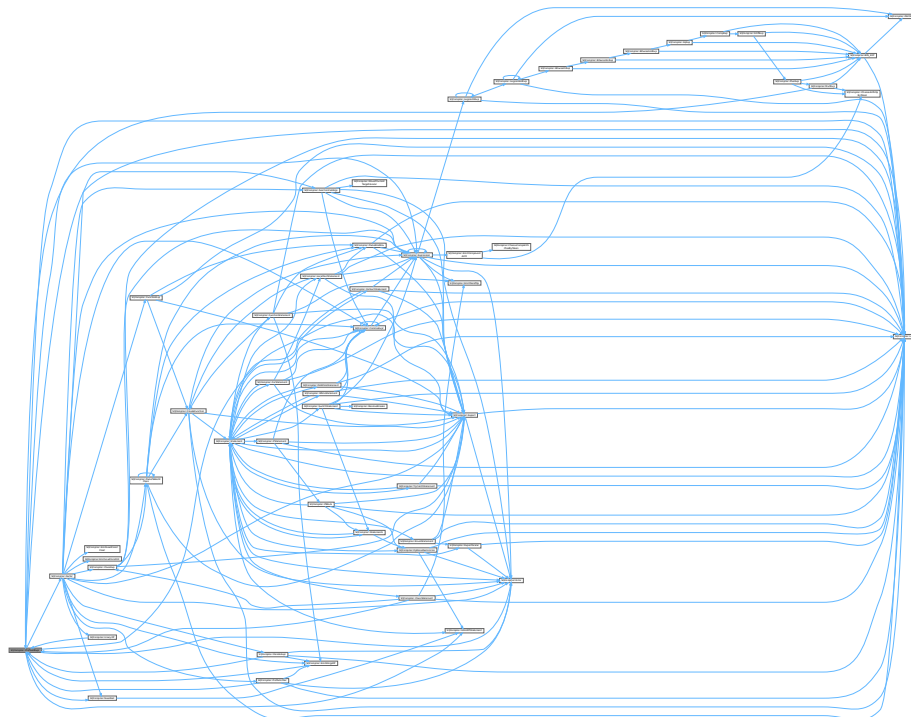
00909         break;
00910     case _SC('('):
00911         switch(_es.etype) {
00912             case OBJECT: {
00913                 SQInteger key = _fs->PopTarget(); /* location of the key */
00914                 SQInteger table = _fs->PopTarget(); /* location of the object */
00915                 SQInteger closure = _fs->PushTarget(); /* location for the closure */
00916                 SQInteger ttarget = _fs->PushTarget(); /* location for 'this' pointer */
00917                 _fs->AddInstruction(_OP_PREPCALL, closure, key, table, ttarget);
00918             }
00919             break;
00920             case BASE:
00921                 //Emit2ArgsOP(_OP_GET);
00922                 _fs->AddInstruction(_OP_MOVE, _fs->PushTarget(), 0);
00923             break;
00924             case OUTER:
00925                 _fs->AddInstruction(_OP_GETOUTER, _fs->PushTarget(), _es.epos);
00926                 _fs->AddInstruction(_OP_MOVE, _fs->PushTarget(), 0);
00927             break;
00928             default:
00929                 _fs->AddInstruction(_OP_MOVE, _fs->PushTarget(), 0);
00930         }
00931         _es.etype = EXPR;
00932         Lex();
00933         FunctionCallArgs();
00934         break;
00935     default: return;
00936 }
00937 }
00938 }

```

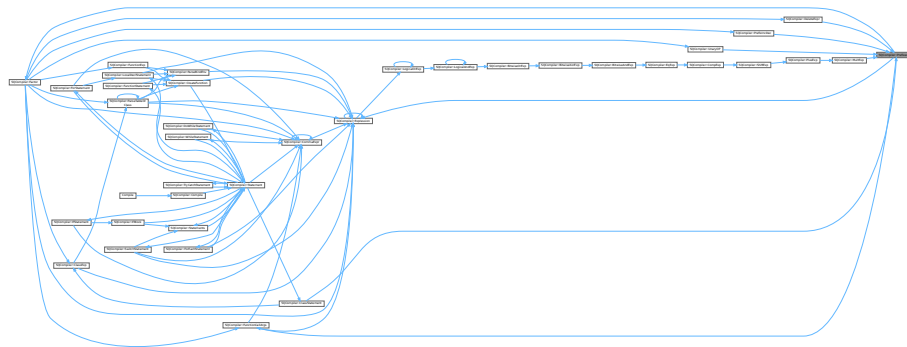
References [_es](#), [_fs](#), [_lex](#), [_token](#), [BASE](#), [SQExpState::donot_get](#), [Emit2ArgsOP\(\)](#), [SQExpState::epos](#), [Error\(\)](#), [SQExpState::etype](#), [Expect\(\)](#), [EXPR](#), [Expression\(\)](#), [Factor\(\)](#), [FunctionCallArgs\(\)](#), [IsEndOfStatement\(\)](#), [Lex\(\)](#), [LOCAL](#), [NeedGet\(\)](#), [OBJECT](#), and [OUTER](#).

Referenced by [ClassStatement\(\)](#), [DeleteExpr\(\)](#), [MultExp\(\)](#), [PrefixIncDec\(\)](#), and [UnaryOP\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.48 PrefixIncDec()

```
void SQCompiler::PrefixIncDec (
    SQInteger token ) [inline]
```

前置インクリメント/デクリメント (++var, --var) を解析します。

式の種類 (ローカル、オブジェクト、アウター) に応じて、適切なインクリメント/デクリメント命令を生成します。

Parameters

<i>token</i>	TK_PLUSPLUS または TK_MINUSMINUS。
--------------	--------------------------------

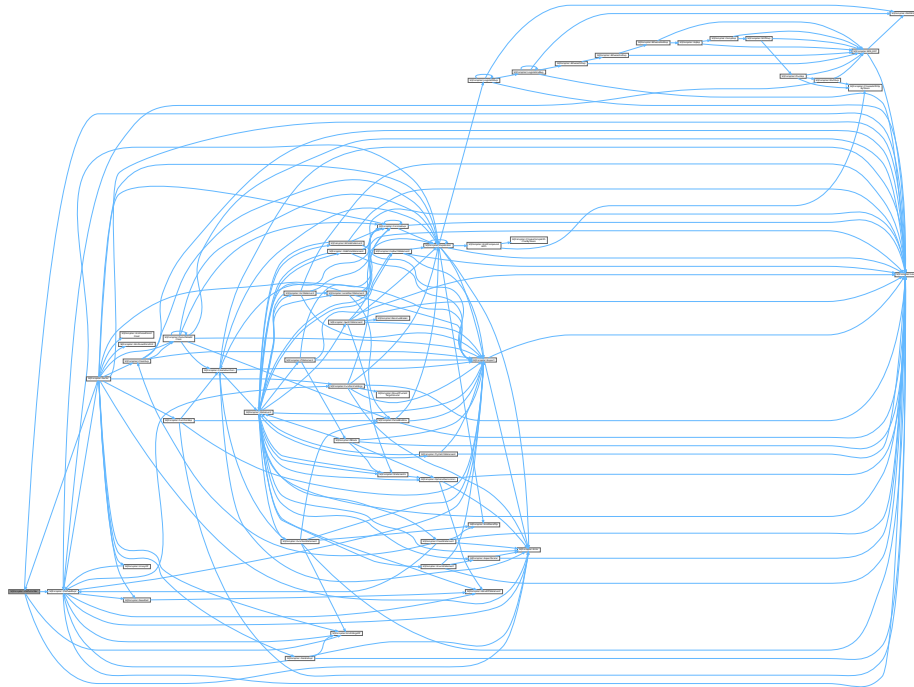
Definition at line 1872 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
01873     {
01874         SQExpState es;
01875         SQInteger diff = (token==TK_MINUSMINUS) ? -1 : 1;
01876         Lex();
01877         es = _es;
01878         _es.donot_get = true;
01879         PrefixedExpr();
01880         if(_es.etype==EXPR) {
01881             Error(_SC("can't '++' or '--' an expression"));
01882         }
01883         else if(_es.etype==OBJECT || _es.etype==BASE) {
01884             Emit2ArgsOP(_OP_INC, diff);
01885         }
01886         else if(_es.etype==LOCAL) {
01887             SQInteger src = _fs->TopTarget();
01888             _fs->AddInstruction(_OP_INCL, src, src, 0, diff);
01889         }
01890         }
01891         else if(_es.etype==OUTER) {
01892             SQInteger tmp = _fs->PushTarget();
01893             _fs->AddInstruction(_OP_GETOUTER, tmp, _es.epos);
01894             _fs->AddInstruction(_OP_INCL, tmp, tmp, 0, diff);
01895             _fs->AddInstruction(_OP_SETOUTER, tmp, _es.epos, tmp);
01896         }
01897         _es = es;
01898     }
```

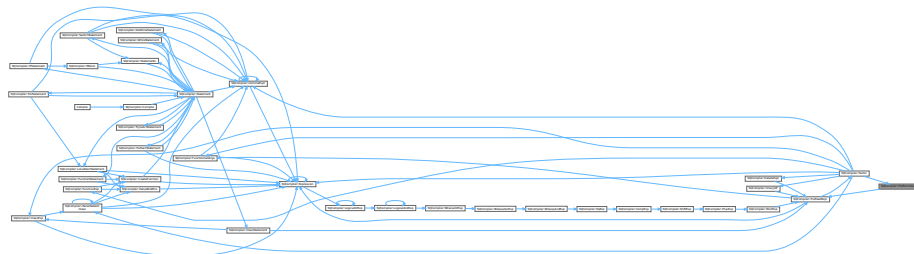
References [_es](#), [_fs](#), [BASE](#), [SQExpState::donot_get](#), [Emit2ArgsOP\(\)](#), [SQExpState::epos](#), [Error\(\)](#), [SQExpState::etype](#), [EXPR](#), [Lex\(\)](#), [LOCAL](#), [OBJECT](#), [OUTER](#), and [PrefixedExpr\(\)](#).

Referenced by [Factor\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.49 ResolveBreaks()

```
void SQCompiler::ResolveBreaks (
    SQFuncState * funcstate,
    SQInteger ntoresolve ) [inline]
```

未解決の break 文のジャンプ先を解決します。

ループや switch ブロックの終端で呼び出され、未解決の break リストを逆順にたどり、JMP 命令の引数に現在の命令位置を設定します。

Parameters

<i>funcstate</i>	現在の関数状態。
<i>ntoresolve</i>	解決する break の数。

Definition at line 1975 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

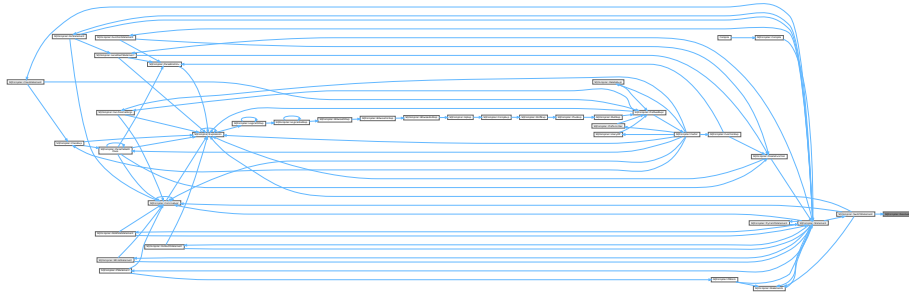
```

01976     {
01977         while (ntoresolve > 0) {
01978             SQInteger pos = funcstate->_unresolvedbreaks.back();
01979             funcstate->_unresolvedbreaks.pop_back();
01980             //set the jmp instruction
01981             funcstate->SetInstructionParams(pos, 0, funcstate->GetCurrentPos() - pos, 0);
01982             ntoresolve--;
01983         }
01984     }

```

Referenced by [SwitchStatement\(\)](#).

Here is the caller graph for this function:



3.1.3.50 ResolveContinues()

```

void SQCompiler::ResolveContinues (
    SQFuncState * funcstate,
    SQInteger ntoresolve,
    SQInteger targetpos ) [inline]

```

未解決の continue 文のジャンプ先を解決します。

ループの終端で呼び出され、未解決の continue リストを逆順にたどり、JMP 命令の引数にループの先頭 (または更新部) の位置を設定します。

Parameters

<i>funcstate</i>	現在の関数状態。
<i>ntoresolve</i>	解決する continue の数。
<i>targetpos</i>	ジャンプ先の命令位置。

Definition at line 1993 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```

01994     {
01995         while (ntoresolve > 0) {
01996             SQInteger pos = funcstate->_unresolvedcontinues.back();
01997             funcstate->_unresolvedcontinues.pop_back();
01998             //set the jmp instruction
01999             funcstate->SetInstructionParams(pos, 0, targetpos - pos, 0);
02000             ntoresolve--;
02001         }
02002     }

```

3.1.3.51 ShiftExp()

```

void SQCompiler::ShiftExp ( ) [inline]

```

ビットシフト (<<, >>, >>>) 式を解析します。

BIN_EXP ヘルパーを利用して、ビットシフト演算命令を生成します。

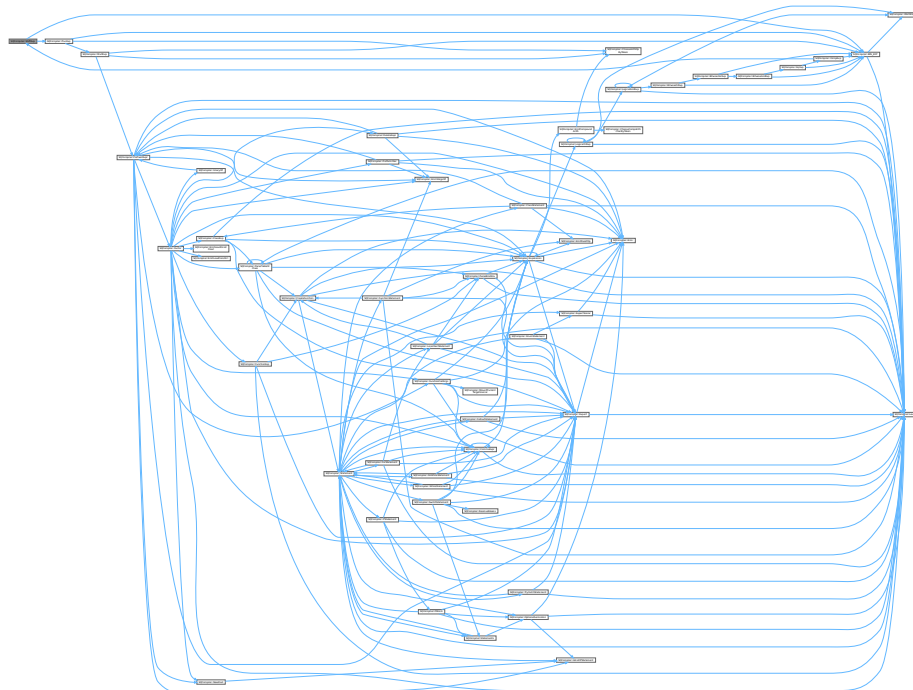
Definition at line 759 of file squirrel3_squirrel_sqcompiler_v1.cpp.

```
00760     {
00761         PlusExp();
00762         for(;;) switch(_token) {
00763             case TK_USHIFTR: BIN_EXP(_OP_BITW, &SQCompiler::PlusExp, BW_USHIFTR); break;
00764             case TK_SHIFTL:  BIN_EXP(_OP_BITW, &SQCompiler::PlusExp, BW_SHIFTL); break;
00765             case TK_SHIFTR:  BIN_EXP(_OP_BITW, &SQCompiler::PlusExp, BW_SHIFTR); break;
00766             default: return;
00767         }
00768     }
```

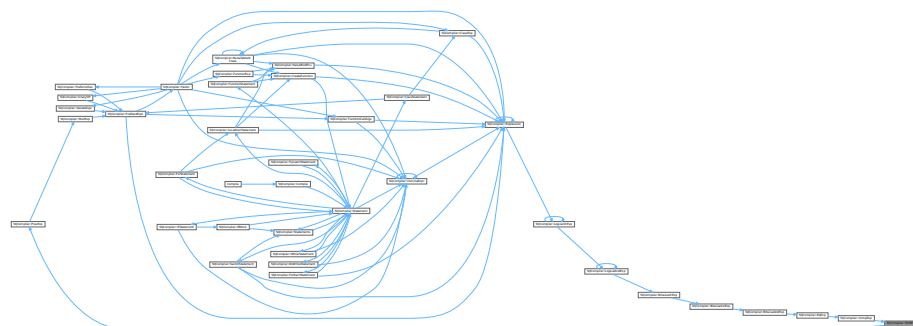
References [_token](#), [BIN_EXP\(\)](#), and [PlusExp\(\)](#).

Referenced by [CompExp\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.52 Statement()

```
void SQCompiler::Statement (
    bool closeframe = true ) [inline]
```

単一の文を解析し、対応するバイトコードを生成する。

現在のトークンに応じて、if, while, for, return などの各種文の解析処理に分岐します。

Parameters

<i>closeframe</i>	現在のスコープの終了時にフレームを閉じる (_OP_CLOSE) かどうか。
-------------------	--

Definition at line 334 of file squirrel3_squirrel_sqcompiler_v1.cpp.

```
00335 {
00336     _fs->AddLineInfos(_lex._currentline, _lineinfo);
00337     switch(_token){
00338     case _SC(';'): Lex(); break;
00339     case TK_IF: IfStatement(); break;
00340     case TK_WHILE: WhileStatement(); break;
00341     case TK_DO: DoWhileStatement(); break;
00342     case TK_FOR: ForStatement(); break;
00343     case TK_FOREACH: ForEachStatement(); break;
00344     case TK_SWITCH: SwitchStatement(); break;
00345     case TK_LOCAL: LocalDeclStatement(); break;
00346     case TK_RETURN:
00347     case TK_YIELD: {
00348         SQOpcode op;
00349         if(_token == TK_RETURN) {
00350             op = _OP_RETURN;
00351         }
00352         else {
00353             op = _OP_YIELD;
00354             _fs->_bgenerator = true;
00355         }
00356         Lex();
00357         if(!IsEndOfStatement()) {
00358             SQInteger retexp = _fs->GetCurrentPos()+1;
00359             CommaExpr();
00360             if(op == _OP_RETURN && _fs->_traps > 0)
00361                 _fs->AddInstruction(_OP_POPTRAP, _fs->_traps, 0);
00362             _fs->_returnexp = retexp;
00363             _fs->AddInstruction(op, 1, _fs->PopTarget(), _fs->GetStackSize());
00364         }
00365         else{
00366             if(op == _OP_RETURN && _fs->_traps > 0)
00367                 _fs->AddInstruction(_OP_POPTRAP, _fs->_traps, 0);
00368             _fs->_returnexp = -1;
00369             _fs->AddInstruction(op, 0xFF, 0, _fs->GetStackSize());
00370         }
00371         break;
00372     case TK_BREAK:
00373         if(_fs->_breaktargets.size() <= 0)Error(_SC("'break' has to be in a loop block"));
00374         if(_fs->_breaktargets.top() > 0){
00375             _fs->AddInstruction(_OP_POPTRAP, _fs->_breaktargets.top(), 0);
00376         }
00377         RESOLVE_OUTERS();
00378         _fs->AddInstruction(_OP_JMP, 0, -1234);
00379         _fs->_unresolvedbreaks.push_back(_fs->GetCurrentPos());
00380         Lex();
00381         break;
00382     case TK_CONTINUE:
00383         if(_fs->_continuetargets.size() <= 0)Error(_SC("'continue' has to be in a loop block"));
00384         if(_fs->_continuetargets.top() > 0) {
00385             _fs->AddInstruction(_OP_POPTRAP, _fs->_continuetargets.top(), 0);
00386         }
00387         RESOLVE_OUTERS();
00388         _fs->AddInstruction(_OP_JMP, 0, -1234);
00389         _fs->_unresolvedcontinues.push_back(_fs->GetCurrentPos());
00390         Lex();
00391         break;
00392     case TK_FUNCTION:
00393         FunctionStatement();
00394         break;
00395     case TK_CLASS:
00396         ClassStatement();
00397         break;
00398     case TK_ENUM:
```

```

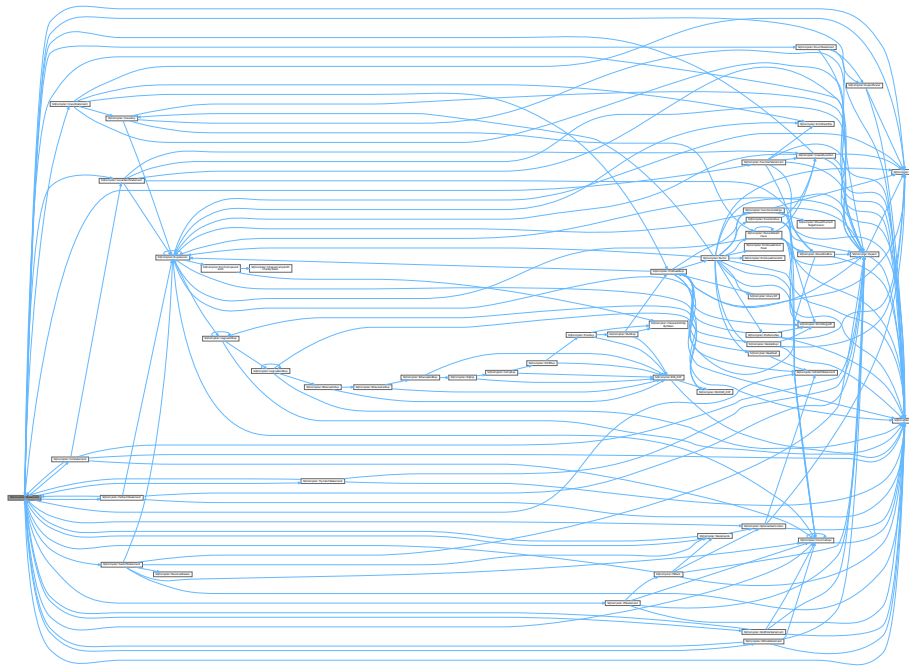
00399         EnumStatement();
00400         break;
00401     case _SC('{'): {
00402         BEGIN_SCOPE();
00403         Lex();
00404         Statements();
00405         Expect(_SC('}'));
00406         if(closeframe) {
00407             END_SCOPE();
00408         }
00409         else {
00410             END_SCOPE_NO_CLOSE();
00411         }
00412     }
00413     break;
00414 case TK_TRY:
00415     TryCatchStatement();
00416     break;
00417 case TK_THROW:
00418     Lex();
00419     CommaExpr();
00420     _fs->AddInstruction(_OP_THROW, _fs->PopTarget());
00421     break;
00422 case TK_CONST:
00423     {
00424         Lex();
00425         SQObject id = Expect(TK_IDENTIFIER);
00426         Expect('=');
00427         SQObject val = ExpectScalar();
00428         OptionalSemicolon();
00429         SQTable *enums = _table(_ss(_vm)->_consts);
00430         SQObjectPtr strongid = id;
00431         enums->NewSlot(strongid, SQObjectPtr(val));
00432         strongid.Null();
00433     }
00434     break;
00435 default:
00436     CommaExpr();
00437     _fs->DiscardTarget();
00438     //_fs->PopTarget();
00439     break;
00440 }
00441 _fs->SnoozeOpt();
00442 }

```

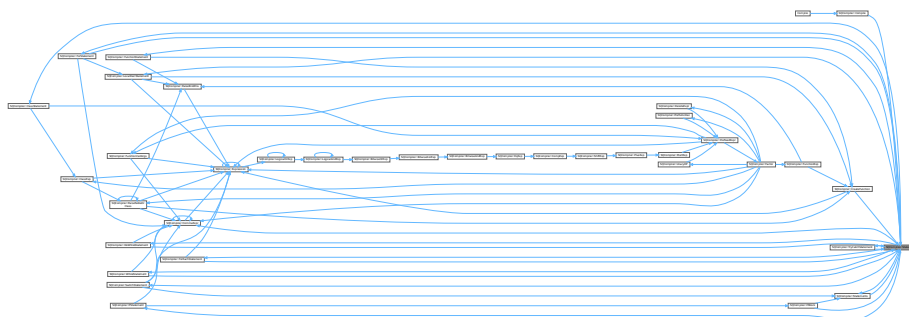
References [_fs](#), [_lex](#), [_lineinfo](#), [_token](#), [_vm](#), [BEGIN_SCOPE](#), [ClassStatement\(\)](#), [CommaExpr\(\)](#), [DoWhileStatement\(\)](#), [END_SCOPE](#), [END_SCOPE_NO_CLOSE](#), [EnumStatement\(\)](#), [Error\(\)](#), [Expect\(\)](#), [ExpectScalar\(\)](#), [ForEachStatement\(\)](#), [ForStatement\(\)](#), [FunctionStatement\(\)](#), [IfStatement\(\)](#), [IsEndOfStatement\(\)](#), [Lex\(\)](#), [LocalDeclStatement\(\)](#), [OptionalSemicolon\(\)](#), [RESOLVE_OUTERS](#), [Statements\(\)](#), [SwitchStatement\(\)](#), [TryCatchStatement\(\)](#), and [WhileStatement\(\)](#).

Referenced by [Compile\(\)](#), [CreateFunction\(\)](#), [DoWhileStatement\(\)](#), [ForEachStatement\(\)](#), [ForStatement\(\)](#), [IfBlock\(\)](#), [Statements\(\)](#), [TryCatchStatement\(\)](#), and [WhileStatement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.53 Statements()

```
void SQCompiler::Statements ( ) [inline]
```

一連の文を解析する。

}', TK_DEFAULT、TK_CASE トークンに遭遇するまで、文の解析を繰り返します。

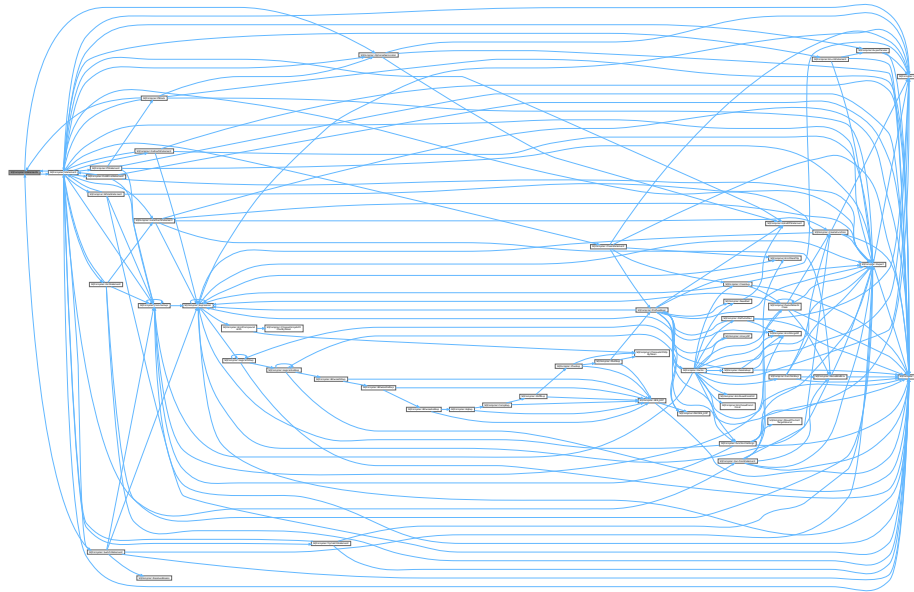
Definition at line 322 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00323     {
00324         while ( _token != _SC('}') && _token != TK_DEFAULT && _token != TK_CASE ) {
00325             Statement();
00326             if ( _lex._prevtoken != _SC(';') && _lex._prevtoken != _SC(';')) OptionalSemicolon();
00327         }
00328     }
```

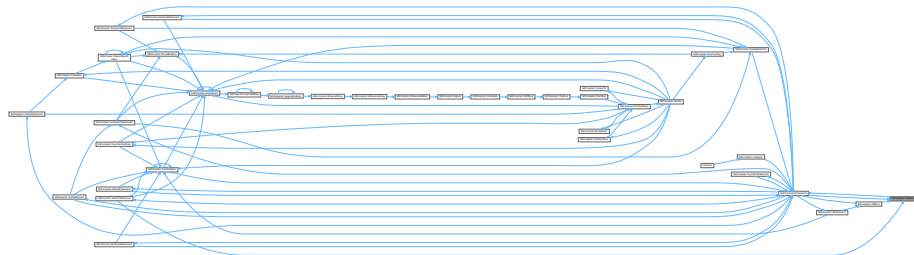
References [_lex](#), [_token](#), [OptionalSemicolon\(\)](#), and [Statement\(\)](#).

Referenced by [IfBlock\(\)](#), [Statement\(\)](#), and [SwitchStatement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.54 SwitchStatement()

```
void SQCompiler::SwitchStatement ( ) [inline]
```

switch 文を解析します。

各 case 節で `_OP_EQ` を使って式と値を比較し、`_OP_JZ` で次の case ヘジャンプするコードを連鎖的に生成します。default 節と break 文の解決も行います。

Definition at line 1563 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
01564     {
01565         Lex(); Expect(_SC('(')); CommaExpr(); Expect(_SC(')'));
01566         Expect(_SC('{'));
01567         SQInteger expr = _fs->TopTarget();
01568         bool bfirst = true;
01569         SQInteger tonextcondjmp = -1;
01570         SQInteger skipcondjmp = -1;
01571         SQInteger __nbreaks__ = _fs->unresolvedbreaks.size();
01572         _fs->breaktargets.push_back(0);
01573         while(_token == TK_CASE) {
01574             if(!bfirst) {
```

```

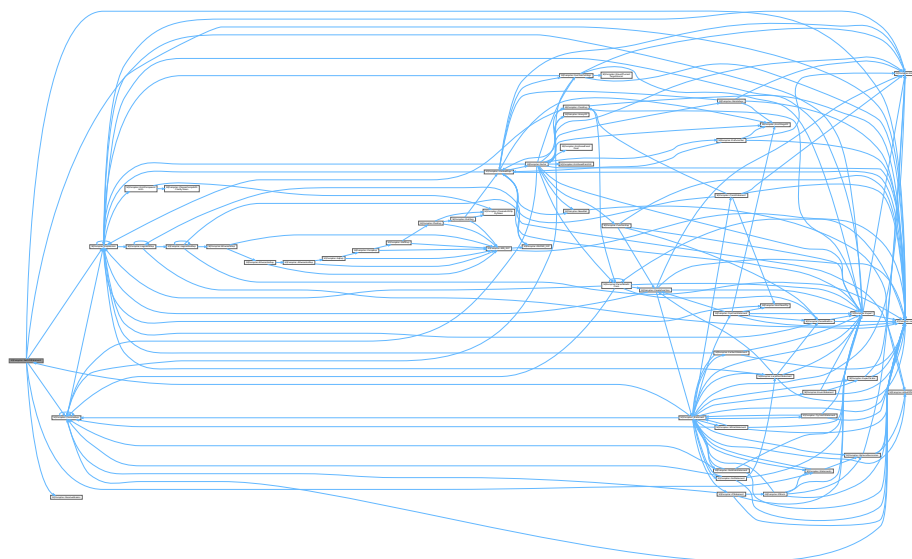
01575         _fs->AddInstruction(_OP_JMP, 0, 0);
01576         skipcondjmp = _fs->GetCurrentPos();
01577         _fs->SetInstructionParam(tonextcondjmp, 1, _fs->GetCurrentPos() - tonextcondjmp);
01578     }
01579     //condition
01580     Lex(); Expression(); Expect(_SC(':'));
01581     SQInteger trg = _fs->PopTarget();
01582     SQInteger eqtarget = trg;
01583     bool local = _fs->IsLocal(trg);
01584     if(local) {
01585         eqtarget = _fs->PushTarget(); //we need to allocate a extra reg
01586     }
01587     _fs->AddInstruction(_OP_EQ, eqtarget, trg, expr);
01588     _fs->AddInstruction(_OP_JZ, eqtarget, 0);
01589     if(local) {
01590         _fs->PopTarget();
01591     }
01592
01593     //end condition
01594     if(skipcondjmp != -1) {
01595         _fs->SetInstructionParam(skipcondjmp, 1, (_fs->GetCurrentPos() - skipcondjmp));
01596     }
01597     tonextcondjmp = _fs->GetCurrentPos();
01598     BEGIN_SCOPE();
01599     Statements();
01600     END_SCOPE();
01601     bfirst = false;
01602 }
01603 if(tonextcondjmp != -1)
01604     _fs->SetInstructionParam(tonextcondjmp, 1, _fs->GetCurrentPos() - tonextcondjmp);
01605 if(_token == TK_DEFAULT) {
01606     Lex(); Expect(_SC(':'));
01607     BEGIN_SCOPE();
01608     Statements();
01609     END_SCOPE();
01610 }
01611 Expect(_SC('}'));
01612 _fs->PopTarget();
01613 __nbreaks__ = _fs->unresolvedbreaks.size() - __nbreaks__;
01614 if(__nbreaks__ > 0) ResolveBreaks(_fs, __nbreaks__);
01615 _fs->_breaktargets.pop_back();
01616 }

```

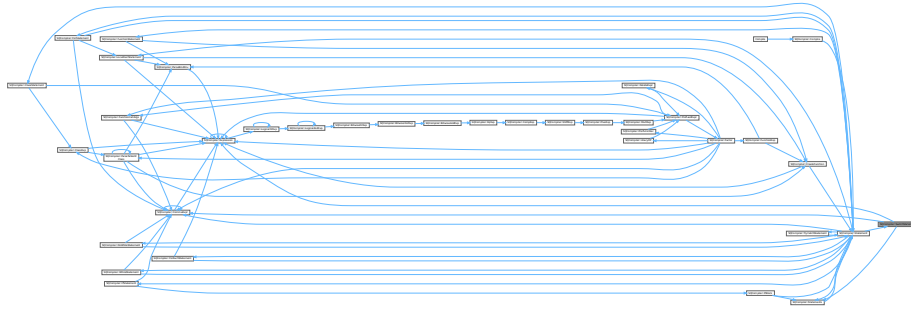
References [_fs](#), [_token](#), [BEGIN_SCOPE](#), [CommaExpr\(\)](#), [END_SCOPE](#), [Expect\(\)](#), [Expression\(\)](#), [Lex\(\)](#), [ResolveBreaks\(\)](#), and [Statements\(\)](#).

Referenced by [Statement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.55 ThrowError()

```
static void SQCompiler::ThrowError (
    void * ud,
    const SQChar * s ) [inline], [static]
```

字句解析器からのエラーを中継する静的コールバック関数。

字句解析器でエラーが発生した際に呼び出され、自身のError メンバ関数に処理を委譲します。

Parameters

<i>ud</i>	SQCompiler インスタンスへのポインタ。
<i>s</i>	エラーメッセージ文字列。

Definition at line 160 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00160 {
00161     SQCompiler *c = (SQCompiler *)ud;
00162     c->Error(s);
00163 }
```

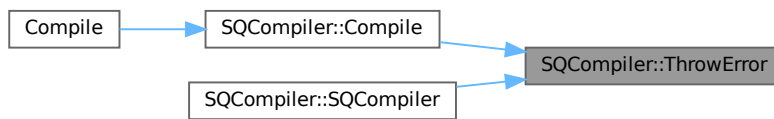
References [Error\(\)](#).

Referenced by [Compile\(\)](#), and [SQCompiler\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.56 TryCatchStatement()

```
void SQCompiler::TryCatchStatement ( ) [inline]
```

try-catch 文を解析します。

`_OP_PUSHTRAP` 命令で例外ハンドラを設定し、try ブロックを解析します。catch ブロックでは、捕捉した例外をローカル変数に束縛し、ブロック内の文を解析します。

Definition at line 1757 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

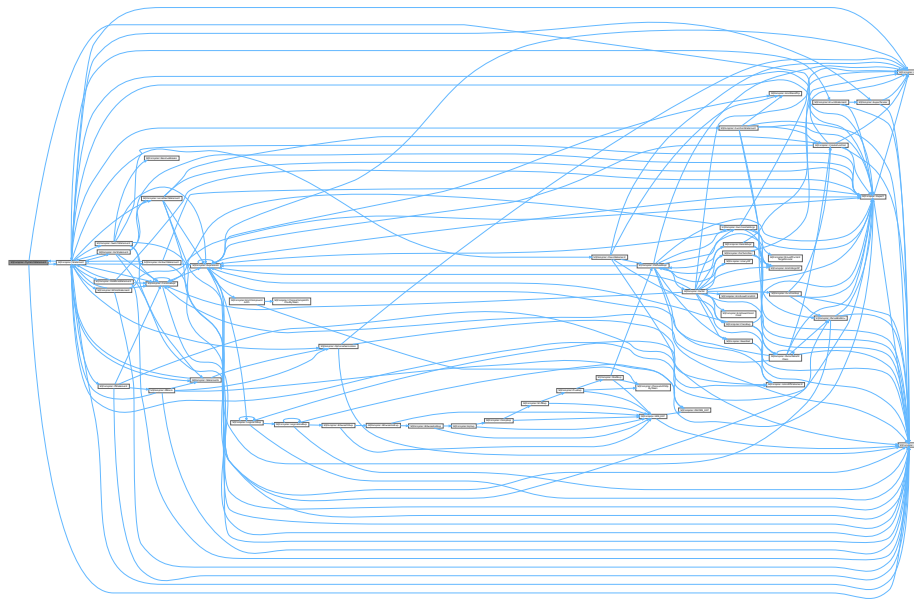
```

1758     {
1759         SQObject exid;
1760         Lex();
1761         _fs->AddInstruction(_OP_PUSHTRAP, 0, 0);
1762         _fs->_traps++;
1763         if(_fs->_breaktargets.size()) _fs->_breaktargets.top()++;
1764         if(_fs->_continuetargets.size()) _fs->_continuetargets.top()++;
1765         SQInteger trappos = _fs->GetCurrentPos();
1766         {
1767             BEGIN_SCOPE();
1768             Statement();
1769             END_SCOPE();
1770         }
1771         _fs->_traps--;
1772         _fs->AddInstruction(_OP_POPTRAP, 1, 0);
1773         if(_fs->_breaktargets.size()) _fs->_breaktargets.top()--;
1774         if(_fs->_continuetargets.size()) _fs->_continuetargets.top()--;
1775         _fs->AddInstruction(_OP_JUMP, 0, 0);
1776         SQInteger jmppos = _fs->GetCurrentPos();
1777         _fs->SetInstructionParam(trappos, 1, (_fs->GetCurrentPos() - trappos));
1778         Expect(TK_CATCH); Expect(_SC('(')); exid = Expect(TK_IDENTIFIER); Expect(_SC(')'));
1779         {
1780             BEGIN_SCOPE();
1781             SQInteger ex_target = _fs->PushLocalVariable(exid);
1782             _fs->SetInstructionParam(trappos, 0, ex_target);
1783             Statement();
1784             _fs->SetInstructionParams(jmppos, 0, (_fs->GetCurrentPos() - jmppos), 0);
1785             END_SCOPE();
1786         }
1787     }
  
```

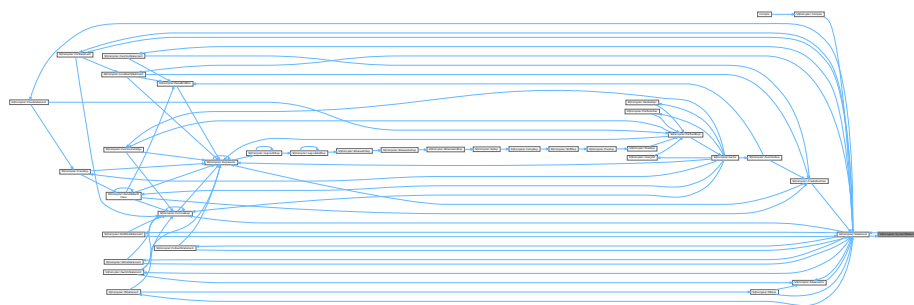
References [_fs](#), [BEGIN_SCOPE](#), [END_SCOPE](#), [Expect\(\)](#), [Lex\(\)](#), and [Statement\(\)](#).

Referenced by [Statement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.57 UnaryOP()

```
void SQCompiler::UnaryOP (
    SQOpcode op ) [inline]
```

単項演算子の命令を生成します。

`PrefixExpr` を呼び出してオペランドを評価し、スタックにプッシュさせます。その後、指定された単項演算オペコードを持つ命令を生成します。

Parameters

<i>op</i>	生成する単項演算のオペコード。
-----------	-----------------

Definition at line 1153 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).
01154 {

```

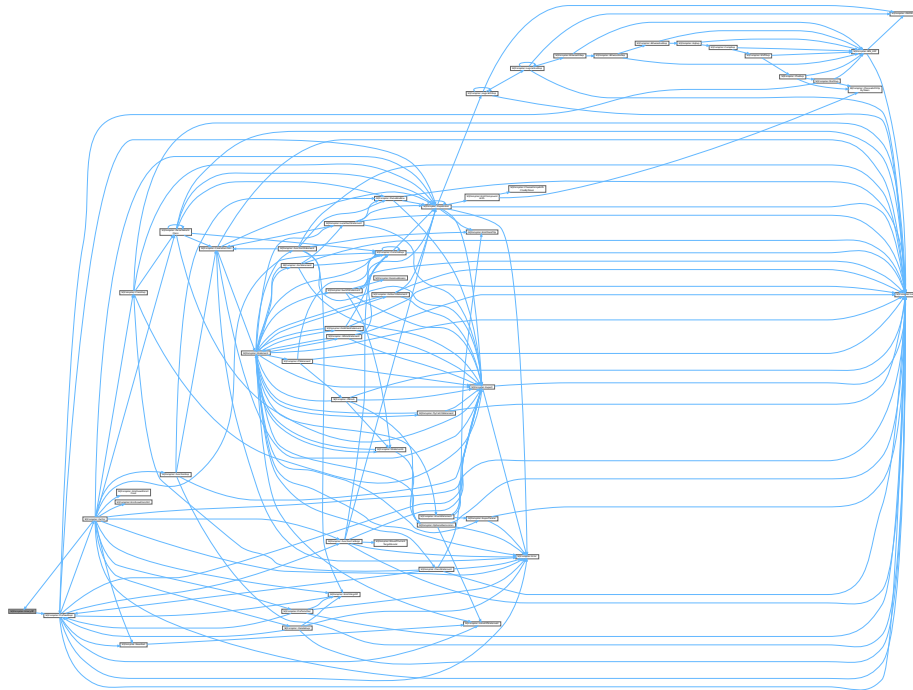
01155     PrefixedExpr();
01156     SQInteger src = _fs->PopTarget();
01157     _fs->AddInstruction(op, _fs->PushTarget(), src);
01158 }

```

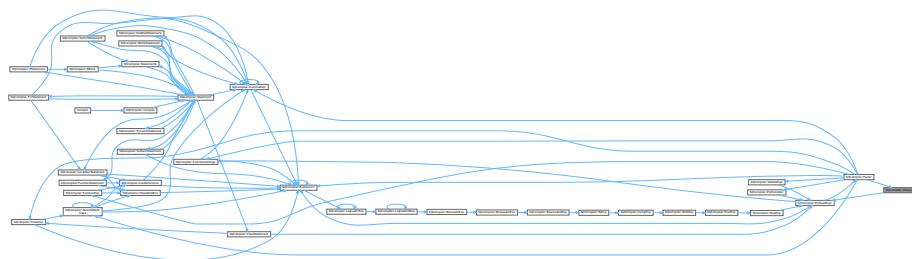
References [_fs](#), and [PrefixedExpr\(\)](#).

Referenced by [Factor\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.58 WhileStatement()

```
void SQCompiler::WhileStatement ( ) [inline]
```

while ループを解析します。

ループ開始前に条件を評価し (`_OP_JZ`)、ループ本体の後にループ先頭への無条件ジャンプ (`_OP_JMP`) を生成します。 `break/continue` にも対応します。

Definition at line 1423 of file squirrel3_squirrel_sqcompiler_v1.cpp.

```

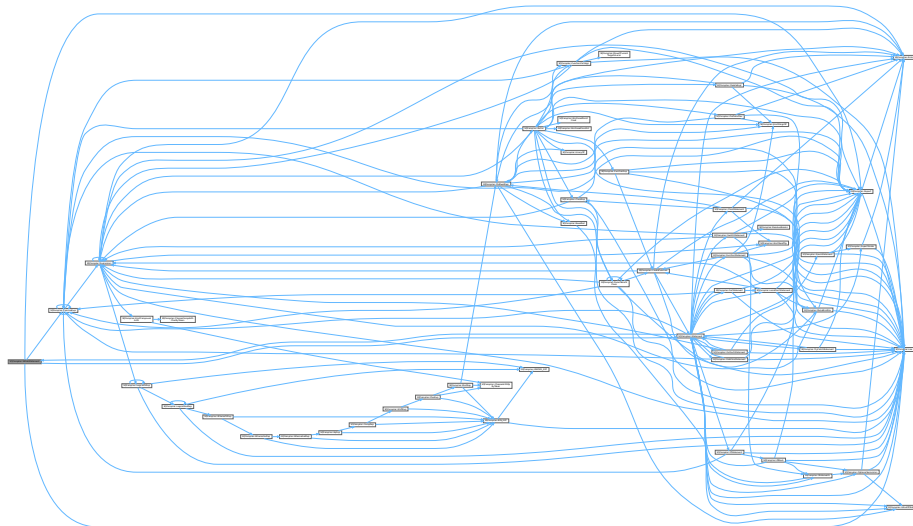
01424     {
01425         SQInteger jzpos, jmppos;
01426         jmppos = _fs->GetCurrentPos();
01427         Lex(); Expect(_SC('(')); CommaExpr(); Expect(_SC(')'));
01428
01429         BEGIN_BREAKABLE_BLOCK();
01430         _fs->AddInstruction(_OP_JZ, _fs->PopTarget());
01431         jzpos = _fs->GetCurrentPos();
01432         BEGIN_SCOPE();
01433
01434         Statement();
01435
01436         END_SCOPE();
01437         _fs->AddInstruction(_OP_JMP, 0, jmppos - _fs->GetCurrentPos() - 1);
01438         _fs->SetInstructionParam(jzpos, 1, _fs->GetCurrentPos() - jzpos);
01439
01440         END_BREAKABLE_BLOCK(jmppos);
01441     }

```

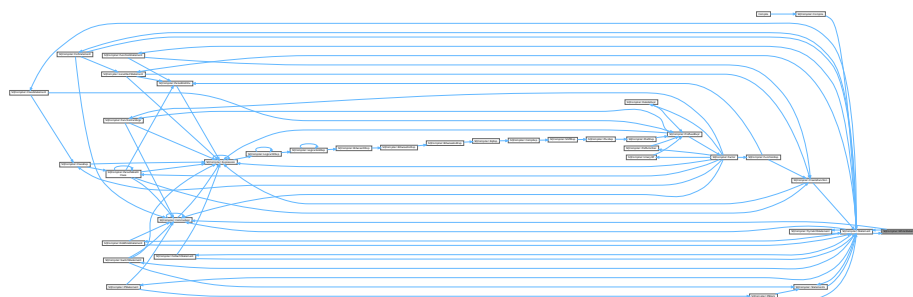
References [_fs](#), [BEGIN_BREAKABLE_BLOCK](#), [BEGIN_SCOPE](#), [CommaExpr\(\)](#), [END_BREAKABLE_BLOCK](#), [END_SCOPE](#), [Expect\(\)](#), [Lex\(\)](#), and [Statement\(\)](#).

Referenced by [Statement\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.4 Field Documentation

3.1.4.1 `_compilererror`

`SQChar SQCompiler::_compilererror[MAX_COMPILER_ERROR_LEN] [private]`

コンパイラエラーメッセージ用バッファ

Definition at line 2014 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

Referenced by [Compile\(\)](#), [Error\(\)](#), and [SQCompiler\(\)](#).

3.1.4.2 `_debugline`

`SQInteger SQCompiler::_debugline [private]`

デバッグ用の行番号

Definition at line 2010 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

Referenced by [Compile\(\)](#).

3.1.4.3 `_debugop`

`SQInteger SQCompiler::_debugop [private]`

デバッグ用のオペコード

Definition at line 2011 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

Referenced by [Compile\(\)](#).

3.1.4.4 `_errorjmp`

`jmp_buf SQCompiler::_errorjmp [private]`

エラー発生時のジャンプ先バッファ

Definition at line 2015 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

Referenced by [Compile\(\)](#), and [Error\(\)](#).

3.1.4.5 `_es`

`SQExpState SQCompiler::_es [private]`

現在の式の状態

Definition at line 2012 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

Referenced by [BIN_EXP\(\)](#), [ClassStatement\(\)](#), [DeleteExpr\(\)](#), [Expression\(\)](#), [Factor\(\)](#), [INVOKE_EXP\(\)](#), [LogicalAndExp\(\)](#), [LogicalOrExp\(\)](#), [NeedGet\(\)](#), [PrefixedExpr\(\)](#), and [PrefixIncDec\(\)](#).

3.1.4.6 `_fs`

```
SQFuncState* SQCompiler::_fs [private]
```

現在の関数状態へのポインタ

Definition at line 2005 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

Referenced by [BIN_EXP\(\)](#), [ClassExp\(\)](#), [ClassStatement\(\)](#), [CommaExpr\(\)](#), [Compile\(\)](#), [CreateFunction\(\)](#), [DoWhileStatement\(\)](#), [Emit2ArgsOP\(\)](#), [EmitCompoundArith\(\)](#), [EmitDerefOp\(\)](#), [EmitLoadConstFloat\(\)](#), [EmitLoadConstInt\(\)](#), [EnumStatement\(\)](#), [Expect\(\)](#), [ExpectScalar\(\)](#), [Expression\(\)](#), [Factor\(\)](#), [ForEachStatement\(\)](#), [ForStatement\(\)](#), [FunctionCallArgs\(\)](#), [FunctionExp\(\)](#), [FunctionStatement\(\)](#), [IfStatement\(\)](#), [LocalDeclStatement\(\)](#), [LogicalAndExp\(\)](#), [LogicalOrExp\(\)](#), [MovelfCurrentTargetIsLocal\(\)](#), [ParseBindEnv\(\)](#), [ParseTableOrClass\(\)](#), [PrefixedExpr\(\)](#), [PrefixIncDec\(\)](#), [Statement\(\)](#), [SwitchStatement\(\)](#), [TryCatchStatement\(\)](#), [UnaryOP\(\)](#), and [WhileStatement\(\)](#).

3.1.4.7 `_lex`

```
SQLexer SQCompiler::_lex [private]
```

字句解析器インスタンス

Definition at line 2007 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

Referenced by [Compile\(\)](#), [CreateFunction\(\)](#), [Expect\(\)](#), [ExpectScalar\(\)](#), [Factor\(\)](#), [IfBlock\(\)](#), [IsEndOfStatement\(\)](#), [Lex\(\)](#), [PrefixedExpr\(\)](#), [SQCompiler\(\)](#), [Statement\(\)](#), and [Statements\(\)](#).

3.1.4.8 `_lineinfo`

```
bool SQCompiler::_lineinfo [private]
```

行情報を生成するかどうかのフラグ

Definition at line 2008 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

Referenced by [Compile\(\)](#), [CreateFunction\(\)](#), [SQCompiler\(\)](#), and [Statement\(\)](#).

3.1.4.9 `_raiseerror`

```
bool SQCompiler::_raiseerror [private]
```

エラーハンドラを呼び出すかどうかのフラグ

Definition at line 2009 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

Referenced by [Compile\(\)](#), and [SQCompiler\(\)](#).

3.1.4.10 `_scope`

```
SQScope SQCompiler::_scope [private]
```

現在のスコープ情報

Definition at line 2013 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

Referenced by [SQCompiler\(\)](#).

3.1.4.11 `_sourcename`

`SQObjectPtr SQCompiler::_sourcename [private]`

ソース名

Definition at line 2006 of file `squirrel3_squirrel_sqcompiler_v1.cpp`.

Referenced by `Compile()`, `CreateFunction()`, `Factor()`, and `SQCompiler()`.

3.1.4.12 `_token`

`SQInteger SQCompiler::_token [private]`

現在のトークン

Definition at line 2004 of file `squirrel3_squirrel_sqcompiler_v1.cpp`.

Referenced by `BitwiseAndExp()`, `BitwiseOrExp()`, `BitwiseXorExp()`, `ClassExp()`, `CommaExpr()`, `CompExp()`, `Compile()`, `CreateFunction()`, `EnumStatement()`, `EqExp()`, `Expect()`, `ExpectScalar()`, `Expression()`, `Factor()`, `ForEachStatement()`, `ForStatement()`, `FunctionCallArgs()`, `FunctionExp()`, `FunctionStatement()`, `IfBlock()`, `IfStatement()`, `IsEndOfStatement()`, `Lex()`, `LocalDeclStatement()`, `LogicalAndExp()`, `LogicalOrExp()`, `MultExp()`, `NeedGet()`, `OptionalSemicolon()`, `ParseTableOrClass()`, `PlusExp()`, `PrefixedExpr()`, `ShiftExp()`, `Statement()`, `Statements()`, and `SwitchStatement()`.

3.1.4.13 `_vm`

`SQVM* SQCompiler::_vm [private]`

Squirrel VM へのポインタ

Definition at line 2016 of file `squirrel3_squirrel_sqcompiler_v1.cpp`.

Referenced by `Compile()`, `CreateFunction()`, `EnumStatement()`, `SQCompiler()`, and `Statement()`.

The documentation for this class was generated from the following file:

- `squirrel3_squirrel_sqcompiler_v1.cpp`

3.2 SQExpState Struct Reference

式の状態を管理する構造体。

Data Fields

- `SQInteger etype`
式の型。
- `SQInteger epos`
スタック上の式の位置。
- `bool donot_get`
デリファレンスを抑制するフラグ。

3.2.1 Detailed Description

式の状態を管理する構造体。

コンパイラが式を評価する際の、式の種類やスタック上の位置などの状態を保持します。

Definition at line 29 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

3.2.2 Field Documentation

3.2.2.1 donot_get

```
bool SQExpState::donot_get
```

デリファレンスを抑制するフラグ。

true の場合、次の値のデリファレンスを行いません。

Definition at line 44 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

Referenced by [SQCompiler::ClassStatement\(\)](#), [SQCompiler::DeleteExpr\(\)](#), [SQCompiler::Expression\(\)](#), [SQCompiler::INVOKE_EXP\(\)](#), [SQCompiler::NeedGet\(\)](#), [SQCompiler::PrefixedExpr\(\)](#), and [SQCompiler::PrefixIncDec\(\)](#).

3.2.2.2 epos

```
SQInteger SQExpState::epos
```

スタック上の式の位置。

OBJECT 型とBASE 型の場合は -1 になります。

Definition at line 39 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

Referenced by [SQCompiler::Expression\(\)](#), [SQCompiler::Factor\(\)](#), [SQCompiler::INVOKE_EXP\(\)](#), [SQCompiler::PrefixedExpr\(\)](#), and [SQCompiler::PrefixIncDec\(\)](#).

3.2.2.3 etype

```
SQInteger SQExpState::etype
```

式の型。

EXPR, OBJECT, BASE, OUTER, LOCAL のいずれかの値を取ります。

Definition at line 34 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

Referenced by [SQCompiler::BIN_EXP\(\)](#), [SQCompiler::ClassStatement\(\)](#), [SQCompiler::DeleteExpr\(\)](#), [SQCompiler::Expression\(\)](#), [SQCompiler::Factor\(\)](#), [SQCompiler::INVOKE_EXP\(\)](#), [SQCompiler::LogicalAndExp\(\)](#), [SQCompiler::LogicalOrExp\(\)](#), [SQCompiler::PrefixedExpr\(\)](#), and [SQCompiler::PrefixIncDec\(\)](#).

The documentation for this struct was generated from the following file:

- [squirrel3_squirrel_sqcompiler_v1.cpp](#)

3.3 SQScope Struct Reference

コンパイル時のスコープ情報を保持する構造体。

Data Fields

- SQInteger [outers](#)
アウター変数の数
- SQInteger [stacksize](#)
スタックサイズ

3.3.1 Detailed Description

コンパイル時のスコープ情報を保持する構造体。

現在のスコープのアウター変数の数とスタックサイズを管理します。

Definition at line 56 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

3.3.2 Field Documentation

3.3.2.1 outers

SQInteger SQScope::outers

アウター変数の数

Definition at line 57 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

Referenced by [SQCompiler::SQCompiler\(\)](#).

3.3.2.2 stacksize

SQInteger SQScope::stacksize

スタックサイズ

Definition at line 58 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

Referenced by [SQCompiler::SQCompiler\(\)](#).

The documentation for this struct was generated from the following file:

- [squirrel3_squirrel_sqcompiler_v1.cpp](#)

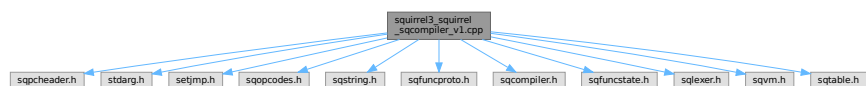
Chapter 4

File Documentation

4.1 squirrel3_squirrel_sqcompiler_v1.cpp File Reference

```
#include "sqpcheader.h"
#include <stdarg.h>
#include <setjmp.h>
#include "sqopcodes.h"
#include "sqstring.h"
#include "sqfuncproto.h"
#include "sqcompiler.h"
#include "sqfuncstate.h"
#include "sqlexer.h"
#include "sqvm.h"
#include "sqtable.h"
```

Include dependency graph for squirrel3_squirrel_sqcompiler_v1.cpp:



Data Structures

- struct [SQExpState](#)
式の状態を管理する構造体。
- struct [SQScope](#)
コンパイル時のスコープ情報を保持する構造体。
- class [SQCompiler](#)
Squirrel スクリプトをバイトコードにコンパイルするクラス。

Macros

- `#define` [EXPR](#) 1
- `#define` [OBJECT](#) 2
- `#define` [BASE](#) 3
- `#define` [LOCAL](#) 4
- `#define` [OUTER](#) 5
- `#define` [MAX_COMPILER_ERROR_LEN](#) 256
コンパイラエラーメッセージの最大長。
- `#define` [BEGIN_SCOPE](#)()
新しいスコープを開始するためのマクロ。
- `#define` [RESOLVE_OUTERS](#)()
アウター変数を解決するためのマクロ。
- `#define` [END_SCOPE_NO_CLOSE](#)()
スコープを終了するマクロ（クローズ命令なし）。
- `#define` [END_SCOPE](#)()
スコープを終了するマクロ。
- `#define` [BEGIN_BREAKBLE_BLOCK](#)()
break/continue が可能なブロックを開始するためのマクロ。
- `#define` [END_BREAKBLE_BLOCK](#)(continue_target)
break/continue が可能なブロックを終了するためのマクロ。

Functions

- `bool` [Compile](#) (SQVM *vm, SQLEXREADFUNC rg, SQUserPointer up, const SQChar *sourcename, SQObjectPtr &out, bool raiseerror, bool lineinfo)
Squirrel スクリプトをコンパイルするグローバル関数。

4.1.1 Macro Definition Documentation

4.1.1.1 BASE

```
#define BASE 3
```

Definition at line 21 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

4.1.1.2 BEGIN_BREAKBLE_BLOCK

```
#define BEGIN_BREAKBLE_BLOCK( )
```

Value:

```
SQInteger __nbreaks__=_fs->_unresolvedbreaks.size(); \
SQInteger __ncontinues__=_fs->_unresolvedcontinues.size(); \
_fs->_breaktargets.push_back(0);_fs->_continuetargets.push_back(0);
```

break/continue が可能なブロックを開始するためのマクロ。

ループなどのブロックの開始時に、未解決の *break/continue* の数を保存し、新しい *break/continue* ターゲットをスタックに追加します。

Definition at line 111 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00121 {__nbreaks__=_fs->_unresolvedbreaks.size()-__nbreaks__; \
```

```

00122         __ncontinues__=_fs->_unresolvedcontinues.size()-__ncontinues__; \
00123         if(__ncontinues__>0) ResolveContinues(_fs,__ncontinues__,continue_target); \
00124         if(__nbreaks__>0) ResolveBreaks(_fs,__nbreaks__); \
00125         _fs->_breaktargets.pop_back();_fs->_continuetargets.pop_back();}
00126
00131 class SQCompiler
00132 {
00133 public:
00144     SQCompiler(SQVM *v, SQLEXREADFUNC rg, SQUserPointer up, const SQChar* sourcename, bool raiseerror,
00145         bool lineinfo)
00146     {
00147         _vm=v;
00148         _lex.Init(_ss(v), rg, up,ThrowError,this);
00149         _sourcename = SQString::Create(_ss(v), sourcename);
00150         _lineinfo = lineinfo;_raiseerror = raiseerror;
00151         _scope.outers = 0;
00152         _scope.stacksize = 0;
00153         _compilererror[0] = _SC('\0');
00154     }
00160     static void ThrowError(void *ud, const SQChar *s) {
00161         SQCompiler *c = (SQCompiler *)ud;
00162         c->Error(s);
00163     }
00171     void Error(const SQChar *s, ...)
00172     {
00173         va_list vl;
00174         va_start(vl, s);
00175         scvsprintf(_compilererror, MAX_COMPILER_ERROR_LEN, s, vl);
00176         va_end(vl);
00177         longjmp(_errorjmp,1);
00178     }
00183     void Lex(){ _token = _lex.Lex();}
00192     SQObject Expect(SQInteger tok)
00193     {
00194
00195         if(_token != tok) {
00196             if(_token == TK_CONSTRUCTOR && tok == TK_IDENTIFIER) {
00197                 //do nothing
00198             }
00199             else {
00200                 const SQChar *etyponame;
00201                 if(tok > 255) {
00202                     switch(tok)
00203                     {
00204                         case TK_IDENTIFIER:
00205                             etyponame = _SC("IDENTIFIER");
00206                             break;
00207                         case TK_STRING_LITERAL:
00208                             etyponame = _SC("STRING_LITERAL");
00209                             break;
00210                         case TK_INTEGER:
00211                             etyponame = _SC("INTEGER");
00212                             break;
00213                         case TK_FLOAT:
00214                             etyponame = _SC("FLOAT");
00215                             break;
00216                         default:
00217                             etyponame = _lex.Tok2Str(tok);
00218                     }
00219                     Error(_SC("expected '%s'"), etyponame);
00220                 }
00221                 Error(_SC("expected '%c'"), tok);
00222             }
00223         }
00224         SQObjectPtr ret;
00225         switch(tok)
00226         {
00227             case TK_IDENTIFIER:
00228                 ret = _fs->CreateString(_lex._svalue);
00229                 break;
00230             case TK_STRING_LITERAL:
00231                 ret = _fs->CreateString(_lex._svalue,_lex._longstr.size()-1);
00232                 break;
00233             case TK_INTEGER:
00234                 ret = SQObjectPtr(_lex._nvalue);
00235                 break;
00236             case TK_FLOAT:
00237                 ret = SQObjectPtr(_lex._fvalue);
00238                 break;
00239         }
00240         Lex();
00241         return ret;
00242     }
00248     bool IsEndOfStatement() { return ((_lex._prevtoken == _SC('\n')) || (_token == SQUIRREL_EOB) ||
00249         (_token == _SC('}')) || (_token == _SC(';'))); }
00254     void OptionalSemicolon()
00255     {

```

```

00256         if(_token == _SC(';')) { Lex(); return; }
00257         if(!IsEndOfStatement()) {
00258             Error(_SC("end of statement expected (; or lf)"));
00259         }
00260     }
00261     void MoveIfCurrentTargetIsLocal() {
00262         SQInteger trg = _fs->TopTarget();
00263         if(_fs->IsLocal(trg)) {
00264             trg = _fs->PopTarget(); //pops the target and moves it
00265             _fs->AddInstruction(_OP_MOVE, _fs->PushTarget(), trg);
00266         }
00267     }
00268     bool Compile(SQObjectPtr &o)
00269     {
00270         _debugline = 1;
00271         _debugop = 0;
00272
00273         SQFuncState funcstate(_ss(_vm), NULL, ThrowError, this);
00274         funcstate._name = SQString::Create(_ss(_vm), _SC("main"));
00275         _fs = &funcstate;
00276         _fs->AddParameter(_fs->CreateString(_SC("this")));
00277         _fs->AddParameter(_fs->CreateString(_SC("vargv")));
00278         _fs->_varparams = true;
00279         _fs->_sourcename = _sourcename;
00280         SQInteger stacksize = _fs->GetStackSize();
00281         if(setjmp(_errorjmp) == 0) {
00282             Lex();
00283             while(_token > 0){
00284                 Statement();
00285                 if(_lex._prevtoken != _SC(';')) && _lex._prevtoken != _SC(';')) OptionalSemicolon();
00286             }
00287             _fs->SetStackSize(stacksize);
00288             _fs->AddLineInfos(_lex._currentline, _lineinfo, true);
00289             _fs->AddInstruction(_OP_RETURN, 0xFF);
00290             _fs->SetStackSize(0);
00291             o = _fs->BuildProto();
00292 #ifdef _DEBUG_DUMP
00293             _fs->Dump(_funcproto(o));
00294 #endif
00295         }
00296         else {
00297             if(_raiseerror && _ss(_vm)->_compilererrorhandler) {
00298                 _ss(_vm)->_compilererrorhandler(_vm, _compilererror, sq_type(_sourcename) ==
00299 OT_STRING?_stringval(_sourcename):_SC("unknown"),
00300                 _lex._currentline, _lex._currentcolumn);
00301             }
00302             _vm->_lasterror = SQString::Create(_ss(_vm), _compilererror, -1);
00303             return false;
00304         }
00305         return true;
00306     }
00307     void Statements()
00308     {
00309         while(_token != _SC(';')) && _token != TK_DEFAULT && _token != TK_CASE) {
00310             Statement();
00311             if(_lex._prevtoken != _SC(';')) && _lex._prevtoken != _SC(';')) OptionalSemicolon();
00312         }
00313     }
00314     void Statement(bool closeframe = true)
00315     {
00316         _fs->AddLineInfos(_lex._currentline, _lineinfo);
00317         switch(_token){
00318             case _SC(';'): Lex(); break;
00319             case TK_IF: IfStatement(); break;
00320             case TK_WHILE: WhileStatement(); break;
00321             case TK_DO: DoWhileStatement(); break;
00322             case TK_FOR: ForStatement(); break;
00323             case TK_FOREACH: ForEachStatement(); break;
00324             case TK_SWITCH: SwitchStatement(); break;
00325             case TK_LOCAL: LocalDeclStatement(); break;
00326             case TK_RETURN:
00327             case TK_YIELD: {
00328                 SQOpcode op;
00329                 if(_token == TK_RETURN) {
00330                     op = _OP_RETURN;
00331                 }
00332                 else {
00333                     op = _OP_YIELD;
00334                     _fs->_bgenerator = true;
00335                 }
00336                 Lex();
00337                 if(!IsEndOfStatement()) {
00338                     SQInteger retexp = _fs->GetCurrentPos()+1;
00339                     CommaExpr();
00340                     if(op == _OP_RETURN && _fs->_traps > 0)
00341                         _fs->AddInstruction(_OP_POPTRAP, _fs->_traps, 0);
00342                     _fs->_returnexp = retexp;
00343                 }
00344             }
00345         }
00346     }

```

```

00363         _fs->AddInstruction(op, 1, _fs->PopTarget(), _fs->GetStackSize());
00364     }
00365     else{
00366         if(op == _OP_RETURN && _fs->_traps > 0)
00367             _fs->AddInstruction(_OP_POPTRAP, _fs->_traps, 0);
00368         _fs->_returnexp = -1;
00369         _fs->AddInstruction(op, 0xFF, 0, _fs->GetStackSize());
00370     }
00371     break;}
00372 case TK_BREAK:
00373     if(_fs->_breaktargets.size() <= 0)Error(_SC("'break' has to be in a loop block"));
00374     if(_fs->_breaktargets.top() > 0){
00375         _fs->AddInstruction(_OP_POPTRAP, _fs->_breaktargets.top(), 0);
00376     }
00377     RESOLVE_OUTERS();
00378     _fs->AddInstruction(_OP_JMP, 0, -1234);
00379     _fs->_unresolvedbreaks.push_back(_fs->GetCurrentPos());
00380     Lex();
00381     break;
00382 case TK_CONTINUE:
00383     if(_fs->_continuetargets.size() <= 0)Error(_SC("'continue' has to be in a loop block"));
00384     if(_fs->_continuetargets.top() > 0){
00385         _fs->AddInstruction(_OP_POPTRAP, _fs->_continuetargets.top(), 0);
00386     }
00387     RESOLVE_OUTERS();
00388     _fs->AddInstruction(_OP_JMP, 0, -1234);
00389     _fs->_unresolvedcontinues.push_back(_fs->GetCurrentPos());
00390     Lex();
00391     break;
00392 case TK_FUNCTION:
00393     FunctionStatement();
00394     break;
00395 case TK_CLASS:
00396     ClassStatement();
00397     break;
00398 case TK_ENUM:
00399     EnumStatement();
00400     break;
00401 case _SC('{'): {
00402     BEGIN_SCOPE();
00403     Lex();
00404     Statements();
00405     Expect(_SC('}'));
00406     if(closeframe) {
00407         END_SCOPE();
00408     }
00409     else {
00410         END_SCOPE_NO_CLOSE();
00411     }
00412 }
00413 break;
00414 case TK_TRY:
00415     TryCatchStatement();
00416     break;
00417 case TK_THROW:
00418     Lex();
00419     CommaExpr();
00420     _fs->AddInstruction(_OP_THROW, _fs->PopTarget());
00421     break;
00422 case TK_CONST:
00423     {
00424         Lex();
00425         SQObject id = Expect(TK_IDENTIFIER);
00426         Expect('=');
00427         SQObject val = ExpectScalar();
00428         OptionalSemicolon();
00429         SQTable *enums = _table(_ss(_vm)->_consts);
00430         SQObjectPtr strongid = id;
00431         enums->NewSlot(strongid, SQObjectPtr(val));
00432         strongid.Null();
00433     }
00434     break;
00435 default:
00436     CommaExpr();
00437     _fs->DiscardTarget();
00438     //_fs->PopTarget();
00439     break;
00440 }
00441 _fs->SnoozeOpt();
00442 }
00443 void EmitDerefOp(SQOpcode op)
00444 {
00450     SQInteger val = _fs->PopTarget();
00451     SQInteger key = _fs->PopTarget();
00452     SQInteger src = _fs->PopTarget();
00453     _fs->AddInstruction(op, _fs->PushTarget(), src, key, val);
00454 }

```

```

00461 void Emit2ArgsOP(SQOpcode op, SQInteger p3 = 0)
00462 {
00463     SQInteger p2 = _fs->PopTarget(); //src in OP_GET
00464     SQInteger p1 = _fs->PopTarget(); //key in OP_GET
00465     _fs->AddInstruction(op, _fs->PushTarget(), p1, p2, p3);
00466 }
00474 void EmitCompoundArith(SQInteger tok, SQInteger etype, SQInteger pos)
00475 {
00476     /* Generate code depending on the expression type */
00477     switch(etype) {
00478     case LOCAL:
00479         SQInteger p2 = _fs->PopTarget(); //src in OP_GET
00480         SQInteger p1 = _fs->PopTarget(); //key in OP_GET
00481         _fs->PushTarget(p1);
00482         //EmitCompArithLocal(tok, p1, p1, p2);
00483         _fs->AddInstruction(ChooseArithOpByToken(tok), p1, p2, p1, 0);
00484         _fs->SnoozeOpt();
00485     }
00486     break;
00487     case OBJECT:
00488     case BASE:
00489     {
00490         SQInteger val = _fs->PopTarget();
00491         SQInteger key = _fs->PopTarget();
00492         SQInteger src = _fs->PopTarget();
00493         /* _OP_COMPARITH mixes dest obj and source val in the arg1 */
00494         _fs->AddInstruction(_OP_COMPARITH, _fs->PushTarget(), (src<<16)|val, key,
ChooseCompArithCharByToken(tok));
00495     }
00496     break;
00497     case OUTER:
00498     {
00499         SQInteger val = _fs->TopTarget();
00500         SQInteger tmp = _fs->PushTarget();
00501         _fs->AddInstruction(_OP_GETOUTER, tmp, pos);
00502         _fs->AddInstruction(ChooseArithOpByToken(tok), tmp, val, tmp, 0);
00503         _fs->PopTarget();
00504         _fs->PopTarget();
00505         _fs->AddInstruction(_OP_SETOUTER, _fs->PushTarget(), pos, tmp);
00506     }
00507     break;
00508 }
00509 }
00515 void CommaExpr()
00516 {
00517     for(Expression(); _token == ','; _fs->PopTarget(), Lex(), CommaExpr());
00518 }
00524 void Expression()
00525 {
00526     SQExpState es = _es;
00527     _es.etype = EXPR;
00528     _es.epos = -1;
00529     _es.donot_get = false;
00530     LogicalOrExp();
00531     switch(_token) {
00532     case _SC('='):
00533     case TK_NEWSLT:
00534     case TK_MINUSEQ:
00535     case TK_PLUSEQ:
00536     case TK_MULEQ:
00537     case TK_DIVEQ:
00538     case TK_MODEQ:
00539         SQInteger op = _token;
00540         SQInteger ds = _es.etype;
00541         SQInteger pos = _es.epos;
00542         if(ds == EXPR) Error(_SC("can't assign expression"));
00543         else if(ds == BASE) Error(_SC("'base' cannot be modified"));
00544         Lex(); Expression();
00545     }
00546     switch(op) {
00547     case TK_NEWSLT:
00548         if(ds == OBJECT || ds == BASE)
00549             EmitDerefOp(_OP_NEWSLT);
00550         else //if _derefstate != DEREf_NO_DEREF && DEREf_FIELD so is the index of a local
00551             Error(_SC("can't 'create' a local slot"));
00552         break;
00553     case _SC('='): //ASSIGN
00554         switch(ds) {
00555         case LOCAL:
00556         {
00557             SQInteger src = _fs->PopTarget();
00558             SQInteger dst = _fs->TopTarget();
00559             _fs->AddInstruction(_OP_MOVE, dst, src);
00560         }
00561         break;
00562     case OBJECT:
00563     case BASE:

```

```

00564         EmitDerefOp(_OP_SET);
00565         break;
00566     case OUTER:
00567     {
00568         SQInteger src = _fs->PopTarget();
00569         SQInteger dst = _fs->PushTarget();
00570         _fs->AddInstruction(_OP_SETOUTER, dst, pos, src);
00571     }
00572 }
00573 break;
00574 case TK_MINUSEQ:
00575 case TK_PLUSEQ:
00576 case TK_MULEQ:
00577 case TK_DIVEQ:
00578 case TK_MODEQ:
00579     EmitCompoundArith(op, ds, pos);
00580     break;
00581 }
00582 }
00583 break;
00584 case _SC('?'): {
00585     Lex();
00586     _fs->AddInstruction(_OP_JZ, _fs->PopTarget());
00587     SQInteger jzpos = _fs->GetCurrentPos();
00588     SQInteger trg = _fs->PushTarget();
00589     Expression();
00590     SQInteger first_exp = _fs->PopTarget();
00591     if(trg != first_exp) _fs->AddInstruction(_OP_MOVE, trg, first_exp);
00592     SQInteger endfirstexp = _fs->GetCurrentPos();
00593     _fs->AddInstruction(_OP_JMP, 0, 0);
00594     Expect(_SC(':'));
00595     SQInteger jmppos = _fs->GetCurrentPos();
00596     Expression();
00597     SQInteger second_exp = _fs->PopTarget();
00598     if(trg != second_exp) _fs->AddInstruction(_OP_MOVE, trg, second_exp);
00599     _fs->SetInstructionParam(jmppos, 1, _fs->GetCurrentPos() - jmppos);
00600     _fs->SetInstructionParam(jzpos, 1, endfirstexp - jzpos + 1);
00601     _fs->SnoozeOpt();
00602 }
00603 break;
00604 }
00605 _es = es;
00606 }
00614 template<typename T> void INVOKE_EXP(T f)
00615 {
00616     SQExpState es = _es;
00617     _es.etype = EXPR;
00618     _es.epos = -1;
00619     _es.donot_get = false;
00620     (this->*f)();
00621     _es = es;
00622 }
00632 template<typename T> void BIN_EXP(SQOpcode op, T f, SQInteger op3 = 0)
00633 {
00634     Lex();
00635     INVOKE_EXP(f);
00636     SQInteger op1 = _fs->PopTarget(); SQInteger op2 = _fs->PopTarget();
00637     _fs->AddInstruction(op, _fs->PushTarget(), op1, op2, op3);
00638     _es.etype = EXPR;
00639 }
00644 void LogicalOrExp()
00645 {
00646     LogicalAndExp();
00647     for(;;) if(_token == TK_OR) {
00648         SQInteger first_exp = _fs->PopTarget();
00649         SQInteger trg = _fs->PushTarget();
00650         _fs->AddInstruction(_OP_OR, trg, 0, first_exp, 0);
00651         SQInteger jpos = _fs->GetCurrentPos();
00652         if(trg != first_exp) _fs->AddInstruction(_OP_MOVE, trg, first_exp);
00653         Lex(); INVOKE_EXP(&SQCompiler::LogicalOrExp);
00654         _fs->SnoozeOpt();
00655         SQInteger second_exp = _fs->PopTarget();
00656         if(trg != second_exp) _fs->AddInstruction(_OP_MOVE, trg, second_exp);
00657         _fs->SnoozeOpt();
00658         _fs->SetInstructionParam(jpos, 1, (_fs->GetCurrentPos() - jpos));
00659         _es.etype = EXPR;
00660         break;
00661     }else return;
00662 }
00667 void LogicalAndExp()
00668 {
00669     BitwiseOrExp();
00670     for(;;) switch(_token) {
00671     case TK_AND: {
00672         SQInteger first_exp = _fs->PopTarget();
00673         SQInteger trg = _fs->PushTarget();
00674         _fs->AddInstruction(_OP_AND, trg, 0, first_exp, 0);

```

```

00675         SQInteger jpos = _fs->GetCurrentPos();
00676         if(trg != first_exp) _fs->AddInstruction(_OP_MOVE, trg, first_exp);
00677         Lex(); INVOKE_EXP(&SQCompiler::LogicalAndExp);
00678         _fs->SnoozeOpt();
00679         SQInteger second_exp = _fs->PopTarget();
00680         if(trg != second_exp) _fs->AddInstruction(_OP_MOVE, trg, second_exp);
00681         _fs->SnoozeOpt();
00682         _fs->SetInstructionParam(jpos, 1, (_fs->GetCurrentPos() - jpos));
00683         _es.etype = EXPR;
00684         break;
00685     }
00686
00687     default:
00688         return;
00689 }
00690
00691 void BitwiseOrExp()
00692 {
00693     BitwiseXorExp();
00694     for(;;) if(_token == _SC('|'))
00695     {BIN_EXP(_OP_BITW, &SQCompiler::BitwiseXorExp, BW_OR);
00696     }else return;
00697 }
00698
00699 void BitwiseXorExp()
00700 {
00701     BitwiseAndExp();
00702     for(;;) if(_token == _SC('^'))
00703     {BIN_EXP(_OP_BITW, &SQCompiler::BitwiseAndExp, BW_XOR);
00704     }else return;
00705 }
00706
00707 void BitwiseAndExp()
00708 {
00709     EqExp();
00710     for(;;) if(_token == _SC('&'))
00711     {BIN_EXP(_OP_BITW, &SQCompiler::EqExp, BW_AND);
00712     }else return;
00713 }
00714
00715 void EqExp()
00716 {
00717     CompExp();
00718     for(;;) switch(_token) {
00719     case TK_EQ: BIN_EXP(_OP_EQ, &SQCompiler::CompExp); break;
00720     case TK_NE: BIN_EXP(_OP_NE, &SQCompiler::CompExp); break;
00721     case TK_3WAYSCMP: BIN_EXP(_OP_CMP, &SQCompiler::CompExp, CMP_3W); break;
00722     default: return;
00723     }
00724 }
00725
00726 void CompExp()
00727 {
00728     ShiftExp();
00729     for(;;) switch(_token) {
00730     case _SC('>'): BIN_EXP(_OP_CMP, &SQCompiler::ShiftExp, CMP_G); break;
00731     case _SC('<'): BIN_EXP(_OP_CMP, &SQCompiler::ShiftExp, CMP_L); break;
00732     case TK_GE: BIN_EXP(_OP_CMP, &SQCompiler::ShiftExp, CMP_GE); break;
00733     case TK_LE: BIN_EXP(_OP_CMP, &SQCompiler::ShiftExp, CMP_LE); break;
00734     case TK_IN: BIN_EXP(_OP_EXISTS, &SQCompiler::ShiftExp); break;
00735     case TK_INSTANCEOF: BIN_EXP(_OP_INSTANCEOF, &SQCompiler::ShiftExp); break;
00736     default: return;
00737     }
00738 }
00739
00740 void ShiftExp()
00741 {
00742     PlusExp();
00743     for(;;) switch(_token) {
00744     case TK_USHIFTR: BIN_EXP(_OP_BITW, &SQCompiler::PlusExp, BW_USHIFTR); break;
00745     case TK_SHIFTL: BIN_EXP(_OP_BITW, &SQCompiler::PlusExp, BW_SHIFTL); break;
00746     case TK_SHIFTR: BIN_EXP(_OP_BITW, &SQCompiler::PlusExp, BW_SHIFTR); break;
00747     default: return;
00748     }
00749 }
00750
00751 SQOpcode ChooseArithOpByToken(SQInteger tok)
00752 {
00753     switch(tok) {
00754     case TK_PLUSEQ: case '+': return _OP_ADD;
00755     case TK_MINUSEQ: case '-': return _OP_SUB;
00756     case TK_MULEQ: case '*': return _OP_MUL;
00757     case TK_DIVEQ: case '/': return _OP_DIV;
00758     case TK_MODEQ: case '%': return _OP_MOD;
00759     default: assert(0);
00760     }
00761     return _OP_ADD;
00762 }
00763
00764 SQInteger ChooseCompArithCharByToken(SQInteger tok)
00765 {
00766     SQInteger oper;
00767     switch(tok){
00768     case TK_MINUSEQ: oper = '-'; break;
00769 }

```

```

00798         case TK_PLUSEQ: oper = '+'; break;
00799         case TK_MULEQ: oper = '*'; break;
00800         case TK_DIVEQ: oper = '/'; break;
00801         case TK_MODEQ: oper = '%'; break;
00802         default: oper = 0; //shut up compiler
00803             assert(0); break;
00804     };
00805     return oper;
00806 }
00811 void PlusExp()
00812 {
00813     MultExp();
00814     for(;;) switch(_token) {
00815         case _SC('+'): case _SC('--'):
00816             BIN_EXP(ChooseArithOpByToken(_token), &SQCompiler::MultExp); break;
00817         default: return;
00818     }
00819 }
00820
00825 void MultExp()
00826 {
00827     PrefixedExpr();
00828     for(;;) switch(_token) {
00829         case _SC('*'): case _SC('/'): case _SC('%'):
00830             BIN_EXP(ChooseArithOpByToken(_token), &SQCompiler::PrefixedExpr); break;
00831         default: return;
00832     }
00833 }
00839 void PrefixedExpr()
00840 {
00841     SQInteger pos = Factor();
00842     for(;;) {
00843         switch(_token) {
00844             case _SC('.'):
00845                 pos = -1;
00846                 Lex();
00847
00848                 _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
00849 _fs->GetConstant(Expect(TK_IDENTIFIER)));
00849                 if(_es.etype==BASE) {
00850                     Emit2ArgsOP(_OP_GET);
00851                     pos = _fs->TopTarget();
00852                     _es.etype = EXPR;
00853                     _es.epos = pos;
00854                 }
00855                 else {
00856                     if(NeedGet()) {
00857                         Emit2ArgsOP(_OP_GET);
00858                     }
00859                     _es.etype = OBJECT;
00860                 }
00861                 break;
00862             case _SC('['):
00863                 if(_lex._prevtoken == _SC('\n')) Error(_SC("cannot break deref/or comma needed after
[exp]=exp slot declaration"));
00864                 Lex(); Expression(); Expect(_SC(']'));
00865                 pos = -1;
00866                 if(_es.etype==BASE) {
00867                     Emit2ArgsOP(_OP_GET);
00868                     pos = _fs->TopTarget();
00869                     _es.etype = EXPR;
00870                     _es.epos = pos;
00871                 }
00872                 else {
00873                     if(NeedGet()) {
00874                         Emit2ArgsOP(_OP_GET);
00875                     }
00876                     _es.etype = OBJECT;
00877                 }
00878                 break;
00879             case TK_MINUSMINUS:
00880             case TK_PLUSPLUS:
00881                 {
00882                     if(IsEndOfStatement()) return;
00883                     SQInteger diff = (_token==TK_MINUSMINUS) ? -1 : 1;
00884                     Lex();
00885                     switch(_es.etype)
00886                     {
00887                         case EXPR: Error(_SC("can't '++' or '--' an expression")); break;
00888                         case OBJECT:
00889                         case BASE:
00890                             if(_es.donot_get == true) { Error(_SC("can't '++' or '--' an
expression")); break; } //mmh dor this make sense?
00891                             Emit2ArgsOP(_OP_PINC, diff);
00892                             break;
00893                         case LOCAL: {
00894                             SQInteger src = _fs->PopTarget();

```

```

00895         _fs->AddInstruction(_OP_PINCL, _fs->PushTarget(), src, 0, diff);
00896     }
00897     break;
00898     case OUTER: {
00899         SQInteger tmp1 = _fs->PushTarget();
00900         SQInteger tmp2 = _fs->PushTarget();
00901         _fs->AddInstruction(_OP_GETOUTER, tmp2, _es.epos);
00902         _fs->AddInstruction(_OP_PINCL, tmp1, tmp2, 0, diff);
00903         _fs->AddInstruction(_OP_SETOUTER, tmp2, _es.epos, tmp2);
00904         _fs->PopTarget();
00905     }
00906 }
00907 }
00908 return;
00909 break;
00910 case _SC('('):
00911     switch(_es.etype) {
00912     case OBJECT: {
00913         SQInteger key = _fs->PopTarget(); /* location of the key */
00914         SQInteger table = _fs->PopTarget(); /* location of the object */
00915         SQInteger closure = _fs->PushTarget(); /* location for the closure */
00916         SQInteger ttarget = _fs->PushTarget(); /* location for 'this' pointer */
00917         _fs->AddInstruction(_OP_PREPCALL, closure, key, table, ttarget);
00918     }
00919     break;
00920     case BASE:
00921         //Emit2ArgsOP(_OP_GET);
00922         _fs->AddInstruction(_OP_MOVE, _fs->PushTarget(), 0);
00923     break;
00924     case OUTER:
00925         _fs->AddInstruction(_OP_GETOUTER, _fs->PushTarget(), _es.epos);
00926         _fs->AddInstruction(_OP_MOVE, _fs->PushTarget(), 0);
00927     break;
00928     default:
00929         _fs->AddInstruction(_OP_MOVE, _fs->PushTarget(), 0);
00930     }
00931     _es.etype = EXPR;
00932     Lex();
00933     FunctionCallArgs();
00934     break;
00935     default: return;
00936 }
00937 }
00938 }
00939 SQInteger Factor()
00940 {
00941     // _es.etype = EXPR;
00942     switch(_token)
00943     {
00944     case TK_STRING_LITERAL:
00945         _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
00946             _fs->GetConstant(_fs->CreateString(_lex._svalue, _lex._longstr.size()-1));
00947         Lex();
00948         break;
00949     case TK_BASE:
00950         Lex();
00951         _fs->AddInstruction(_OP_GETBASE, _fs->PushTarget());
00952         _es.etype = BASE;
00953         _es.epos = _fs->TopTarget();
00954         return _es.epos;
00955     break;
00956     case TK_IDENTIFIER:
00957     case TK_CONSTRUCTOR:
00958     case TK_THIS: {
00959         SQObject id;
00960         SQObject constant;
00961
00962         switch(_token) {
00963             case TK_IDENTIFIER: id = _fs->CreateString(_lex._svalue); break;
00964             case TK_THIS: id = _fs->CreateString(_SC("this"), 4); break;
00965             case TK_CONSTRUCTOR: id = _fs->CreateString(_SC("constructor"), 11); break;
00966         }
00967
00968         SQInteger pos = -1;
00969         Lex();
00970         if((pos = _fs->GetLocalVariable(id)) != -1) {
00971             /* Handle a local variable (includes 'this') */
00972             _fs->PushTarget(pos);
00973             _es.etype = LOCAL;
00974             _es.epos = pos;
00975         }
00976
00977         else if((pos = _fs->GetOuterVariable(id)) != -1) {
00978             /* Handle a free var */
00979             if(NeedGet()) {
00980                 _es.epos = _fs->PushTarget();
00981                 _fs->AddInstruction(_OP_GETOUTER, _es.epos, pos);
00982             }
00983         }
00984     }
00985 }

```

```

00987             /* _es.etype = EXPR; already default value */
00988         }
00989         else {
00990             _es.etype = OUTER;
00991             _es.epos = pos;
00992         }
00993     }
00994
00995     else if(_fs->IsConstant(id, constant)) {
00996         /* Handle named constant */
00997         SQObjectPtr constval;
00998         SQObject constid;
00999         if(sq_type(constant) == OT_TABLE) {
01000             Expect(' ');
01001             constid = Expect(TK_IDENTIFIER);
01002             if(!_table(constant)->Get(constid, constval)) {
01003                 constval.Null();
01004                 Error(_SC("invalid constant [%s.%s]"), _stringval(id),
01005                     _stringval(constid));
01006             }
01007         }
01008         else {
01009             constval = constant;
01010         }
01011         _es.epos = _fs->PushTarget();
01012
01013         /* generate direct or literal function depending on size */
01014         SQObjectType ctype = sq_type(constval);
01015         switch(ctype) {
01016             case OT_INTEGER: EmitLoadConstInt(_integer(constval), _es.epos); break;
01017             case OT_FLOAT: EmitLoadConstFloat(_float(constval), _es.epos); break;
01018             case OT_BOOL: _fs->AddInstruction(_OP_LOADBOOL, _es.epos, _integer(constval));
01019             break;
01020             default: _fs->AddInstruction(_OP_LOAD, _es.epos, _fs->GetConstant(constval));
01021             break;
01022         }
01023         _es.etype = EXPR;
01024     }
01025     else {
01026         /* Handle a non-local variable, aka a field. Push the 'this' pointer on
01027         * the virtual stack (always found in offset 0, so no instruction needs to
01028         * be generated), and push the key next. Generate an _OP_LOAD instruction
01029         * for the latter. If we are not using the variable as a dref expr, generate
01030         * the _OP_GET instruction.
01031         */
01032         _fs->PushTarget(0);
01033         _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(), _fs->GetConstant(id));
01034         if(NeedGet()) {
01035             Emit2ArgsOP(_OP_GET);
01036         }
01037         _es.etype = OBJECT;
01038     }
01039     return _es.epos;
01040 }
01041 break;
01042 case TK_DOUBLE_COLON: // "::"
01043     _fs->AddInstruction(_OP_LOADROOT, _fs->PushTarget());
01044     _es.etype = OBJECT;
01045     _token = _SC('.'); /* hack: drop into PrefixExpr, case '.' */
01046     _es.epos = -1;
01047     return _es.epos;
01048 break;
01049 case TK_NULL:
01050     _fs->AddInstruction(_OP_LOADNULLS, _fs->PushTarget(), 1);
01051     Lex();
01052     break;
01053 case TK_INTEGER: EmitLoadConstInt(_lex._nvalue, -1); Lex(); break;
01054 case TK_FLOAT: EmitLoadConstFloat(_lex._fvalue, -1); Lex(); break;
01055 case TK_TRUE: case TK_FALSE:
01056     _fs->AddInstruction(_OP_LOADBOOL, _fs->PushTarget(), _token == TK_TRUE?1:0);
01057     Lex();
01058     break;
01059 case _SC('{'): {
01060     _fs->AddInstruction(_OP_NEWOBJ, _fs->PushTarget(), 0, 0, NOT_ARRAY);
01061     SQInteger apos = _fs->GetCurrentPos(), key = 0;
01062     Lex();
01063     while(_token != _SC(']')) {
01064         Expression();
01065         if(_token == _SC(',')) Lex();
01066         SQInteger val = _fs->PopTarget();
01067         SQInteger array = _fs->TopTarget();
01068         _fs->AddInstruction(_OP_APPENDARRAY, array, val, AAT_STACK);
01069         key++;
01070     }
01071     _fs->SetInstructionParam(apos, 1, key);
01072     Lex();
01073 }

```

```

01071         break;
01072     case _SC('\''):
01073         _fs->AddInstruction(_OP_NEWOBJ, _fs->PushTarget(), 0, NOT_TABLE);
01074         Lex(); ParseTableOrClass(_SC('\''), _SC('\''));
01075         break;
01076     case TK_FUNCTION: FunctionExp(); break;
01077     case _SC('@'): FunctionExp(true); break;
01078     case TK_CLASS: Lex(); ClassExp(); break;
01079     case _SC('~'):
01080         Lex();
01081         switch(_token) {
01082             case TK_INTEGER: EmitLoadConstInt(~_lex._nvalue, -1); Lex(); break;
01083             case TK_FLOAT: EmitLoadConstFloat(~_lex._fvalue, -1); Lex(); break;
01084             default: UnaryOP(_OP_NEG);
01085         }
01086         break;
01087     case _SC('!'): Lex(); UnaryOP(_OP_NOT); break;
01088     case _SC('~'):
01089         Lex();
01090         if(_token == TK_INTEGER) { EmitLoadConstInt(~_lex._nvalue, -1); Lex(); break; }
01091         UnaryOP(_OP_BWNOT);
01092         break;
01093     case TK_TYPEOF : Lex(); UnaryOP(_OP_TYPEOF); break;
01094     case TK_RESUME : Lex(); UnaryOP(_OP_RESUME); break;
01095     case TK_CLONE : Lex(); UnaryOP(_OP_CLONE); break;
01096     case TK_RAWCALL: Lex(); Expect('('); FunctionCallArgs(true); break;
01097     case TK_MINUSMINUS :
01098     case TK_PLUSPLUS : PrefixIncDec(_token); break;
01099     case TK_DELETE : DeleteExpr(); break;
01100     case _SC('('): Lex(); CommaExpr(); Expect(_SC(')'));
01101         break;
01102     case TK__LINE__: EmitLoadConstInt(_lex._currentline, -1); Lex(); break;
01103     case TK__FILE__: _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
01104         _fs->GetConstant(_sourcename)); Lex(); break;
01105     default: Error(_SC("expression expected"));
01106     }
01107     _es.etype = EXPR;
01108     return -1;
01109 }
01110 void EmitLoadConstInt(SQInteger value, SQInteger target)
01111 {
01112     if(target < 0) {
01113         target = _fs->PushTarget();
01114     }
01115     if(value <= INT_MAX && value > INT_MIN) { //does it fit in 32 bits?
01116         _fs->AddInstruction(_OP_LOADINT, target, value);
01117     }
01118     else {
01119         _fs->AddInstruction(_OP_LOAD, target, _fs->GetNumericConstant(value));
01120     }
01121 }
01122 void EmitLoadConstFloat(SQFloat value, SQInteger target)
01123 {
01124     if(target < 0) {
01125         target = _fs->PushTarget();
01126     }
01127     if(sizeof(SQFloat) == sizeof(SQInt32)) {
01128         _fs->AddInstruction(_OP_LOADFLOAT, target, *((SQInt32 *)&value));
01129     }
01130     else {
01131         _fs->AddInstruction(_OP_LOAD, target, _fs->GetNumericConstant(value));
01132     }
01133 }
01134 void UnaryOP(SQOpcode op)
01135 {
01136     PrefixedExpr();
01137     SQInteger src = _fs->PopTarget();
01138     _fs->AddInstruction(op, _fs->PushTarget(), src);
01139 }
01140 bool NeedGet()
01141 {
01142     switch(_token) {
01143     case _SC('='): case _SC('<'): case TK_NEWSLOT: case TK_MODEQ: case TK_MULEQ:
01144     case TK_DIVEQ: case TK_MINUSEQ: case TK_PLUSEQ:
01145         return false;
01146     case TK_PLUSPLUS: case TK_MINUSMINUS:
01147         if (!IsEndOfStatement()) {
01148             return false;
01149         }
01150     }
01151     break;
01152 }
01153 return (!_es.donot_get || (_es.donot_get && (_token == _SC('.') || _token == _SC('['))));
01154 }
01155 void FunctionCallArgs(bool rawcall = false)
01156 {
01157     SQInteger nargs = 1; //this
01158     while(_token != _SC(')')) {

```

```

01190         Expression();
01191         MoveIfCurrentTargetIsLocal();
01192         nargs++;
01193         if(_token == _SC(',')){
01194             Lex();
01195             if(_token == ')') Error(_SC("expression expected, found ')')");
01196         }
01197     }
01198     Lex();
01199     if (rawcall) {
01200         if (nargs < 3) Error(_SC("rawcall requires at least 2 parameters (callee and this)"));
01201         nargs -= 2; //removes callee and this from count
01202     }
01203     for(SQInteger i = 0; i < (nargs - 1); i++) _fs->PopTarget();
01204     SQInteger stackbase = _fs->PopTarget();
01205     SQInteger closure = _fs->PopTarget();
01206     _fs->AddInstruction(_OP_CALL, _fs->PushTarget(), closure, stackbase, nargs);
01207     if (_token == '{')
01208     {
01209         SQInteger retval = _fs->TopTarget();
01210         SQInteger nkeys = 0;
01211         Lex();
01212         while (_token != ')') {
01213             switch (_token) {
01214                 case _SC('['):
01215                     Lex(); CommaExpr(); Expect(_SC(']'));
01216                     Expect(_SC('=')); Expression();
01217                     break;
01218                 default:
01219                     _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
01220 _fs->GetConstant(Expect(TK_IDENTIFIER)));
01221                     Expect(_SC('=')); Expression();
01222                     break;
01223             }
01224             if (_token == ',') Lex();
01225             nkeys++;
01226             SQInteger val = _fs->PopTarget();
01227             SQInteger key = _fs->PopTarget();
01228             _fs->AddInstruction(_OP_SET, 0xFF, retval, key, val);
01229         }
01230         Lex();
01231     }
01232 void ParseTableOrClass(SQInteger separator, SQInteger terminator)
01233 {
01234     SQInteger tpos = _fs->GetCurrentPos(), nkeys = 0;
01235     while(_token != terminator) {
01236         bool hasattrs = false;
01237         bool isstatic = false;
01238         //check if is an attribute
01239         if(separator == ';') {
01240             if(_token == TK_ATTR_OPEN) {
01241                 _fs->AddInstruction(_OP_NEWOBJ, _fs->PushTarget(), 0, NOT_TABLE); Lex();
01242                 ParseTableOrClass(',', TK_ATTR_CLOSE);
01243                 hasattrs = true;
01244             }
01245             if(_token == TK_STATIC) {
01246                 isstatic = true;
01247                 Lex();
01248             }
01249         }
01250         switch(_token) {
01251             case TK_FUNCTION:
01252             case TK_CONSTRUCTOR: {
01253                 SQInteger tk = _token;
01254                 Lex();
01255                 SQObject id = tk == TK_FUNCTION ? Expect(TK_IDENTIFIER) :
01256 _fs->CreateString(_SC("constructor"));
01257                 _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(), _fs->GetConstant(id));
01258                 SQInteger boundtarget = 0xFF;
01259                 if (_token == _SC('{')) {
01260                     boundtarget = ParseBindEnv();
01261                 }
01262                 Expect(_SC('('));
01263                 CreateFunction(id, boundtarget);
01264                 _fs->AddInstruction(_OP_CLOSURE, _fs->PushTarget(), _fs->_functions.size() - 1,
01265 boundtarget);
01266             }
01267             break;
01268             case _SC('['):
01269                 Lex(); CommaExpr(); Expect(_SC(']'));
01270                 Expect(_SC('=')); Expression();
01271                 break;
01272             case TK_STRING_LITERAL: //JSON
01273                 if(separator == ',') { //only works for tables
01274                     _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),

```

```

    _fs->GetConstant(Expect(TK_STRING_LITERAL));
01282         Expect(_SC(':')); Expression();
01283         break;
01284     }
01285     default :
01286         _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
    _fs->GetConstant(Expect(TK_IDENTIFIER)));
01287         Expect(_SC('=')); Expression();
01288     }
01289     if(_token == separator) Lex(); //optional comma/semicolon
01290     nkeys++;
01291     SQInteger val = _fs->PopTarget();
01292     SQInteger key = _fs->PopTarget();
01293     SQInteger attrs = hasattrs ? _fs->PopTarget() :-1;
01294     ((void)attrs);
01295     assert((hasattrs && (attrs == key-1)) || !hasattrs);
01296     unsigned char flags =
    (hasattrs?NEW_SLOT_ATTRIBUTES_FLAG:0)|(isstatic?NEW_SLOT_STATIC_FLAG:0);
01297     SQInteger table = _fs->TopTarget(); //«BECAUSE OF THIS NO COMMON EMIT FUNC IS POSSIBLE
01298     if(separator == _SC(',')) { //hack recognizes a table from the separator
01299         _fs->AddInstruction(_OP_NEWSLOT, 0xFF, table, key, val);
01300     }
01301     else {
01302         _fs->AddInstruction(_OP_NEWSLOTA, flags, table, key, val); //this for classes only as
    it invokes _newmember
01303     }
01304 }
01305 if(separator == _SC(',')) //hack recognizes a table from the separator
01306     _fs->SetInstructionParam(tpos, 1, nkeys);
01307 Lex();
01308 }
01314 void LocalDeclStatement()
01315 {
01316     SQObject varname;
01317     Lex();
01318     if( _token == TK_FUNCTION) {
01319         SQInteger boundtarget = 0xFF;
01320         Lex();
01321         varname = Expect(TK_IDENTIFIER);
01322         if( _token == _SC('(')) {
01323             boundtarget = ParseBindEnv();
01324         }
01325         Expect(_SC('('));
01326         CreateFunction(varname, 0xFF, false);
01327         _fs->AddInstruction(_OP_CLOSURE, _fs->PushTarget(), _fs->_functions.size() - 1,
    boundtarget);
01328         _fs->PopTarget();
01329         _fs->PushLocalVariable(varname);
01330         return;
01331     }
01332 }
01333 do {
01334     varname = Expect(TK_IDENTIFIER);
01335     if(_token == _SC('=')) {
01336         Lex(); Expression();
01337         SQInteger src = _fs->PopTarget();
01338         SQInteger dest = _fs->PushTarget();
01339         if(dest != src) _fs->AddInstruction(_OP_MOVE, dest, src);
01340     }
01341     else{
01342         _fs->AddInstruction(_OP_LOADNULLS, _fs->PushTarget(), 1);
01343     }
01344     _fs->PopTarget();
01345     _fs->PushLocalVariable(varname);
01346     if(_token == _SC(',')) Lex(); else break;
01347 } while(1);
01348 }
01353 void IfBlock()
01354 {
01355     if( _token == _SC('{'))
01356     {
01357         BEGIN_SCOPE();
01358         Lex();
01359         Statements();
01360         Expect(_SC('}'));
01361         if( true) {
01362             END_SCOPE();
01363         }
01364         else {
01365             END_SCOPE_NO_CLOSE();
01366         }
01367     }
01368     else {
01369         //BEGIN_SCOPE();
01370         Statement();
01371         if( _lex._prevtoken != _SC('}') && _lex._prevtoken != _SC(';')) OptionalSemicolon();
01372         //END_SCOPE();

```

```

01373     }
01374 }
01380 void IfStatement()
01381 {
01382     SQInteger jmppos;
01383     bool haselse = false;
01384     Lex(); Expect(_SC('(')); CommaExpr(); Expect(_SC(')'));
01385     _fs->AddInstruction(_OP_JZ, _fs->PopTarget());
01386     SQInteger jnepos = _fs->GetCurrentPos();
01387
01388
01389
01390     IfBlock();
01391     //
01392     /*static int n = 0;
01393     if (_token != _SC('}') && _token != TK_ELSE) {
01394         printf("IF %d-----!!!!!!!!\n", n);
01395         if (n == 5)
01396         {
01397             printf("asd");
01398         }
01399         n++;
01400         //OptionalSemicolon();
01401     }*/
01402
01403
01404     SQInteger endifblock = _fs->GetCurrentPos();
01405     if(_token == TK_ELSE){
01406         haselse = true;
01407         //BEGIN_SCOPE();
01408         _fs->AddInstruction(_OP_JMP);
01409         jmppos = _fs->GetCurrentPos();
01410         Lex();
01411         //Statement(); if(_lex._prevtoken != _SC('}')) OptionalSemicolon();
01412         IfBlock();
01413         //END_SCOPE();
01414         _fs->SetInstructionParam(jmppos, 1, _fs->GetCurrentPos() - jmppos);
01415     }
01416     _fs->SetInstructionParam(jnepos, 1, endifblock - jnepos + (haselse?1:0));
01417 }
01423 void WhileStatement()
01424 {
01425     SQInteger jzpos, jmppos;
01426     jmppos = _fs->GetCurrentPos();
01427     Lex(); Expect(_SC('(')); CommaExpr(); Expect(_SC(')'));
01428
01429     BEGIN_BREAKABLE_BLOCK();
01430     _fs->AddInstruction(_OP_JZ, _fs->PopTarget());
01431     jzpos = _fs->GetCurrentPos();
01432     BEGIN_SCOPE();
01433
01434     Statement();
01435
01436     END_SCOPE();
01437     _fs->AddInstruction(_OP_JMP, 0, jmppos - _fs->GetCurrentPos() - 1);
01438     _fs->SetInstructionParam(jzpos, 1, _fs->GetCurrentPos() - jzpos);
01439
01440     END_BREAKABLE_BLOCK(jmppos);
01441 }
01446 void DoWhileStatement()
01447 {
01448     Lex();
01449     SQInteger jmptrg = _fs->GetCurrentPos();
01450     BEGIN_BREAKABLE_BLOCK()
01451     BEGIN_SCOPE();
01452     Statement();
01453     END_SCOPE();
01454     Expect(TK_WHILE);
01455     SQInteger continuetr = _fs->GetCurrentPos();
01456     Expect(_SC('(')); CommaExpr(); Expect(_SC(')'));
01457     _fs->AddInstruction(_OP_JZ, _fs->PopTarget(), 1);
01458     _fs->AddInstruction(_OP_JMP, 0, jmptrg - _fs->GetCurrentPos() - 1);
01459     END_BREAKABLE_BLOCK(continuetr);
01460 }
01466 void ForStatement()
01467 {
01468     Lex();
01469     BEGIN_SCOPE();
01470     Expect(_SC('('));
01471     if(_token == TK_LOCAL) LocalDeclStatement();
01472     else if(_token != _SC(';')){
01473         CommaExpr();
01474         _fs->PopTarget();
01475     }
01476     Expect(_SC(';'));
01477     _fs->SnoozeOpt();
01478     SQInteger jmppos = _fs->GetCurrentPos();

```

```

01479         SQInteger jzpos = -1;
01480         if(_token != _SC(';')) { CommaExpr(); _fs->AddInstruction(_OP_JZ, _fs->PopTarget()); jzpos =
_fs->GetCurrentPos(); }
01481         Expect(_SC(';'));
01482         _fs->SnoozeOpt();
01483         SQInteger expstart = _fs->GetCurrentPos() + 1;
01484         if(_token != _SC('')) {
01485             CommaExpr();
01486             _fs->PopTarget();
01487         }
01488         Expect(_SC(''));
01489         _fs->SnoozeOpt();
01490         SQInteger expend = _fs->GetCurrentPos();
01491         SQInteger expsize = (expend - expstart) + 1;
01492         SQInstructionVec exp;
01493         if(expsize > 0) {
01494             for(SQInteger i = 0; i < expsize; i++)
01495                 exp.push_back(_fs->GetInstruction(expstart + i));
01496             _fs->PopInstructions(expsize);
01497         }
01498         BEGIN_BREAKBLE_BLOCK()
01499         Statement();
01500         SQInteger continuetrq = _fs->GetCurrentPos();
01501         if(expsize > 0) {
01502             for(SQInteger i = 0; i < expsize; i++)
01503                 _fs->AddInstruction(exp[i]);
01504         }
01505         _fs->AddInstruction(_OP_JMP, 0, jmppos - _fs->GetCurrentPos() - 1, 0);
01506         if(jzpos > 0) _fs->SetInstructionParam(jzpos, 1, _fs->GetCurrentPos() - jzpos);
01507
01508         END_BREAKBLE_BLOCK(continuetrq);
01509
01510         END_SCOPE();
01511     }
01512     void ForEachStatement()
01513     {
01514         SQObject idxname, valname;
01515         Lex(); Expect(_SC('(')); valname = Expect(TK_IDENTIFIER);
01516         if(_token == _SC(',')) {
01517             idxname = valname;
01518             Lex(); valname = Expect(TK_IDENTIFIER);
01519         }
01520         else{
01521             idxname = _fs->CreateString(_SC("@INDEX@"));
01522         }
01523         Expect(TK_IN);
01524
01525         //save the stack size
01526         BEGIN_SCOPE();
01527         //put the table in the stack(evaluate the table expression)
01528         Expression(); Expect(_SC(''));
01529         SQInteger container = _fs->TopTarget();
01530         //push the index local var
01531         SQInteger indexpos = _fs->PushLocalVariable(idxname);
01532         _fs->AddInstruction(_OP_LOADNULLS, indexpos, 1);
01533         //push the value local var
01534         SQInteger valuepos = _fs->PushLocalVariable(valname);
01535         _fs->AddInstruction(_OP_LOADNULLS, valuepos, 1);
01536         //push reference index
01537         SQInteger itrpos = _fs->PushLocalVariable(_fs->CreateString(_SC("@ITERATOR@"))); //use invalid
01538         id to make it inaccessible
01539         _fs->AddInstruction(_OP_LOADNULLS, itrpos, 1);
01540         SQInteger jmppos = _fs->GetCurrentPos();
01541         _fs->AddInstruction(_OP_FOREACH, container, 0, indexpos);
01542         SQInteger foreachpos = _fs->GetCurrentPos();
01543         _fs->AddInstruction(_OP_POSTFOREACH, container, 0, indexpos);
01544         //generate the statement code
01545         BEGIN_BREAKBLE_BLOCK()
01546         Statement();
01547         _fs->AddInstruction(_OP_JMP, 0, jmppos - _fs->GetCurrentPos() - 1);
01548         _fs->SetInstructionParam(foreachpos, 1, _fs->GetCurrentPos() - foreachpos);
01549         _fs->SetInstructionParam(foreachpos + 1, 1, _fs->GetCurrentPos() - foreachpos);
01550         END_BREAKBLE_BLOCK(foreachpos - 1);
01551         //restore the local variable stack(remove index,val and ref idx)
01552         _fs->PopTarget();
01553         END_SCOPE();
01554     }
01555     void SwitchStatement()
01556     {
01557         Lex(); Expect(_SC('(')); CommaExpr(); Expect(_SC(''));
01558         Expect(_SC('{'));
01559         SQInteger expr = _fs->TopTarget();
01560         bool bfirst = true;
01561         SQInteger tonextcondjmp = -1;
01562         SQInteger skipcondjmp = -1;
01563         SQInteger nbreaks__ = _fs->unresolvedbreaks.size();
01564         _fs->breaktargets.push_back(0);

```

```

01573     while(_token == TK_CASE) {
01574         if(!bfirst) {
01575             _fs->AddInstruction(_OP_JMP, 0, 0);
01576             skipcondjmp = _fs->GetCurrentPos();
01577             _fs->SetInstructionParam(tonextcondjmp, 1, _fs->GetCurrentPos() - tonextcondjmp);
01578         }
01579         //condition
01580         Lex(); Expression(); Expect(_SC(':'));
01581         SQInteger trg = _fs->PopTarget();
01582         SQInteger eqtarget = trg;
01583         bool local = _fs->IsLocal(trg);
01584         if(local) {
01585             eqtarget = _fs->PushTarget(); //we need to allocate a extra reg
01586         }
01587         _fs->AddInstruction(_OP_EQ, eqtarget, trg, expr);
01588         _fs->AddInstruction(_OP_JZ, eqtarget, 0);
01589         if(local) {
01590             _fs->PopTarget();
01591         }
01592
01593         //end condition
01594         if(skipcondjmp != -1) {
01595             _fs->SetInstructionParam(skipcondjmp, 1, (_fs->GetCurrentPos() - skipcondjmp));
01596         }
01597         tonextcondjmp = _fs->GetCurrentPos();
01598         BEGIN_SCOPE();
01599         Statements();
01600         END_SCOPE();
01601         bfirst = false;
01602     }
01603     if(tonextcondjmp != -1)
01604         _fs->SetInstructionParam(tonextcondjmp, 1, _fs->GetCurrentPos() - tonextcondjmp);
01605     if(_token == TK_DEFAULT) {
01606         Lex(); Expect(_SC(':'));
01607         BEGIN_SCOPE();
01608         Statements();
01609         END_SCOPE();
01610     }
01611     Expect(_SC(')'));
01612     _fs->PopTarget();
01613     __nbreaks__ = _fs->unresolvedbreaks.size() - __nbreaks__;
01614     if(__nbreaks__ > 0) ResolveBreaks(_fs, __nbreaks__);
01615     _fs->_breaktargets.pop_back();
01616 }
01622 void FunctionStatement()
01623 {
01624     SQObject id;
01625     Lex(); id = Expect(TK_IDENTIFIER);
01626     _fs->PushTarget(0);
01627     _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(), _fs->GetConstant(id));
01628     if(_token == TK_DOUBLE_COLON) Emit2ArgsOP(_OP_GET);
01629
01630     while(_token == TK_DOUBLE_COLON) {
01631         Lex();
01632         id = Expect(TK_IDENTIFIER);
01633         _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(), _fs->GetConstant(id));
01634         if(_token == TK_DOUBLE_COLON) Emit2ArgsOP(_OP_GET);
01635     }
01636     SQInteger boundtarget = 0xFF;
01637     if (_token == _SC('[')) {
01638         boundtarget = ParseBindEnv();
01639     }
01640     Expect(_SC('('));
01641     CreateFunction(id, boundtarget);
01642     _fs->AddInstruction(_OP_CLOSURE, _fs->PushTarget(), _fs->_functions.size() - 1, boundtarget);
01643     EmitDerefOp(_OP_NEWSLOT);
01644     _fs->PopTarget();
01645 }
01651 void ClassStatement()
01652 {
01653     SQExpState es;
01654     Lex();
01655     es = _es;
01656     _es.donot_get = true;
01657     PrefixedExpr();
01658     if(_es.etype == EXPR) {
01659         Error(_SC("invalid class name"));
01660     }
01661     else if(_es.etype == OBJECT || _es.etype == BASE) {
01662         ClassExp();
01663         EmitDerefOp(_OP_NEWSLOT);
01664         _fs->PopTarget();
01665     }
01666     else {
01667         Error(_SC("cannot create a class in a local with the syntax(class <local>)"));
01668     }
01669     _es = es;

```

```

01670     }
01671     SQObject ExpectScalar()
01672     {
01673         SQObject val;
01674         val._type = OT_NULL; val._unVal.nInteger = 0; //shut up GCC 4.x
01675         switch(_token) {
01676             case TK_INTEGER:
01677                 val._type = OT_INTEGER;
01678                 val._unVal.nInteger = _lex._nvalue;
01679                 break;
01680             case TK_FLOAT:
01681                 val._type = OT_FLOAT;
01682                 val._unVal.fFloat = _lex._fvalue;
01683                 break;
01684             case TK_STRING_LITERAL:
01685                 val = _fs->CreateString(_lex._svalue, _lex._longstr.size()-1);
01686                 break;
01687             case TK_TRUE:
01688             case TK_FALSE:
01689                 val._type = OT_BOOL;
01690                 val._unVal.nInteger = _token == TK_TRUE ? 1 : 0;
01691                 break;
01692             case '-':
01693                 Lex();
01694                 switch(_token)
01695                 {
01696                     case TK_INTEGER:
01697                         val._type = OT_INTEGER;
01698                         val._unVal.nInteger = -_lex._nvalue;
01699                         break;
01700                     case TK_FLOAT:
01701                         val._type = OT_FLOAT;
01702                         val._unVal.fFloat = -_lex._fvalue;
01703                         break;
01704                     default:
01705                         Error(_SC("scalar expected : integer, float"));
01706                 }
01707                 break;
01708             default:
01709                 Error(_SC("scalar expected : integer, float, or string"));
01710         }
01711         Lex();
01712         return val;
01713     }
01714 void EnumStatement()
01715 {
01716     Lex();
01717     SQObject id = Expect(TK_IDENTIFIER);
01718     Expect(_SC('{'));
01719
01720     SQObject table = _fs->CreateTable();
01721     SQInteger nval = 0;
01722     while(_token != _SC('}')) {
01723         SQObject key = Expect(TK_IDENTIFIER);
01724         SQObject val;
01725         if(_token == _SC('=')) {
01726             Lex();
01727             val = ExpectScalar();
01728         }
01729         else {
01730             val._type = OT_INTEGER;
01731             val._unVal.nInteger = nval++;
01732         }
01733         _table(table)->NewSlot(SQObjectPtr(key), SQObjectPtr(val));
01734         if(_token == ',') Lex();
01735     }
01736     SQTable *enums = _table(_ss(_vm)->_consts);
01737     SQObjectPtr strongid = id;
01738     enums->NewSlot(SQObjectPtr(strongid), SQObjectPtr(table));
01739     strongid.Null();
01740     Lex();
01741 }
01742 void TryCatchStatement()
01743 {
01744     SQObject exid;
01745     Lex();
01746     _fs->AddInstruction(_OP_PUSHTRAP, 0, 0);
01747     _fs->_traps++;
01748     if(_fs->_breaktargets.size()) _fs->_breaktargets.top()++;
01749     if(_fs->_continuetargets.size()) _fs->_continuetargets.top()++;
01750     SQInteger trappos = _fs->GetCurrentPos();
01751     {
01752         BEGIN_SCOPE();
01753         Statement();
01754         END_SCOPE();
01755     }
01756     _fs->_traps--;
01757 }

```

```

01772     _fs->AddInstruction(_OP_POPTRAP, 1, 0);
01773     if(_fs->_breaktargets.size()) _fs->_breaktargets.top()--;
01774     if(_fs->_continuetargets.size()) _fs->_continuetargets.top()--;
01775     _fs->AddInstruction(_OP_JMP, 0, 0);
01776     SQInteger jmppos = _fs->GetCurrentPos();
01777     _fs->SetInstructionParam(trappos, 1, (_fs->GetCurrentPos() - trappos));
01778     Expect(TK_CATCH); Expect(_SC('(')); exid = Expect(TK_IDENTIFIER); Expect(_SC(')'));
01779     {
01780         BEGIN_SCOPE();
01781         SQInteger ex_target = _fs->PushLocalVariable(exid);
01782         _fs->SetInstructionParam(trappos, 0, ex_target);
01783         Statement();
01784         _fs->SetInstructionParams(jmppos, 0, (_fs->GetCurrentPos() - jmppos), 0);
01785         END_SCOPE();
01786     }
01787 }
01794 SQInteger ParseBindEnv()
01795 {
01796     SQInteger boundtarget;
01797     Lex();
01798     Expression();
01799     boundtarget = _fs->TopTarget();
01800     Expect(_SC(']'));
01801     return boundtarget;
01802 }
01808 void FunctionExp(bool lambda = false)
01809 {
01810     Lex();
01811     SQInteger boundtarget = 0xFF;
01812     if (_token == _SC('(')) {
01813         boundtarget = ParseBindEnv();
01814     }
01815     Expect(_SC('('));
01816     SQObjectPtr dummy;
01817     CreateFunction(dummy, boundtarget, lambda);
01818     _fs->AddInstruction(_OP_CLOSURE, _fs->PushTarget(), _fs->_functions.size() - 1, boundtarget);
01819 }
01826 void ClassExp()
01827 {
01828     SQInteger base = -1;
01829     SQInteger attrs = -1;
01830     if(_token == TK_EXTENDS) {
01831         Lex(); Expression();
01832         base = _fs->TopTarget();
01833     }
01834     if(_token == TK_ATTR_OPEN) {
01835         Lex();
01836         _fs->AddInstruction(_OP_NEWOBJ, _fs->PushTarget(), 0, NOT_TABLE);
01837         ParseTableOrClass(_SC(','), TK_ATTR_CLOSE);
01838         attrs = _fs->TopTarget();
01839     }
01840     Expect(_SC('{'));
01841     if(attrs != -1) _fs->PopTarget();
01842     if(base != -1) _fs->PopTarget();
01843     _fs->AddInstruction(_OP_NEWOBJ, _fs->PushTarget(), base, attrs, NOT_CLASS);
01844     ParseTableOrClass(_SC(';'), _SC('}'));
01845 }
01851 void DeleteExpr()
01852 {
01853     SQExpState es;
01854     Lex();
01855     es = _es;
01856     _es.donot_get = true;
01857     PrefixedExpr();
01858     if(_es.etype==EXPR) Error(_SC("can't delete an expression"));
01859     if(_es.etype==OBJECT || _es.etype==BASE) {
01860         Emit2ArgsOP(_OP_DELETE);
01861     }
01862     else {
01863         Error(_SC("cannot delete an (outer) local"));
01864     }
01865     _es = es;
01866 }
01872 void PrefixIncDec(SQInteger token)
01873 {
01874     SQExpState es;
01875     SQInteger diff = (token==TK_MINUSMINUS) ? -1 : 1;
01876     Lex();
01877     es = _es;
01878     _es.donot_get = true;
01879     PrefixedExpr();
01880     if(_es.etype==EXPR) {
01881         Error(_SC("can't '++' or '--' an expression"));
01882     }
01883     else if(_es.etype==OBJECT || _es.etype==BASE) {
01884         Emit2ArgsOP(_OP_INC, diff);
01885     }

```

```

01886     else if(_es.etype==LOCAL) {
01887         SQInteger src = _fs->TopTarget();
01888         _fs->AddInstruction(_OP_INCL, src, src, 0, diff);
01889     }
01890 }
01891 else if(_es.etype==OUTER) {
01892     SQInteger tmp = _fs->PushTarget();
01893     _fs->AddInstruction(_OP_GETOUTER, tmp, _es.epos);
01894     _fs->AddInstruction(_OP_INCL, tmp, tmp, 0, diff);
01895     _fs->AddInstruction(_OP_SETOUTER, tmp, _es.epos, tmp);
01896 }
01897 _es = es;
01898 }
01900 void CreateFunction(SQObject &name, SQInteger boundtarget, bool lambda = false)
01901 {
01902     SQFuncState *funcstate = _fs->PushChildState(_ss(_vm));
01903     funcstate->_name = name;
01904     SQObject paramname;
01905     funcstate->AddParameter(_fs->CreateString(_SC("this")));
01906     funcstate->_sourcename = _sourcename;
01907     SQInteger defparams = 0;
01908     while(_token!=_SC('(')) {
01909         if(_token == TK_VARPARAMS) {
01910             if(defparams > 0) Error(_SC("function with default parameters cannot have variable
number of parameters"));
01911             funcstate->AddParameter(_fs->CreateString(_SC("vargv")));
01912             funcstate->_varparams = true;
01913             Lex();
01914             if(_token != _SC('(')) Error(_SC("expected ' '"));
01915             break;
01916         }
01917         else {
01918             paramname = Expect(TK_IDENTIFIER);
01919             funcstate->AddParameter(paramname);
01920             if(_token == _SC('=')) {
01921                 Lex();
01922                 Expression();
01923                 funcstate->AddDefaultParam(_fs->TopTarget());
01924                 defparams++;
01925             }
01926             else {
01927                 if(defparams > 0) Error(_SC("expected '='"));
01928             }
01929             if(_token == _SC(',',')) Lex();
01930             else if(_token != _SC('(')) Error(_SC("expected ')' or ','"));
01931         }
01932     }
01933     Expect(_SC('('));
01934     if (boundtarget != 0xFF) {
01935         _fs->PopTarget();
01936     }
01937     for(SQInteger n = 0; n < defparams; n++) {
01938         _fs->PopTarget();
01939     }
01940     SQFuncState *currchunk = _fs;
01941     _fs = funcstate;
01942     if(lambda) {
01943         Expression();
01944         _fs->AddInstruction(_OP_RETURN, 1, _fs->PopTarget());
01945     }
01946     else {
01947         Statement(false);
01948     }
01949     funcstate->AddLineInfos(_lex._prevtoken == _SC('\n')?_lex._lasttokenline:_lex._currentline,
_lineinfo, true);
01950     funcstate->AddInstruction(_OP_RETURN, -1);
01951     funcstate->SetStackSize(0);
01952     SQFunctionProto *func = funcstate->BuildProto();
01953     #ifdef _DEBUG_DUMP
01954     funcstate->Dump(func);
01955     #endif
01956     _fs = currchunk;
01957     _fs->_functions.push_back(func);
01958     _fs->PopChildState();
01959 }
01960 void ResolveBreaks(SQFuncState *funcstate, SQInteger ntoresolve)
01961 {
01962     while(ntoresolve > 0) {
01963         SQInteger pos = funcstate->_unresolvedbreaks.back();
01964         funcstate->_unresolvedbreaks.pop_back();
01965         //set the jmp instruction
01966         funcstate->SetInstructionParams(pos, 0, funcstate->GetCurrentPos() - pos, 0);
01967         ntoresolve--;
01968     }
01969 }
01970 void ResolveContinues(SQFuncState *funcstate, SQInteger ntoresolve, SQInteger targetpos)

```

```

01994     {
01995         while (ntoresolve > 0) {
01996             SQInteger pos = funcstate->unresolvedcontinues.back();
01997             funcstate->unresolvedcontinues.pop_back();
01998             //set the jmp instruction
01999             funcstate->SetInstructionParams(pos, 0, targetpos - pos, 0);
02000             ntoresolve--;
02001         }
02002     }
02003 private:
02004     SQInteger _token;
02005     SQFuncState *_fs;
02006     SQObjectPtr _sourcename;
02007     SQLexer _lex;
02008     bool _lineinfo;
02009     bool _raiseerror;
02010     SQInteger _debugline;
02011     SQInteger _debugop;
02012     SQExpState _es;
02013     SQScope _scope;
02014     SQChar _compilererror[MAX_COMPILER_ERROR_LEN];
02015     jmp_buf _errorjmp;
02016     SQVM *_vm;
02017 };
02018
02031 bool Compile(SQVM *vm, SQLEXREADFUNC rg, SQUserPointer up, const SQChar *sourcename, SQObjectPtr &out,
02032             bool raiseerror, bool lineinfo)
02033 {
02034     SQCompiler p(vm, rg, up, sourcename, raiseerror, lineinfo);
02035     return p.Compile(out);
02036 }
02037 #endif

```

4.1.1.3 BEGIN_SCOPE

```
#define BEGIN_SCOPE( )
```

Value:

```

SQScope __oldscope__ = _scope; \
_scope.outers = _fs->outers; \
_scope.stacksize = _fs->GetStackSize();

```

新しいスコープを開始するためのマクロ。

現在のスコープ情報を保存し、新しいスコープのスタックサイズとアウター変数の数を初期化します。

Definition at line 65 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

4.1.1.4 END_BREAKBLE_BLOCK

```
#define END_BREAKBLE_BLOCK(
    continue_target )
```

Value:

```

{__nbreaks__=_fs->unresolvedbreaks.size()-__nbreaks__; \
 __ncontinues__=_fs->unresolvedcontinues.size()-__ncontinues__; \
 if(__ncontinues__>0) ResolveContinues(_fs,__ncontinues__,continue_target); \
 if(__nbreaks__>0) ResolveBreaks(_fs,__nbreaks__); \
 _fs->breaktargets.pop_back();_fs->continuetargets.pop_back();}

```

break/continue が可能なブロックを終了するためのマクロ。

ブロック内で追加された未解決の break および continue を解決します。その後、break/continue ターゲットをスタックからポップします。

Parameters

<code>continue_target</code>	continue 文のジャンプ先となる命令の位置。
------------------------------	---------------------------

Definition at line 121 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00121         {__nbreaks__=_fs->_unresolvedbreaks.size()-__nbreaks__; \
00122         __ncontinues__=_fs->_unresolvedcontinues.size()-__ncontinues__; \
00123         if(__ncontinues__>0)ResolveContinues(_fs,__ncontinues__,continue_target); \
00124         if(__nbreaks__>0)ResolveBreaks(_fs,__nbreaks__); \
00125         _fs->_breaktargets.pop_back();_fs->_continuetargets.pop_back();}
```

4.1.1.5 END_SCOPE

```
#define END_SCOPE( )
```

Value:

```
{   SQInteger oldouters = _fs->_outers;\
if(_fs->GetStackSize() != _scope.stacksize) { \
    _fs->SetStackSize(_scope.stacksize); \
    if(oldouters != _fs->_outers) { \
        _fs->AddInstruction(_OP_CLOSE,0,_scope.stacksize); \
    } \
} \
_scope = __oldscope__; \
}
```

スコープを終了するマクロ。

スタックサイズをスコープ開始時の状態に戻します。スコープ内でアウター変数の数が変化した場合は、クローズ命令 (`_OP_CLOSE`) を生成します。最後にスコープ情報を復元します。

Definition at line 96 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00096         {   SQInteger oldouters = _fs->_outers;\
00097             if(_fs->GetStackSize() != _scope.stacksize) { \
00098                 _fs->SetStackSize(_scope.stacksize); \
00099                 if(oldouters != _fs->_outers) { \
00100                     _fs->AddInstruction(_OP_CLOSE,0,_scope.stacksize); \
00101                 } \
00102             } \
00103             _scope = __oldscope__; \
00104         }
```

4.1.1.6 END_SCOPE_NO_CLOSE

```
#define END_SCOPE_NO_CLOSE( )
```

Value:

```
{   if(_fs->GetStackSize() != _scope.stacksize) { \
    _fs->SetStackSize(_scope.stacksize); \
} \
_scope = __oldscope__; \
}
```

スコープを終了するマクロ（クローズ命令なし）。

スタックサイズをスコープ開始時の状態に戻し、スコープ情報を復元します。アウター変数の解決（クローズ命令の生成）は行いません。

Definition at line 84 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00084         {   if(_fs->GetStackSize() != _scope.stacksize) { \
00085             _fs->SetStackSize(_scope.stacksize); \
00086         } \
00087         _scope = __oldscope__; \
00088     }
```

4.1.1.7 EXPR

```
#define EXPR 1
```

Definition at line 19 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

4.1.1.8 LOCAL

```
#define LOCAL 4
```

Definition at line 22 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

4.1.1.9 MAX_COMPILER_ERROR_LEN

```
#define MAX_COMPILER_ERROR_LEN 256
```

コンパイラエラーメッセージの最大長。

Definition at line 50 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

4.1.1.10 OBJECT

```
#define OBJECT 2
```

Definition at line 20 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

4.1.1.11 OUTER

```
#define OUTER 5
```

Definition at line 23 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

4.1.1.12 RESOLVE_OUTERS

```
#define RESOLVE_OUTERS( )
```

Value:

```
    if(_fs->GetStackSize() != _scope.stacksize) { \
    if(_fs->CountOuters(_scope.stacksize)) { \
        _fs->AddInstruction(_OP_CLOSE,0,_scope.stacksize); \
    } \
}
```

アウター変数を解決するためのマクロ。

現在のスコープを抜ける際に、必要であればクローズ命令 (_OP_CLOSE) を追加します。

Definition at line 73 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
00073                                     { \
00074         if(_fs->CountOuters(_scope.stacksize)) { \
00075             _fs->AddInstruction(_OP_CLOSE,0,_scope.stacksize); \
00076         } \
00077     }
```

4.1.2 Function Documentation

4.1.2.1 Compile()

```
bool Compile (
    SQVM * vm,
    SQLEXREADFUNC rg,
    SQUserPointer up,
    const SQChar * sourcename,
    SQObjectPtr & out,
    bool raiseerror,
    bool lineinfo )
```

Squirrel スクリプトをコンパイルするグローバル関数。

SQCompiler クラスのラッパーとして機能します。

Parameters

	<i>vm</i>	関連付けられるSquirrel VM へのポインタ。
	<i>rg</i>	字句解析器がソースコードを読み込むための関数ポインタ。
	<i>up</i>	rg 関数に渡されるユーザーポインタ。
	<i>sourcename</i>	コンパイル対象のソース名。
out	<i>out</i>	コンパイル結果として生成された関数プロトタイプの格納先。
	<i>raiseerror</i>	エラー発生時にエラーハンドラを呼び出すかどうか。
	<i>lineinfo</i>	バイトコードに行情報を含めるかどうか。

Returns

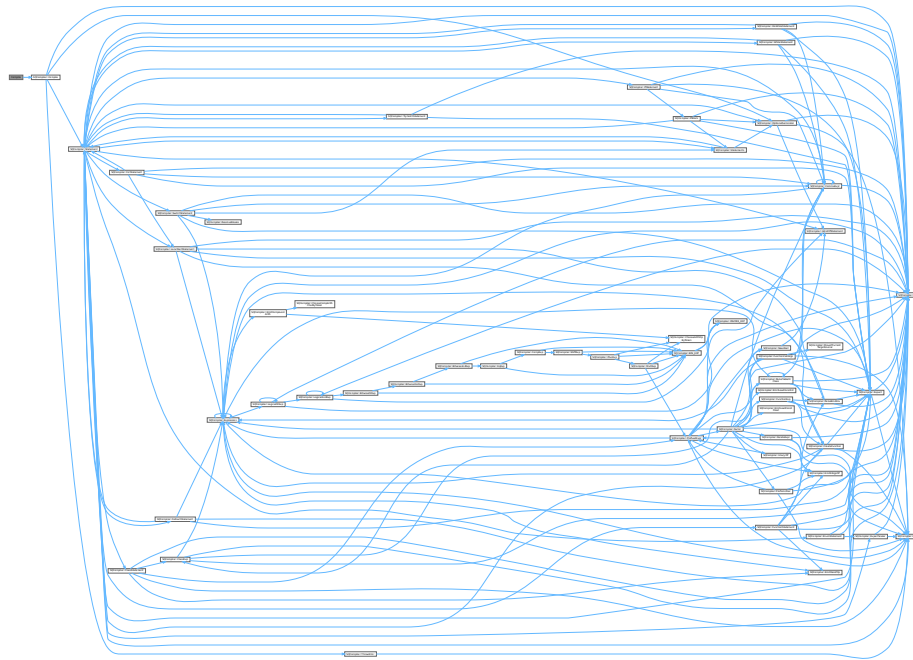
コンパイルが成功した場合は true、失敗した場合は false。

Definition at line 2031 of file [squirrel3_squirrel_sqcompiler_v1.cpp](#).

```
02032 {
02033     SQCompiler p(vm, rg, up, sourcename, raiseerror, lineinfo);
02034     return p.Compile(out);
02035 }
```

References [SQCompiler::Compile\(\)](#).

Here is the call graph for this function:



4.2 squirrel3_squirrel_sqcompiler_v1.cpp

[Go to the documentation of this file.](#)

```

00001 //cpp
00002
00003 /*
00004     Written By Gemini
00005 */
00006 #include "sqpcheader.h"
00007 #ifndef NO_COMPILER
00008 #include <stdarg.h>
00009 #include <setjmp.h>
00010 #include "sqopcodes.h"
00011 #include "sqstring.h"
00012 #include "sqfuncproto.h"
00013 #include "sqcompiler.h"
00014 #include "sqfuncstate.h"
00015 #include "sqlexer.h"
00016 #include "sqvm.h"
00017 #include "sqtable.h"
00018
00019 #define EXPR 1
00020 #define OBJECT 2
00021 #define BASE 3
00022 #define LOCAL 4
00023 #define OUTER 5
00024
00029 struct SQExpState {
00034     SQInteger etype; /* expr. type; one of EXPR, OBJECT, BASE, OUTER or LOCAL */
00039     SQInteger epos; /* expr. location on stack; -1 for OBJECT and BASE */
00044     bool donot_get; /* signal not to deref the next value */
00045 };
00046
00050 #define MAX_COMPILER_ERROR_LEN 256
00051
00056 struct SQScope {
00057     SQInteger outers;
00058     SQInteger stacksize;
00059 };
00060
00065 #define BEGIN_SCOPE() SQScope __oldscope__ = _scope; \
00066                     _scope.outers = _fs->outers; \
00067                     _scope.stacksize = _fs->GetStackSize();
00068

```

```

00073 #define RESOLVE_OUTERS() if(_fs->GetStackSize() != _scope.stacksize) { \
00074     if(_fs->CountOuters(_scope.stacksize)) { \
00075         _fs->AddInstruction(_OP_CLOSE,0,_scope.stacksize); \
00076     } \
00077 }
00078
00084 #define END_SCOPE_NO_CLOSE() { if(_fs->GetStackSize() != _scope.stacksize) { \
00085     _fs->SetStackSize(_scope.stacksize); \
00086 } \
00087 _scope = __oldscope__; \
00088 }
00089
00096 #define END_SCOPE() { SQInteger oldouters = _fs->_outers;\
00097     if(_fs->GetStackSize() != _scope.stacksize) { \
00098         _fs->SetStackSize(_scope.stacksize); \
00099         if(oldouters != _fs->_outers) { \
00100             _fs->AddInstruction(_OP_CLOSE,0,_scope.stacksize); \
00101         } \
00102     } \
00103     _scope = __oldscope__; \
00104 }
00105
00111 #define BEGIN_BREAKABLE_BLOCK() SQInteger __nbreaks__=_fs->_unresolvedbreaks.size(); \
00112     SQInteger __ncontinues__=_fs->_unresolvedcontinues.size(); \
00113     _fs->_breaktargets.push_back(0);_fs->_continuetargets.push_back(0);
00114
00121 #define END_BREAKABLE_BLOCK(continue_target) {__nbreaks__=_fs->_unresolvedbreaks.size()-__nbreaks__; \
00122     __ncontinues__=_fs->_unresolvedcontinues.size()-__ncontinues__; \
00123     if(__ncontinues__>0)ResolveContinues(_fs,__ncontinues__,continue_target); \
00124     if(__nbreaks__>0)ResolveBreaks(_fs,__nbreaks__); \
00125     _fs->_breaktargets.pop_back();_fs->_continuetargets.pop_back();}
00126
00131 class SQCompiler
00132 {
00133 public:
00144     SQCompiler(SQVM *v, SQLEXREADFUNC rg, SQUserPointer up, const SQChar* sourcename, bool raiseerror,
00145         bool lineinfo)
00146     {
00147         _vm=v;
00148         _lex.Init(_ss(v), rg, up,ThrowError,this);
00149         _sourcename = SQString::Create(_ss(v), sourcename);
00150         _lineinfo = lineinfo;_raiseerror = raiseerror;
00151         _scope.outers = 0;
00152         _scope.stacksize = 0;
00153         _compilererror[0] = _SC('\0');
00154     }
00160     static void ThrowError(void *ud, const SQChar *s) {
00161         SQCompiler *c = (SQCompiler *)ud;
00162         c->Error(s);
00163     }
00171     void Error(const SQChar *s, ...)
00172     {
00173         va_list vl;
00174         va_start(vl, s);
00175         scvsprintf(_compilererror, MAX_COMPILER_ERROR_LEN, s, vl);
00176         va_end(vl);
00177         longjmp(_errorjmp,1);
00178     }
00183     void Lex(){ _token = _lex.Lex();}
00192     SQObject Expect(SQInteger tok)
00193     {
00194
00195         if(_token != tok) {
00196             if(_token == TK_CONSTRUCTOR && tok == TK_IDENTIFIER) {
00197                 //do nothing
00198             }
00199             else {
00200                 const SQChar *etyename;
00201                 if(tok > 255) {
00202                     switch(tok)
00203                     {
00204                         case TK_IDENTIFIER:
00205                             etyename = _SC("IDENTIFIER");
00206                             break;
00207                         case TK_STRING_LITERAL:
00208                             etyename = _SC("STRING_LITERAL");
00209                             break;
00210                         case TK_INTEGER:
00211                             etyename = _SC("INTEGER");
00212                             break;
00213                         case TK_FLOAT:
00214                             etyename = _SC("FLOAT");
00215                             break;
00216                         default:
00217                             etyename = _lex.Tok2Str(tok);
00218                     }
00219                     Error(_SC("expected '%s'"), etyename);

```

```

00220         }
00221         Error(_SC("expected '%c'", tok);
00222     }
00223 }
00224 SQObjectPtr ret;
00225 switch(tok)
00226 {
00227     case TK_IDENTIFIER:
00228         ret = _fs->CreateString(_lex._svalue);
00229         break;
00230     case TK_STRING_LITERAL:
00231         ret = _fs->CreateString(_lex._svalue, _lex._longstr.size()-1);
00232         break;
00233     case TK_INTEGER:
00234         ret = SQObjectPtr(_lex._nvalue);
00235         break;
00236     case TK_FLOAT:
00237         ret = SQObjectPtr(_lex._fvalue);
00238         break;
00239 }
00240 Lex();
00241 return ret;
00242 }
00243 bool IsEndOfStatement() { return ((_lex._prevtoken == _SC('\n')) || (_token == SQUIRREL_EOB) ||
00244 (_token == _SC('(')) || (_token == _SC(';'))); }
00245 void OptionalSemicolon()
00246 {
00247     if(_token == _SC(';')) { Lex(); return; }
00248     if(!IsEndOfStatement()) {
00249         Error(_SC("end of statement expected (; or lf)"));
00250     }
00251 }
00252 void MoveIfCurrentTargetIsLocal() {
00253     SQInteger trg = _fs->TopTarget();
00254     if(_fs->IsLocal(trg)) {
00255         trg = _fs->PopTarget(); //pops the target and moves it
00256         _fs->AddInstruction(_OP_MOVE, _fs->PushTarget(), trg);
00257     }
00258 }
00259 bool Compile(SQObjectPtr &o)
00260 {
00261     _debugline = 1;
00262     _debugop = 0;
00263
00264     SQFuncState funcstate(_ss(_vm), NULL, ThrowError, this);
00265     funcstate._name = SQString::Create(_ss(_vm), _SC("main"));
00266     _fs = &funcstate;
00267     _fs->AddParameter(_fs->CreateString(_SC("this")));
00268     _fs->AddParameter(_fs->CreateString(_SC("vargv")));
00269     _fs->_varparams = true;
00270     _fs->_sourcename = _sourcename;
00271     SQInteger stacksize = _fs->GetStackSize();
00272     if(setjmp(_errorjmp) == 0) {
00273         Lex();
00274         while(_token > 0) {
00275             Statement();
00276             if(_lex._prevtoken != _SC('(') && _lex._prevtoken != _SC(';')) OptionalSemicolon();
00277         }
00278         _fs->SetStackSize(stacksize);
00279         _fs->AddLineInfos(_lex._currentline, _lineinfo, true);
00280         _fs->AddInstruction(_OP_RETURN, 0xFF);
00281         _fs->SetStackSize(0);
00282         o = _fs->BuildProto();
00283 #ifdef _DEBUG_DUMP
00284         _fs->Dump(_funcproto(o));
00285 #endif
00286     }
00287     else {
00288         if(_raiseerror && _ss(_vm)->_compilererrorhandler) {
00289             _ss(_vm)->_compilererrorhandler(_vm, _compilererror, sq_type(_sourcename) ==
00290 OT_STRING?_stringval(_sourcename):_SC("unknown"),
00291             _lex._currentline, _lex._currentcolumn);
00292         }
00293         _vm->_lasterror = SQString::Create(_ss(_vm), _compilererror, -1);
00294         return false;
00295     }
00296     return true;
00297 }
00298 void Statements()
00299 {
00300     while(_token != _SC('(') && _token != TK_DEFAULT && _token != TK_CASE) {
00301         Statement();
00302         if(_lex._prevtoken != _SC('(') && _lex._prevtoken != _SC(';')) OptionalSemicolon();
00303     }
00304 }
00305 void Statement(bool closeframe = true)
00306 {

```

```

00336     _fs->AddLineInfos(_lex._currentline, _lineinfo);
00337     switch(_token){
00338     case _SC(';'): Lex(); break;
00339     case TK_IF: IfStatement(); break;
00340     case TK_WHILE: WhileStatement(); break;
00341     case TK_DO: DoWhileStatement(); break;
00342     case TK_FOR: ForStatement(); break;
00343     case TK_FOREACH: ForEachStatement(); break;
00344     case TK_SWITCH: SwitchStatement(); break;
00345     case TK_LOCAL: LocalDeclStatement(); break;
00346     case TK_RETURN:
00347     case TK_YIELD: {
00348         SQOpcode op;
00349         if(_token == TK_RETURN) {
00350             op = _OP_RETURN;
00351         }
00352         else {
00353             op = _OP_YIELD;
00354             _fs->bgenerator = true;
00355         }
00356         Lex();
00357         if(!IsEndOfStatement()) {
00358             SQInteger retexp = _fs->GetCurrentPos()+1;
00359             CommaExpr();
00360             if(op == _OP_RETURN && _fs->_traps > 0)
00361                 _fs->AddInstruction(_OP_POPTRAP, _fs->_traps, 0);
00362             _fs->_returnexp = retexp;
00363             _fs->AddInstruction(op, 1, _fs->PopTarget(), _fs->GetStackSize());
00364         }
00365         else{
00366             if(op == _OP_RETURN && _fs->_traps > 0)
00367                 _fs->AddInstruction(_OP_POPTRAP, _fs->_traps, 0);
00368             _fs->_returnexp = -1;
00369             _fs->AddInstruction(op, 0xFF, 0, _fs->GetStackSize());
00370         }
00371         break;}
00372     case TK_BREAK:
00373         if(_fs->_breaktargets.size() <= 0)Error(_SC("'break' has to be in a loop block"));
00374         if(_fs->_breaktargets.top() > 0){
00375             _fs->AddInstruction(_OP_POPTRAP, _fs->_breaktargets.top(), 0);
00376         }
00377         RESOLVE_OUTERS();
00378         _fs->AddInstruction(_OP_JMP, 0, -1234);
00379         _fs->_unresolvedbreaks.push_back(_fs->GetCurrentPos());
00380         Lex();
00381         break;
00382     case TK_CONTINUE:
00383         if(_fs->_continuetargets.size() <= 0)Error(_SC("'continue' has to be in a loop block"));
00384         if(_fs->_continuetargets.top() > 0) {
00385             _fs->AddInstruction(_OP_POPTRAP, _fs->_continuetargets.top(), 0);
00386         }
00387         RESOLVE_OUTERS();
00388         _fs->AddInstruction(_OP_JMP, 0, -1234);
00389         _fs->_unresolvedcontinues.push_back(_fs->GetCurrentPos());
00390         Lex();
00391         break;
00392     case TK_FUNCTION:
00393         FunctionStatement();
00394         break;
00395     case TK_CLASS:
00396         ClassStatement();
00397         break;
00398     case TK_ENUM:
00399         EnumStatement();
00400         break;
00401     case _SC('{'): {
00402         BEGIN_SCOPE();
00403         Lex();
00404         Statements();
00405         Expect(_SC('}'));
00406         if(closeframe) {
00407             END_SCOPE();
00408         }
00409         else {
00410             END_SCOPE_NO_CLOSE();
00411         }
00412     }
00413     break;
00414     case TK_TRY:
00415         TryCatchStatement();
00416         break;
00417     case TK_THROW:
00418         Lex();
00419         CommaExpr();
00420         _fs->AddInstruction(_OP_THROW, _fs->PopTarget());
00421         break;
00422     case TK_CONST:

```

```

00423         {
00424             Lex();
00425             SQObject id = Expect(TK_IDENTIFIER);
00426             Expect('=');
00427             SQObject val = ExpectScalar();
00428             OptionalSemicolon();
00429             SQTable *enums = _table(_ss(_vm)->_consts);
00430             SQObjectPtr strongid = id;
00431             enums->NewSlot(strongid, SQObjectPtr(val));
00432             strongid.Null();
00433         }
00434         break;
00435     default:
00436         CommaExpr();
00437         _fs->DiscardTarget();
00438         //_fs->PopTarget();
00439         break;
00440     }
00441     _fs->SnoozeOpt();
00442 }
00448 void EmitDerefOp(SQOpcode op)
00449 {
00450     SQInteger val = _fs->PopTarget();
00451     SQInteger key = _fs->PopTarget();
00452     SQInteger src = _fs->PopTarget();
00453     _fs->AddInstruction(op, _fs->PushTarget(), src, key, val);
00454 }
00461 void Emit2ArgsOP(SQOpcode op, SQInteger p3 = 0)
00462 {
00463     SQInteger p2 = _fs->PopTarget(); //src in OP_GET
00464     SQInteger p1 = _fs->PopTarget(); //key in OP_GET
00465     _fs->AddInstruction(op, _fs->PushTarget(), p1, p2, p3);
00466 }
00474 void EmitCompoundArith(SQInteger tok, SQInteger etype, SQInteger pos)
00475 {
00476     /* Generate code depending on the expression type */
00477     switch(etype) {
00478     case LOCAL:
00479         SQInteger p2 = _fs->PopTarget(); //src in OP_GET
00480         SQInteger p1 = _fs->PopTarget(); //key in OP_GET
00481         _fs->PushTarget(p1);
00482         //EmitCompArithLocal(tok, p1, p1, p2);
00483         _fs->AddInstruction(ChooseArithOpByToken(tok), p1, p2, p1, 0);
00484         _fs->SnoozeOpt();
00485     }
00486     break;
00487     case OBJECT:
00488     case BASE:
00489     {
00490         SQInteger val = _fs->PopTarget();
00491         SQInteger key = _fs->PopTarget();
00492         SQInteger src = _fs->PopTarget();
00493         /* _OP_COMPARITH mixes dest obj and source val in the arg1 */
00494         _fs->AddInstruction(_OP_COMPARITH, _fs->PushTarget(), (src<<16)|val, key,
ChooseCompArithCharByToken(tok));
00495     }
00496     break;
00497     case OUTER:
00498     {
00499         SQInteger val = _fs->TopTarget();
00500         SQInteger tmp = _fs->PushTarget();
00501         _fs->AddInstruction(_OP_GETOUTER, tmp, pos);
00502         _fs->AddInstruction(ChooseArithOpByToken(tok), tmp, val, tmp, 0);
00503         _fs->PopTarget();
00504         _fs->PopTarget();
00505         _fs->AddInstruction(_OP_SETOUTER, _fs->PushTarget(), pos, tmp);
00506     }
00507     break;
00508 }
00509 }
00515 void CommaExpr()
00516 {
00517     for(Expression(); _token == ','; _fs->PopTarget(), Lex(), CommaExpr());
00518 }
00524 void Expression()
00525 {
00526     SQExpState es = _es;
00527     _es.etype = EXPR;
00528     _es.epos = -1;
00529     _es.donot_get = false;
00530     LogicalOrExp();
00531     switch(_token) {
00532     case _SC('='):
00533     case TK_NEWSLOT:
00534     case TK_MINUSEQ:
00535     case TK_PLUSEQ:
00536     case TK_MULEQ:

```

```

00537     case TK_DIVEQ:
00538     case TK_MODEQ: {
00539         SQInteger op = _token;
00540         SQInteger ds = _es.etype;
00541         SQInteger pos = _es.epos;
00542         if(ds == EXPR) Error(_SC("can't assign expression"));
00543         else if(ds == BASE) Error(_SC("'base' cannot be modified"));
00544         Lex(); Expression();
00545
00546         switch(op) {
00547         case TK_NEWSLOT:
00548             if(ds == OBJECT || ds == BASE)
00549                 EmitDerefOp(_OP_NEWSLOT);
00550             else //if _derefstate != Deref_NO_DEREF && Deref_FIELD so is the index of a local
00551                 Error(_SC("can't 'create' a local slot"));
00552             break;
00553         case _SC('='): //ASSIGN
00554             switch(ds) {
00555             case LOCAL:
00556                 {
00557                     SQInteger src = _fs->PopTarget();
00558                     SQInteger dst = _fs->TopTarget();
00559                     _fs->AddInstruction(_OP_MOVE, dst, src);
00560                 }
00561             break;
00562             case OBJECT:
00563             case BASE:
00564                 EmitDerefOp(_OP_SET);
00565                 break;
00566             case OUTER:
00567                 {
00568                     SQInteger src = _fs->PopTarget();
00569                     SQInteger dst = _fs->PushTarget();
00570                     _fs->AddInstruction(_OP_SETOUTER, dst, pos, src);
00571                 }
00572             break;
00573             case TK_MINUSEQ:
00574             case TK_PLUSEQ:
00575             case TK_MULEQ:
00576             case TK_DIVEQ:
00577             case TK_MODEQ:
00578                 EmitCompoundArith(op, ds, pos);
00579                 break;
00580             }
00581         }
00582         break;
00583     case _SC('?'): {
00584         Lex();
00585         _fs->AddInstruction(_OP_JZ, _fs->PopTarget());
00586         SQInteger jzpos = _fs->GetCurrentPos();
00587         SQInteger trg = _fs->PushTarget();
00588         Expression();
00589         SQInteger first_exp = _fs->PopTarget();
00590         if(trg != first_exp) _fs->AddInstruction(_OP_MOVE, trg, first_exp);
00591         SQInteger endfirstexp = _fs->GetCurrentPos();
00592         _fs->AddInstruction(_OP_JUMP, 0, 0);
00593         Expect(_SC(':'));
00594         SQInteger jmppos = _fs->GetCurrentPos();
00595         Expression();
00596         SQInteger second_exp = _fs->PopTarget();
00597         if(trg != second_exp) _fs->AddInstruction(_OP_MOVE, trg, second_exp);
00598         _fs->SetInstructionParam(jmppos, 1, _fs->GetCurrentPos() - jmppos);
00599         _fs->SetInstructionParam(jzpos, 1, endfirstexp - jzpos + 1);
00600         _fs->SnoozeOpt();
00601     }
00602     break;
00603 }
00604 }
00605 _es = es;
00606 }
00614 template<typename T> void INVOKE_EXP(T f)
00615 {
00616     SQExpState es = _es;
00617     _es.etype = EXPR;
00618     _es.epos = -1;
00619     _es.donot_get = false;
00620     (this->*f)();
00621     _es = es;
00622 }
00632 template<typename T> void BIN_EXP(SQOpcode op, T f, SQInteger op3 = 0)
00633 {
00634     Lex();
00635     INVOKE_EXP(f);
00636     SQInteger op1 = _fs->PopTarget(); SQInteger op2 = _fs->PopTarget();
00637     _fs->AddInstruction(op, _fs->PushTarget(), op1, op2, op3);
00638     _es.etype = EXPR;
00639 }

```

```

00644 void LogicalOrExp()
00645 {
00646     LogicalAndExp();
00647     for(;;) if(_token == TK_OR) {
00648         SQInteger first_exp = _fs->PopTarget();
00649         SQInteger trg = _fs->PushTarget();
00650         _fs->AddInstruction(_OP_OR, trg, 0, first_exp, 0);
00651         SQInteger jpos = _fs->GetCurrentPos();
00652         if(trg != first_exp) _fs->AddInstruction(_OP_MOVE, trg, first_exp);
00653         Lex(); INVOKE_EXP(&SQCompiler::LogicalOrExp);
00654         _fs->SnoozeOpt();
00655         SQInteger second_exp = _fs->PopTarget();
00656         if(trg != second_exp) _fs->AddInstruction(_OP_MOVE, trg, second_exp);
00657         _fs->SnoozeOpt();
00658         _fs->SetInstructionParam(jpos, 1, (_fs->GetCurrentPos() - jpos));
00659         _es.etype = EXPR;
00660         break;
00661     }else return;
00662 }
00663 void LogicalAndExp()
00664 {
00665     BitwiseOrExp();
00666     for(;;) switch(_token) {
00667     case TK_AND: {
00668         SQInteger first_exp = _fs->PopTarget();
00669         SQInteger trg = _fs->PushTarget();
00670         _fs->AddInstruction(_OP_AND, trg, 0, first_exp, 0);
00671         SQInteger jpos = _fs->GetCurrentPos();
00672         if(trg != first_exp) _fs->AddInstruction(_OP_MOVE, trg, first_exp);
00673         Lex(); INVOKE_EXP(&SQCompiler::LogicalAndExp);
00674         _fs->SnoozeOpt();
00675         SQInteger second_exp = _fs->PopTarget();
00676         if(trg != second_exp) _fs->AddInstruction(_OP_MOVE, trg, second_exp);
00677         _fs->SnoozeOpt();
00678         _fs->SetInstructionParam(jpos, 1, (_fs->GetCurrentPos() - jpos));
00679         _es.etype = EXPR;
00680         break;
00681     }
00682     default:
00683         return;
00684     }
00685 }
00686 void BitwiseOrExp()
00687 {
00688     BitwiseXorExp();
00689     for(;;) if(_token == _SC('|'))
00690     {BIN_EXP(_OP_BITW, &SQCompiler::BitwiseXorExp, BW_OR);
00691     }else return;
00692 }
00693 void BitwiseXorExp()
00694 {
00695     BitwiseAndExp();
00696     for(;;) if(_token == _SC('^'))
00697     {BIN_EXP(_OP_BITW, &SQCompiler::BitwiseAndExp, BW_XOR);
00698     }else return;
00699 }
00700 void BitwiseAndExp()
00701 {
00702     EqExp();
00703     for(;;) if(_token == _SC('&'))
00704     {BIN_EXP(_OP_BITW, &SQCompiler::EqExp, BW_AND);
00705     }else return;
00706 }
00707 void EqExp()
00708 {
00709     CompExp();
00710     for(;;) switch(_token) {
00711     case TK_EQ: BIN_EXP(_OP_EQ, &SQCompiler::CompExp); break;
00712     case TK_NE: BIN_EXP(_OP_NE, &SQCompiler::CompExp); break;
00713     case TK_3WAYSCMP: BIN_EXP(_OP_CMP, &SQCompiler::CompExp, CMP_3W); break;
00714     default: return;
00715     }
00716 }
00717 void CompExp()
00718 {
00719     ShiftExp();
00720     for(;;) switch(_token) {
00721     case _SC('>'): BIN_EXP(_OP_CMP, &SQCompiler::ShiftExp, CMP_G); break;
00722     case _SC('<'): BIN_EXP(_OP_CMP, &SQCompiler::ShiftExp, CMP_L); break;
00723     case TK_GE: BIN_EXP(_OP_CMP, &SQCompiler::ShiftExp, CMP_GE); break;
00724     case TK_LE: BIN_EXP(_OP_CMP, &SQCompiler::ShiftExp, CMP_LE); break;
00725     case TK_IN: BIN_EXP(_OP_EXISTS, &SQCompiler::ShiftExp); break;
00726     case TK_INSTANCEOF: BIN_EXP(_OP_INSTANCEOF, &SQCompiler::ShiftExp); break;
00727     default: return;
00728     }
00729 }
00730 }

```

```

00759 void ShiftExp()
00760 {
00761     PlusExp();
00762     for(;;) switch(_token) {
00763         case TK_USHIFTR: BIN_EXP(_OP_BITW, &SQCompiler::PlusExp, BW_USHIFTR); break;
00764         case TK_SHIFTL: BIN_EXP(_OP_BITW, &SQCompiler::PlusExp, BW_SHIFTL); break;
00765         case TK_SHIFTR: BIN_EXP(_OP_BITW, &SQCompiler::PlusExp, BW_SHIFTR); break;
00766         default: return;
00767     }
00768 }
00775 SQOpcode ChooseArithOpByToken(SQInteger tok)
00776 {
00777     switch(tok) {
00778         case TK_PLUSEQ: case '+': return _OP_ADD;
00779         case TK_MINUSEQ: case '-': return _OP_SUB;
00780         case TK_MULEQ: case '*': return _OP_MUL;
00781         case TK_DIVEQ: case '/': return _OP_DIV;
00782         case TK_MODEQ: case '%': return _OP_MOD;
00783         default: assert(0);
00784     }
00785     return _OP_ADD;
00786 }
00793 SQInteger ChooseCompArithCharByToken(SQInteger tok)
00794 {
00795     SQInteger oper;
00796     switch(tok){
00797         case TK_MINUSEQ: oper = '-'; break;
00798         case TK_PLUSEQ: oper = '+'; break;
00799         case TK_MULEQ: oper = '*'; break;
00800         case TK_DIVEQ: oper = '/'; break;
00801         case TK_MODEQ: oper = '%'; break;
00802         default: oper = 0; //shut up compiler
00803         assert(0); break;
00804     };
00805     return oper;
00806 }
00811 void PlusExp()
00812 {
00813     MultExp();
00814     for(;;) switch(_token) {
00815         case _SC('+'): case _SC('-'):
00816             BIN_EXP(ChooseArithOpByToken(_token), &SQCompiler::MultExp); break;
00817         default: return;
00818     }
00819 }
00820
00825 void MultExp()
00826 {
00827     PrefixedExpr();
00828     for(;;) switch(_token) {
00829         case _SC('*'): case _SC('/'): case _SC('%'):
00830             BIN_EXP(ChooseArithOpByToken(_token), &SQCompiler::PrefixedExpr); break;
00831         default: return;
00832     }
00833 }
00839 void PrefixedExpr()
00840 {
00841     SQInteger pos = Factor();
00842     for(;;) {
00843         switch(_token) {
00844             case _SC('.'):
00845                 pos = -1;
00846                 Lex();
00847
00848                 _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
00849 _fs->GetConstant(Expect(TK_IDENTIFIER)));
00850                 if(_es.etype==BASE) {
00851                     Emit2ArgsOP(_OP_GET);
00852                     pos = _fs->TopTarget();
00853                     _es.etype = EXPR;
00854                     _es.epos = pos;
00855                 }
00856                 else {
00857                     if(NeedGet()) {
00858                         Emit2ArgsOP(_OP_GET);
00859                     }
00860                     _es.etype = OBJECT;
00861                 }
00862                 break;
00863             case _SC('['):
00864                 if(_lex._prevtoken == _SC('\n')) Error(_SC("cannot break deref/or comma needed after
[exp]=exp slot declaration"));
00865                 Lex(); Expression(); Expect(_SC(']'));
00866                 pos = -1;
00867                 if(_es.etype==BASE) {
00868                     Emit2ArgsOP(_OP_GET);
00869                     pos = _fs->TopTarget();

```

```

00869         _es.etype = EXPR;
00870         _es.epos = pos;
00871     }
00872     else {
00873         if(NeedGet()) {
00874             Emit2ArgsOP(_OP_GET);
00875         }
00876         _es.etype = OBJECT;
00877     }
00878     break;
00879 case TK_MINUSMINUS:
00880 case TK_PLUSPLUS:
00881     {
00882         if(IsEndOfStatement()) return;
00883         SQInteger diff = (_token==TK_MINUSMINUS) ? -1 : 1;
00884         Lex();
00885         switch(_es.etype)
00886         {
00887             case EXPR: Error(_SC("can't '++' or '--' an expression")); break;
00888             case OBJECT:
00889             case BASE:
00890                 if(_es.donot_get == true) { Error(_SC("can't '++' or '--' an
expression")); break; } //mmh dor this make sense?
00891                 Emit2ArgsOP(_OP_PINC, diff);
00892                 break;
00893             case LOCAL: {
00894                 SQInteger src = _fs->PopTarget();
00895                 _fs->AddInstruction(_OP_PINCL, _fs->PushTarget(), src, 0, diff);
00896             }
00897             break;
00898             case OUTER: {
00899                 SQInteger tmp1 = _fs->PushTarget();
00900                 SQInteger tmp2 = _fs->PushTarget();
00901                 _fs->AddInstruction(_OP_GETOUTER, tmp2, _es.epos);
00902                 _fs->AddInstruction(_OP_PINCL, tmp1, tmp2, 0, diff);
00903                 _fs->AddInstruction(_OP_SETOUTER, tmp2, _es.epos, tmp2);
00904                 _fs->PopTarget();
00905             }
00906         }
00907     }
00908     return;
00909     break;
00910 case _SC('('):
00911     switch(_es.etype) {
00912         case OBJECT: {
00913             SQInteger key = _fs->PopTarget(); /* location of the key */
00914             SQInteger table = _fs->PopTarget(); /* location of the object */
00915             SQInteger closure = _fs->PushTarget(); /* location for the closure */
00916             SQInteger ttarget = _fs->PushTarget(); /* location for 'this' pointer */
00917             _fs->AddInstruction(_OP_PREPCALL, closure, key, table, ttarget);
00918         }
00919         break;
00920         case BASE:
00921             //Emit2ArgsOP(_OP_GET);
00922             _fs->AddInstruction(_OP_MOVE, _fs->PushTarget(), 0);
00923             break;
00924         case OUTER:
00925             _fs->AddInstruction(_OP_GETOUTER, _fs->PushTarget(), _es.epos);
00926             _fs->AddInstruction(_OP_MOVE, _fs->PushTarget(), 0);
00927             break;
00928         default:
00929             _fs->AddInstruction(_OP_MOVE, _fs->PushTarget(), 0);
00930     }
00931     _es.etype = EXPR;
00932     Lex();
00933     FunctionCallArgs();
00934     break;
00935     default: return;
00936 }
00937 }
00938 }
00945 SQInteger Factor()
00946 {
00947     // _es.etype = EXPR;
00948     switch(_token)
00949     {
00950         case TK_STRING_LITERAL:
00951             _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
00952             _fs->GetConstant(_fs->CreateString(_lex._svalue, _lex._longstr.size()-1));
00953             Lex();
00954             break;
00955         case TK_BASE:
00956             Lex();
00957             _fs->AddInstruction(_OP_GETBASE, _fs->PushTarget());
00958             _es.etype = BASE;
00959             _es.epos = _fs->TopTarget();
00959             return _es.epos;

```

```

00960         break;
00961     case TK_IDENTIFIER:
00962     case TK_CONSTRUCTOR:
00963     case TK_THIS: {
00964         SQObject id;
00965         SQObject constant;
00966
00967         switch(_token) {
00968             case TK_IDENTIFIER: id = _fs->CreateString(_lex._svalue); break;
00969             case TK_THIS: id = _fs->CreateString(_SC("this"), 4); break;
00970             case TK_CONSTRUCTOR: id = _fs->CreateString(_SC("constructor"), 11); break;
00971         }
00972
00973         SQInteger pos = -1;
00974         Lex();
00975         if((pos = _fs->GetLocalVariable(id)) != -1) {
00976             /* Handle a local variable (includes 'this') */
00977             _fs->PushTarget(pos);
00978             _es.etype = LOCAL;
00979             _es.epos = pos;
00980         }
00981
00982         else if((pos = _fs->GetOuterVariable(id)) != -1) {
00983             /* Handle a free var */
00984             if(NeedGet()) {
00985                 _es.epos = _fs->PushTarget();
00986                 _fs->AddInstruction(_OP_GETOUTER, _es.epos, pos);
00987                 /* _es.etype = EXPR; already default value */
00988             }
00989             else {
00990                 _es.etype = OUTER;
00991                 _es.epos = pos;
00992             }
00993         }
00994
00995         else if(_fs->IsConstant(id, constant)) {
00996             /* Handle named constant */
00997             SQObjectPtr constval;
00998             SQObject constid;
00999             if(sq_type(constant) == OT_TABLE) {
01000                 Expect('.');
01001                 constid = Expect(TK_IDENTIFIER);
01002                 if(!_table(constant)->Get(constid, constval)) {
01003                     constval.Null();
01004                     Error(_SC("invalid constant [%s.%s]"), _stringval(id),
01005                         _stringval(constid));
01006                 }
01007             }
01008             else {
01009                 constval = constant;
01010             }
01011             _es.epos = _fs->PushTarget();
01012
01013             /* generate direct or literal function depending on size */
01014             SQObjectType ctype = sq_type(constval);
01015             switch(ctype) {
01016                 case OT_INTEGER: EmitLoadConstInt(_integer(constval), _es.epos); break;
01017                 case OT_FLOAT: EmitLoadConstFloat(_float(constval), _es.epos); break;
01018                 case OT_BOOL: _fs->AddInstruction(_OP_LOADBOOL, _es.epos, _integer(constval));
01019             }
01020             default: _fs->AddInstruction(_OP_LOAD, _es.epos, _fs->GetConstant(constval));
01021         }
01022         _es.etype = EXPR;
01023     }
01024     else {
01025         /* Handle a non-local variable, aka a field. Push the 'this' pointer on
01026         * the virtual stack (always found in offset 0, so no instruction needs to
01027         * be generated), and push the key next. Generate an _OP_LOAD instruction
01028         * for the latter. If we are not using the variable as a dref expr, generate
01029         * the _OP_GET instruction.
01030         */
01031         _fs->PushTarget(0);
01032         _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(), _fs->GetConstant(id));
01033         if(NeedGet()) {
01034             Emit2ArgsOP(_OP_GET);
01035         }
01036         _es.etype = OBJECT;
01037     }
01038     return _es.epos;
01039 }
01040 break;
01041 case TK_DOUBLE_COLON: // "::"
01042     _fs->AddInstruction(_OP_LOADROOT, _fs->PushTarget());
01043     _es.etype = OBJECT;
01044     _token = _SC('.'); /* hack: drop into PrefixExpr, case '.' */
01045     _es.epos = -1;

```

```

01044         return _es.epos;
01045         break;
01046     case TK_NULL:
01047         _fs->AddInstruction(_OP_LOADNULLS, _fs->PushTarget(), 1);
01048         Lex();
01049         break;
01050     case TK_INTEGER: EmitLoadConstInt(_lex._nvalue, -1); Lex(); break;
01051     case TK_FLOAT: EmitLoadConstFloat(_lex._fvalue, -1); Lex(); break;
01052     case TK_TRUE: case TK_FALSE:
01053         _fs->AddInstruction(_OP_LOADBOOL, _fs->PushTarget(), _token == TK_TRUE?1:0);
01054         Lex();
01055         break;
01056     case _SC('['): {
01057         _fs->AddInstruction(_OP_NEWOBJ, _fs->PushTarget(), 0, 0, NOT_ARRAY);
01058         SQInteger apos = _fs->GetCurrentPos(), key = 0;
01059         Lex();
01060         while(_token != _SC(']')) {
01061             Expression();
01062             if(_token == _SC(',')) Lex();
01063             SQInteger val = _fs->PopTarget();
01064             SQInteger array = _fs->TopTarget();
01065             _fs->AddInstruction(_OP_APPENDARRAY, array, val, AAT_STACK);
01066             key++;
01067         }
01068         _fs->SetInstructionParam(apos, 1, key);
01069         Lex();
01070     }
01071     break;
01072     case _SC('{'):
01073         _fs->AddInstruction(_OP_NEWOBJ, _fs->PushTarget(), 0, NOT_TABLE);
01074         Lex(); ParseTableOrClass(_SC(', '), _SC('}'));
01075         break;
01076     case TK_FUNCTION: FunctionExp(); break;
01077     case _SC('@'): FunctionExp(true); break;
01078     case TK_CLASS: Lex(); ClassExp(); break;
01079     case _SC('~'):
01080         Lex();
01081         switch(_token) {
01082             case TK_INTEGER: EmitLoadConstInt(~_lex._nvalue, -1); Lex(); break;
01083             case TK_FLOAT: EmitLoadConstFloat(~_lex._fvalue, -1); Lex(); break;
01084             default: UnaryOP(_OP_NEG);
01085         }
01086         break;
01087     case _SC('!'): Lex(); UnaryOP(_OP_NOT); break;
01088     case _SC('~'):
01089         Lex();
01090         if(_token == TK_INTEGER) { EmitLoadConstInt(~_lex._nvalue, -1); Lex(); break; }
01091         UnaryOP(_OP_BWNOT);
01092         break;
01093     case TK_TYPEOF : Lex(); UnaryOP(_OP_TYPEOF); break;
01094     case TK_RESUME : Lex(); UnaryOP(_OP_RESUME); break;
01095     case TK_CLONE : Lex(); UnaryOP(_OP_CLONE); break;
01096     case TK_RAWCALL: Lex(); Expect('('); FunctionCallArgs(true); break;
01097     case TK_MINUSMINUS :
01098     case TK_PLUSPLUS : PrefixIncDec(_token); break;
01099     case TK_DELETE : DeleteExpr(); break;
01100     case _SC('('): Lex(); CommaExpr(); Expect(_SC(')'));
01101     break;
01102     case TK__LINE__: EmitLoadConstInt(_lex._currentline, -1); Lex(); break;
01103     case TK__FILE__: _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
01104     _fs->GetConstant(_sourcename)); Lex(); break;
01105     default: Error(_SC("expression expected"));
01106     }
01107     _es.etype = EXPR;
01108     return -1;
01109 }
01110
01111 void EmitLoadConstInt(SQInteger value, SQInteger target)
01112 {
01113     if(target < 0) {
01114         target = _fs->PushTarget();
01115     }
01116     if(value <= INT_MAX && value > INT_MIN) { //does it fit in 32 bits?
01117         _fs->AddInstruction(_OP_LOADINT, target, value);
01118     }
01119     else {
01120         _fs->AddInstruction(_OP_LOAD, target, _fs->GetNumericConstant(value));
01121     }
01122 }
01123
01124 void EmitLoadConstFloat(SQFloat value, SQInteger target)
01125 {
01126     if(target < 0) {
01127         target = _fs->PushTarget();
01128     }
01129     if(sizeof(SQFloat) == sizeof(SQInt32)) {
01130         _fs->AddInstruction(_OP_LOADFLOAT, target, *((SQInt32 *)&value));
01131     }
01132     else {

```

```

01144         _fs->AddInstruction(_OP_LOAD, target, _fs->GetNumericConstant(value));
01145     }
01146 }
01153 void UnaryOP(SQOpcode op)
01154 {
01155     PrefixedExpr();
01156     SQInteger src = _fs->PopTarget();
01157     _fs->AddInstruction(op, _fs->PushTarget(), src);
01158 }
01165 bool NeedGet()
01166 {
01167     switch(_token) {
01168     case _SC('='): case _SC('('): case TK_NEWSLIST: case TK_MODEQ: case TK_MULEQ:
01169     case TK_DIVEQ: case TK_MINUSEQ: case TK_PLUSEQ:
01170         return false;
01171     case TK_PLUSPLUS: case TK_MINUSMINUS:
01172         if (!IsEndOfStatement()) {
01173             return false;
01174         }
01175     }
01176     break;
01177 }
01178 return (!_es.donot_get || (_es.donot_get && (_token == _SC('.') || _token == _SC('['))));
01186 void FunctionCallArgs(bool rawcall = false)
01187 {
01188     SQInteger nargs = 1; //this
01189     while(_token != _SC(' ')) {
01190         Expression();
01191         MoveIfCurrentTargetIsLocal();
01192         nargs++;
01193         if(_token == _SC(',')) {
01194             Lex();
01195             if(_token == ' ') Error(_SC("expression expected, found ' '"));
01196         }
01197     }
01198     Lex();
01199     if (rawcall) {
01200         if (nargs < 3) Error(_SC("rawcall requires at least 2 parameters (callee and this)"));
01201         nargs -= 2; //removes callee and this from count
01202     }
01203     for(SQInteger i = 0; i < (nargs - 1); i++) _fs->PopTarget();
01204     SQInteger stackbase = _fs->PopTarget();
01205     SQInteger closure = _fs->PopTarget();
01206     _fs->AddInstruction(_OP_CALL, _fs->PushTarget(), closure, stackbase, nargs);
01207     if (_token == '{')
01208     {
01209         SQInteger retval = _fs->TopTarget();
01210         SQInteger nkeys = 0;
01211         Lex();
01212         while (_token != ' ') {
01213             switch (_token) {
01214             case _SC('['):
01215                 Lex(); CommaExpr(); Expect(_SC(']'));
01216                 Expect(_SC('=')); Expression();
01217                 break;
01218             default:
01219                 _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
01220                 _fs->GetConstant(Expect(TK_IDENTIFIER)));
01221                 Expect(_SC('=')); Expression();
01222                 break;
01223             }
01224             if (_token == ',') Lex();
01225             nkeys++;
01226             SQInteger val = _fs->PopTarget();
01227             SQInteger key = _fs->PopTarget();
01228             _fs->AddInstruction(_OP_SET, 0xFF, retval, key, val);
01229         }
01230     }
01231 }
01240 void ParseTableOrClass(SQInteger separator, SQInteger terminator)
01241 {
01242     SQInteger tpos = _fs->GetCurrentPos(), nkeys = 0;
01243     while(_token != terminator) {
01244         bool hasattrs = false;
01245         bool isstatic = false;
01246         //check if is an attribute
01247         if(separator == ';') {
01248             if(_token == TK_ATTR_OPEN) {
01249                 _fs->AddInstruction(_OP_NEWOBJ, _fs->PushTarget(), 0, NOT_TABLE); Lex();
01250                 ParseTableOrClass(' ', TK_ATTR_CLOSE);
01251                 hasattrs = true;
01252             }
01253             if(_token == TK_STATIC) {
01254                 isstatic = true;
01255                 Lex();
01256             }

```

```

01257         }
01258         switch(_token) {
01259         case TK_FUNCTION:
01260         case TK_CONSTRUCTOR:{
01261             SQInteger tk = _token;
01262             Lex();
01263             SQObject id = tk == TK_FUNCTION ? Expect(TK_IDENTIFIER) :
01264             _fs->CreateString(_SC("constructor"));
01265             _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(), _fs->GetConstant(id));
01266             SQInteger boundtarget = 0xFF;
01267             if (_token == _SC('(')) {
01268                 boundtarget = ParseBindEnv();
01269             }
01270             Expect(_SC('('));
01271             CreateFunction(id, boundtarget);
01272             _fs->AddInstruction(_OP_CLOSURE, _fs->PushTarget(), _fs->_functions.size() - 1,
01273             boundtarget);
01274             break;
01275         case _SC('['):
01276             Lex(); CommaExpr(); Expect(_SC(']'));
01277             Expect(_SC('=')); Expression();
01278             break;
01279         case TK_STRING_LITERAL: //JSON
01280             if(separator == ',') { //only works for tables
01281                 _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
01282                 _fs->GetConstant(Expect(TK_STRING_LITERAL)));
01283                 Expect(_SC(':')); Expression();
01284                 break;
01285             }
01286             default :
01287                 _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(),
01288                 _fs->GetConstant(Expect(TK_IDENTIFIER)));
01289                 Expect(_SC('=')); Expression();
01290             }
01291             if(_token == separator) Lex();//optional comma/semicolon
01292             nkeys++;
01293             SQInteger val = _fs->PopTarget();
01294             SQInteger key = _fs->PopTarget();
01295             SQInteger attrs = hasattrs ? _fs->PopTarget():-1;
01296             (void)attrs;
01297             assert((hasattrs && (attrs == key-1)) || !hasattrs);
01298             unsigned char flags =
01299             (hasattrs?NEW_SLOT_ATTRIBUTES_FLAG:0)|(isstatic?NEW_SLOT_STATIC_FLAG:0);
01300             SQInteger table = _fs->TopTarget(); //«BECAUSE OF THIS NO COMMON EMIT FUNC IS POSSIBLE
01301             if(separator == _SC(',')) { //hack recognizes a table from the separator
01302                 _fs->AddInstruction(_OP_NEWSLOT, 0xFF, table, key, val);
01303             }
01304             else {
01305                 _fs->AddInstruction(_OP_NEWSLOTA, flags, table, key, val); //this for classes only as
01306                 it invokes _newmember
01307             }
01308             if(separator == _SC(',')) //hack recognizes a table from the separator
01309                 _fs->SetInstructionParam(tpos, 1, nkeys);
01310             Lex();
01311         }
01312         void LocalDeclStatement()
01313         {
01314             SQObject varname;
01315             Lex();
01316             if(_token == TK_FUNCTION) {
01317                 SQInteger boundtarget = 0xFF;
01318                 Lex();
01319                 varname = Expect(TK_IDENTIFIER);
01320                 if (_token == _SC('(')) {
01321                     boundtarget = ParseBindEnv();
01322                 }
01323                 Expect(_SC('('));
01324                 CreateFunction(varname, 0xFF, false);
01325                 _fs->AddInstruction(_OP_CLOSURE, _fs->PushTarget(), _fs->_functions.size() - 1,
01326                 boundtarget);
01327                 _fs->PopTarget();
01328                 _fs->PushLocalVariable(varname);
01329                 return;
01330             }
01331             do {
01332                 varname = Expect(TK_IDENTIFIER);
01333                 if(_token == _SC('=')) {
01334                     Lex(); Expression();
01335                     SQInteger src = _fs->PopTarget();
01336                     SQInteger dest = _fs->PushTarget();
01337                     if(dest != src) _fs->AddInstruction(_OP_MOVE, dest, src);
01338                 }
01339                 else{

```

```

01342         _fs->AddInstruction(_OP_LOADNULLS, _fs->PushTarget(), 1);
01343     }
01344     _fs->PopTarget();
01345     _fs->PushLocalVariable(varname);
01346     if(_token == _SC(',',')) Lex(); else break;
01347 } while(1);
01348 }
01353 void IfBlock()
01354 {
01355     if (_token == _SC('{'))
01356     {
01357         BEGIN_SCOPE();
01358         Lex();
01359         Statements();
01360         Expect(_SC('}'));
01361         if (true) {
01362             END_SCOPE();
01363         }
01364         else {
01365             END_SCOPE_NO_CLOSE();
01366         }
01367     }
01368     else {
01369         //BEGIN_SCOPE();
01370         Statement();
01371         if (_lex._prevtoken != _SC('}') && _lex._prevtoken != _SC(';')) OptionalSemicolon();
01372         //END_SCOPE();
01373     }
01374 }
01380 void IfStatement()
01381 {
01382     SQInteger jmppos;
01383     bool haselse = false;
01384     Lex(); Expect(_SC('(')); CommaExpr(); Expect(_SC(')'));
01385     _fs->AddInstruction(_OP_JZ, _fs->PopTarget());
01386     SQInteger jnpos = _fs->GetCurrentPos();
01387
01388     IfBlock();
01389     //
01390     /*static int n = 0;
01391     if (_token != _SC('}') && _token != TK_ELSE) {
01392         printf("IF %d-----!!!!!!!\n", n);
01393         if (n == 5)
01394         {
01395             printf("asd");
01396         }
01397         n++;
01398         //OptionalSemicolon();
01399     }*/
01400
01401     SQInteger endifblock = _fs->GetCurrentPos();
01402     if(_token == TK_ELSE){
01403         haselse = true;
01404         //BEGIN_SCOPE();
01405         _fs->AddInstruction(_OP_JMP);
01406         jmppos = _fs->GetCurrentPos();
01407         Lex();
01408         //Statement(); if(_lex._prevtoken != _SC('}')) OptionalSemicolon();
01409         IfBlock();
01410         //END_SCOPE();
01411         _fs->SetInstructionParam(jmppos, 1, _fs->GetCurrentPos() - jmppos);
01412     }
01413     _fs->SetInstructionParam(jnpos, 1, endifblock - jnpos + (haselse?1:0));
01414 }
01423 void WhileStatement()
01424 {
01425     SQInteger jzpos, jmppos;
01426     jmppos = _fs->GetCurrentPos();
01427     Lex(); Expect(_SC('(')); CommaExpr(); Expect(_SC(')'));
01428
01429     BEGIN_BREAKABLE_BLOCK();
01430     _fs->AddInstruction(_OP_JZ, _fs->PopTarget());
01431     jzpos = _fs->GetCurrentPos();
01432     BEGIN_SCOPE();
01433
01434     Statement();
01435
01436     END_SCOPE();
01437     _fs->AddInstruction(_OP_JMP, 0, jmppos - _fs->GetCurrentPos() - 1);
01438     _fs->SetInstructionParam(jzpos, 1, _fs->GetCurrentPos() - jzpos);
01439
01440     END_BREAKABLE_BLOCK(jmppos);
01441 }
01446 void DoWhileStatement()

```

```

01447     {
01448         Lex();
01449         SQInteger jmptrg = _fs->GetCurrentPos();
01450         BEGIN_BREAKABLE_BLOCK()
01451         BEGIN_SCOPE();
01452         Statement();
01453         END_SCOPE();
01454         Expect(TK_WHILE);
01455         SQInteger continuetr = _fs->GetCurrentPos();
01456         Expect(_SC('(')); CommaExpr(); Expect(_SC(')'));
01457         _fs->AddInstruction(_OP_JZ, _fs->PopTarget(), 1);
01458         _fs->AddInstruction(_OP_JMP, 0, jmptrg - _fs->GetCurrentPos() - 1);
01459         END_BREAKABLE_BLOCK(continuetr);
01460     }
01461 void ForStatement()
01462 {
01463     Lex();
01464     BEGIN_SCOPE();
01465     Expect(_SC('('));
01466     if(_token == TK_LOCAL) LocalDeclStatement();
01467     else if(_token != _SC(';')) {
01468         CommaExpr();
01469         _fs->PopTarget();
01470     }
01471     Expect(_SC(';'));
01472     _fs->SnoozeOpt();
01473     SQInteger jmppos = _fs->GetCurrentPos();
01474     SQInteger jzpos = -1;
01475     if(_token != _SC(';')) { CommaExpr(); _fs->AddInstruction(_OP_JZ, _fs->PopTarget()); jzpos =
01476         _fs->GetCurrentPos(); }
01477     Expect(_SC(';'));
01478     _fs->SnoozeOpt();
01479     SQInteger expstart = _fs->GetCurrentPos() + 1;
01480     if(_token != _SC(')')) {
01481         CommaExpr();
01482         _fs->PopTarget();
01483     }
01484     Expect(_SC(')'));
01485     _fs->SnoozeOpt();
01486     SQInteger expend = _fs->GetCurrentPos();
01487     SQInteger expsize = (expend - expstart) + 1;
01488     SQInstructionVec exp;
01489     if(expsize > 0) {
01490         for(SQInteger i = 0; i < expsize; i++)
01491             exp.push_back(_fs->GetInstruction(expstart + i));
01492         _fs->PopInstructions(expsize);
01493     }
01494     BEGIN_BREAKABLE_BLOCK()
01495     Statement();
01496     SQInteger continuetr = _fs->GetCurrentPos();
01497     if(expsize > 0) {
01498         for(SQInteger i = 0; i < expsize; i++)
01499             _fs->AddInstruction(exp[i]);
01500     }
01501     _fs->AddInstruction(_OP_JMP, 0, jmppos - _fs->GetCurrentPos() - 1, 0);
01502     if(jzpos > 0) _fs->SetInstructionParam(jzpos, 1, _fs->GetCurrentPos() - jzpos);
01503     END_BREAKABLE_BLOCK(continuetr);
01504     END_SCOPE();
01505 }
01506 void ForEachStatement()
01507 {
01508     SQObject idxname, valname;
01509     Lex(); Expect(_SC('(')); valname = Expect(TK_IDENTIFIER);
01510     if(_token == _SC(',')) {
01511         idxname = valname;
01512         Lex(); valname = Expect(TK_IDENTIFIER);
01513     }
01514     else {
01515         idxname = _fs->CreateString(_SC("@INDEX@"));
01516     }
01517     Expect(TK_IN);
01518     //save the stack size
01519     BEGIN_SCOPE();
01520     //put the table in the stack(evaluate the table expression)
01521     Expression(); Expect(_SC(')'));
01522     SQInteger container = _fs->TopTarget();
01523     //push the index local var
01524     SQInteger indexpos = _fs->PushLocalVariable(idxname);
01525     _fs->AddInstruction(_OP_LOADNULLS, indexpos, 1);
01526     //push the value local var
01527     SQInteger valuepos = _fs->PushLocalVariable(valname);
01528     _fs->AddInstruction(_OP_LOADNULLS, valuepos, 1);
01529     //push reference index
01530     SQInteger itrpos = _fs->PushLocalVariable(_fs->CreateString(_SC("@ITERATOR@"))); //use invalid

```

```

    id to make it inaccessible
01542     _fs->AddInstruction(_OP_LOADNULLS, itrpos, 1);
01543     SQInteger jmppos = _fs->GetCurrentPos();
01544     _fs->AddInstruction(_OP_FOREACH, container, 0, indexpos);
01545     SQInteger foreachpos = _fs->GetCurrentPos();
01546     _fs->AddInstruction(_OP_POSTFOREACH, container, 0, indexpos);
01547     //generate the statement code
01548     BEGIN_BREAKABLE_BLOCK()
01549     Statement();
01550     _fs->AddInstruction(_OP_JMP, 0, jmppos - _fs->GetCurrentPos() - 1);
01551     _fs->SetInstructionParam(foreachpos, 1, _fs->GetCurrentPos() - foreachpos);
01552     _fs->SetInstructionParam(foreachpos + 1, 1, _fs->GetCurrentPos() - foreachpos);
01553     END_BREAKABLE_BLOCK(foreachpos - 1);
01554     //restore the local variable stack(remove index, val and ref idx)
01555     _fs->PopTarget();
01556     END_SCOPE();
01557 }
01563 void SwitchStatement()
01564 {
01565     Lex(); Expect(_SC('(')); CommaExpr(); Expect(_SC(')'));
01566     Expect(_SC('{'));
01567     SQInteger expr = _fs->TopTarget();
01568     bool bfirst = true;
01569     SQInteger tonextcondjmp = -1;
01570     SQInteger skipcondjmp = -1;
01571     SQInteger __nbreaks__ = _fs->unresolvedbreaks.size();
01572     _fs->breaktargets.push_back(0);
01573     while(_token == TK_CASE) {
01574         if(!bfirst) {
01575             _fs->AddInstruction(_OP_JMP, 0, 0);
01576             skipcondjmp = _fs->GetCurrentPos();
01577             _fs->SetInstructionParam(tonextcondjmp, 1, _fs->GetCurrentPos() - tonextcondjmp);
01578         }
01579         //condition
01580         Lex(); Expression(); Expect(_SC(':'));
01581         SQInteger trg = _fs->PopTarget();
01582         SQInteger eqtarget = trg;
01583         bool local = _fs->IsLocal(trg);
01584         if(local) {
01585             eqtarget = _fs->PushTarget(); //we need to allocate a extra reg
01586         }
01587         _fs->AddInstruction(_OP_EQ, eqtarget, trg, expr);
01588         _fs->AddInstruction(_OP_JZ, eqtarget, 0);
01589         if(local) {
01590             _fs->PopTarget();
01591         }
01592
01593         //end condition
01594         if(skipcondjmp != -1) {
01595             _fs->SetInstructionParam(skipcondjmp, 1, (_fs->GetCurrentPos() - skipcondjmp));
01596         }
01597         tonextcondjmp = _fs->GetCurrentPos();
01598         BEGIN_SCOPE();
01599         Statements();
01600         END_SCOPE();
01601         bfirst = false;
01602     }
01603     if(tonextcondjmp != -1)
01604         _fs->SetInstructionParam(tonextcondjmp, 1, _fs->GetCurrentPos() - tonextcondjmp);
01605     if(_token == TK_DEFAULT) {
01606         Lex(); Expect(_SC(':'));
01607         BEGIN_SCOPE();
01608         Statements();
01609         END_SCOPE();
01610     }
01611     Expect(_SC('}'));
01612     _fs->PopTarget();
01613     __nbreaks__ = _fs->unresolvedbreaks.size() - __nbreaks__;
01614     if(__nbreaks__ > 0) ResolveBreaks(_fs, __nbreaks__);
01615     _fs->breaktargets.pop_back();
01616 }
01622 void FunctionStatement()
01623 {
01624     SQObject id;
01625     Lex(); id = Expect(TK_IDENTIFIER);
01626     _fs->PushTarget(0);
01627     _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(), _fs->GetConstant(id));
01628     if(_token == TK_DOUBLE_COLON) Emit2ArgsOP(_OP_GET);
01629
01630     while(_token == TK_DOUBLE_COLON) {
01631         Lex();
01632         id = Expect(TK_IDENTIFIER);
01633         _fs->AddInstruction(_OP_LOAD, _fs->PushTarget(), _fs->GetConstant(id));
01634         if(_token == TK_DOUBLE_COLON) Emit2ArgsOP(_OP_GET);
01635     }
01636     SQInteger boundtarget = 0xFF;
01637     if(_token == _SC('[')) {

```

```

01638         boundtarget = ParseBindEnv();
01639     }
01640     Expect(_SC('('));
01641     CreateFunction(id, boundtarget);
01642     _fs->AddInstruction(_OP_CLOSURE, _fs->PushTarget(), _fs->_functions.size() - 1, boundtarget);
01643     EmitDerefOp(_OP_NEWSLOT);
01644     _fs->PopTarget();
01645 }
01651 void ClassStatement()
01652 {
01653     SQExpState es;
01654     Lex();
01655     es = _es;
01656     _es.donot_get = true;
01657     PrefixedExpr();
01658     if(_es.etype == EXPR) {
01659         Error(_SC("invalid class name"));
01660     }
01661     else if(_es.etype == OBJECT || _es.etype == BASE) {
01662         ClassExp();
01663         EmitDerefOp(_OP_NEWSLOT);
01664         _fs->PopTarget();
01665     }
01666     else {
01667         Error(_SC("cannot create a class in a local with the syntax(class <local>)"));
01668     }
01669     _es = es;
01670 }
01676 SQObject ExpectScalar()
01677 {
01678     SQObject val;
01679     val._type = OT_NULL; val._unVal.nInteger = 0; //shut up GCC 4.x
01680     switch(_token) {
01681         case TK_INTEGER:
01682             val._type = OT_INTEGER;
01683             val._unVal.nInteger = _lex._nvalue;
01684             break;
01685         case TK_FLOAT:
01686             val._type = OT_FLOAT;
01687             val._unVal.fFloat = _lex._fvalue;
01688             break;
01689         case TK_STRING_LITERAL:
01690             val = _fs->CreateString(_lex._svalue, _lex._longstr.size()-1);
01691             break;
01692         case TK_TRUE:
01693         case TK_FALSE:
01694             val._type = OT_BOOL;
01695             val._unVal.nInteger = _token == TK_TRUE ? 1 : 0;
01696             break;
01697         case '-':
01698             Lex();
01699             switch(_token)
01700             {
01701                 case TK_INTEGER:
01702                     val._type = OT_INTEGER;
01703                     val._unVal.nInteger = -_lex._nvalue;
01704                     break;
01705                 case TK_FLOAT:
01706                     val._type = OT_FLOAT;
01707                     val._unVal.fFloat = -_lex._fvalue;
01708                     break;
01709                 default:
01710                     Error(_SC("scalar expected : integer, float"));
01711             }
01712             break;
01713         default:
01714             Error(_SC("scalar expected : integer, float, or string"));
01715     }
01716     Lex();
01717     return val;
01718 }
01724 void EnumStatement()
01725 {
01726     Lex();
01727     SQObject id = Expect(TK_IDENTIFIER);
01728     Expect(_SC('{'));
01729
01730     SQObject table = _fs->CreateTable();
01731     SQInteger nval = 0;
01732     while(_token != _SC('}')) {
01733         SQObject key = Expect(TK_IDENTIFIER);
01734         SQObject val;
01735         if(_token == _SC('=')) {
01736             Lex();
01737             val = ExpectScalar();
01738         }
01739         else {

```

```

01740         val._type = OT_INTEGER;
01741         val._unVal.nInteger = nval++;
01742     }
01743     _table(table)->NewSlot(SQObjectPtr(key),SQObjectPtr(val));
01744     if(_token == ',') Lex();
01745 }
01746 SQTable *enums = _table(_ss(_vm)->_consts);
01747 SQObjectPtr strongid = id;
01748 enums->NewSlot(SQObjectPtr(strongid),SQObjectPtr(table));
01749 strongid.Null();
01750 Lex();
01751 }
01752 void TryCatchStatement()
01753 {
01754     SQObject exid;
01755     Lex();
01756     _fs->AddInstruction(_OP_PUSHTRAP,0,0);
01757     _fs->_traps++;
01758     if(_fs->_breaktargets.size()) _fs->_breaktargets.top()++;
01759     if(_fs->_continuetargets.size()) _fs->_continuetargets.top()++;
01760     SQInteger trappos = _fs->GetCurrentPos();
01761     {
01762         BEGIN_SCOPE();
01763         Statement();
01764         END_SCOPE();
01765     }
01766     _fs->_traps--;
01767     _fs->AddInstruction(_OP_POPTRAP, 1, 0);
01768     if(_fs->_breaktargets.size()) _fs->_breaktargets.top()--;
01769     if(_fs->_continuetargets.size()) _fs->_continuetargets.top()--;
01770     _fs->AddInstruction(_OP_JMP, 0, 0);
01771     SQInteger jmppos = _fs->GetCurrentPos();
01772     _fs->SetInstructionParam(trappos, 1, (_fs->GetCurrentPos() - trappos));
01773     Expect(TK_CATCH); Expect(_SC('(')); exid = Expect(TK_IDENTIFIER); Expect(_SC(')'));
01774     {
01775         BEGIN_SCOPE();
01776         SQInteger ex_target = _fs->PushLocalVariable(exid);
01777         _fs->SetInstructionParam(trappos, 0, ex_target);
01778         Statement();
01779         _fs->SetInstructionParams(jmppos, 0, (_fs->GetCurrentPos() - jmppos), 0);
01780         END_SCOPE();
01781     }
01782 }
01783 SQInteger ParseBindEnv()
01784 {
01785     SQInteger boundtarget;
01786     Lex();
01787     Expression();
01788     boundtarget = _fs->TopTarget();
01789     Expect(_SC(']'));
01790     return boundtarget;
01791 }
01792 void FunctionExp(bool lambda = false)
01793 {
01794     Lex();
01795     SQInteger boundtarget = 0xFF;
01796     if (_token == _SC('(')) {
01797         boundtarget = ParseBindEnv();
01798     }
01799     Expect(_SC('('));
01800     SQObjectPtr dummy;
01801     CreateFunction(dummy, boundtarget, lambda);
01802     _fs->AddInstruction(_OP_CLOSURE, _fs->PushTarget(), _fs->_functions.size() - 1, boundtarget);
01803 }
01804 void ClassExp()
01805 {
01806     SQInteger base = -1;
01807     SQInteger attrs = -1;
01808     if(_token == TK_EXTENDS) {
01809         Lex(); Expression();
01810         base = _fs->TopTarget();
01811     }
01812     if(_token == TK_ATTR_OPEN) {
01813         Lex();
01814         _fs->AddInstruction(_OP_NEWOBJ, _fs->PushTarget(),0,NOT_TABLE);
01815         ParseTableOrClass(_SC(','),TK_ATTR_CLOSE);
01816         attrs = _fs->TopTarget();
01817     }
01818     Expect(_SC('{'));
01819     if(attrs != -1) _fs->PopTarget();
01820     if(base != -1) _fs->PopTarget();
01821     _fs->AddInstruction(_OP_NEWOBJ, _fs->PushTarget(), base, attrs,NOT_CLASS);
01822     ParseTableOrClass(_SC(';'),_SC('}'));
01823 }
01824 void DeleteExpr()
01825 {
01826     SQExpState es;

```

```

01854     Lex();
01855     es = _es;
01856     _es.donot_get = true;
01857     PrefixedExpr();
01858     if(_es.etype==EXPR) Error(_SC("can't delete an expression"));
01859     if(_es.etype==OBJECT || _es.etype==BASE) {
01860         Emit2ArgsOP(_OP_DELETE);
01861     }
01862     else {
01863         Error(_SC("cannot delete an (outer) local"));
01864     }
01865     _es = es;
01866 }
01872 void PrefixIncDec(SQInteger token)
01873 {
01874     SQExpState es;
01875     SQInteger diff = (token==TK_MINUSMINUS) ? -1 : 1;
01876     Lex();
01877     es = _es;
01878     _es.donot_get = true;
01879     PrefixedExpr();
01880     if(_es.etype==EXPR) {
01881         Error(_SC("can't '++' or '--' an expression"));
01882     }
01883     else if(_es.etype==OBJECT || _es.etype==BASE) {
01884         Emit2ArgsOP(_OP_INC, diff);
01885     }
01886     else if(_es.etype==LOCAL) {
01887         SQInteger src = _fs->TopTarget();
01888         _fs->AddInstruction(_OP_INCL, src, src, 0, diff);
01889     }
01890 }
01891 else if(_es.etype==OUTER) {
01892     SQInteger tmp = _fs->PushTarget();
01893     _fs->AddInstruction(_OP_GETOUTER, tmp, _es.epos);
01894     _fs->AddInstruction(_OP_INCL, tmp, tmp, 0, diff);
01895     _fs->AddInstruction(_OP_SETOUTER, tmp, _es.epos, tmp);
01896 }
01897 _es = es;
01898 }
01907 void CreateFunction(SQObject &name, SQInteger boundtarget, bool lambda = false)
01908 {
01909     SQFuncState *funcstate = _fs->PushChildState(_ss(_vm));
01910     funcstate->_name = name;
01911     SQObject paramname;
01912     funcstate->AddParameter(_fs->CreateString(_SC("this")));
01913     funcstate->_sourcename = _sourcename;
01914     SQInteger defparams = 0;
01915     while(_token!=_SC('(')) {
01916         if(_token == TK_VARPARAMS) {
01917             if(defparams > 0) Error(_SC("function with default parameters cannot have variable
number of parameters"));
            funcstate->AddParameter(_fs->CreateString(_SC("vargv")));
            funcstate->_varparams = true;
01920             Lex();
01921             if(_token != _SC('(')) Error(_SC("expected ' '"));
01922             break;
01923         }
01924         else {
01925             paramname = Expect(TK_IDENTIFIER);
01926             funcstate->AddParameter(paramname);
01927             if(_token == _SC('=')) {
01928                 Lex();
01929                 Expression();
01930                 funcstate->AddDefaultParam(_fs->TopTarget());
01931                 defparams++;
01932             }
01933             else {
01934                 if(defparams > 0) Error(_SC("expected '='"));
01935             }
01936             if(_token == _SC(',',')) Lex();
01937             else if(_token != _SC('(')) Error(_SC("expected ' ' or ', '"));
01938         }
01939     }
01940     Expect(_SC('('));
01941     if (boundtarget != 0xFF) {
01942         _fs->PopTarget();
01943     }
01944     for(SQInteger n = 0; n < defparams; n++) {
01945         _fs->PopTarget();
01946     }
01947     SQFuncState *currchunk = _fs;
01948     _fs = funcstate;
01949     if(lambda) {
01950         Expression();
01951         _fs->AddInstruction(_OP_RETURN, 1, _fs->PopTarget());
01952     }

```

```

01953         else {
01954             Statement(false);
01955         }
01956         funcstate->AddLineInfos(_lex._prevtoken == _SC('\n')?_lex._lasttokenline:_lex._currentline,
_lineinfo, true);
01957         funcstate->AddInstruction(_OP_RETURN, -1);
01958         funcstate->SetStackSize(0);
01959
01960         SQFunctionProto *func = funcstate->BuildProto();
01961 #ifdef _DEBUG_DUMP
01962         funcstate->Dump(func);
01963 #endif
01964         _fs = currchunk;
01965         _fs->_functions.push_back(func);
01966         _fs->PopChildState();
01967     }
01975     void ResolveBreaks(SQFuncState *funcstate, SQInteger ntoresolve)
01976     {
01977         while(ntoresolve > 0) {
01978             SQInteger pos = funcstate->_unresolvedbreaks.back();
01979             funcstate->_unresolvedbreaks.pop_back();
01980             //set the jmp instruction
01981             funcstate->SetInstructionParams(pos, 0, funcstate->GetCurrentPos() - pos, 0);
01982             ntoresolve--;
01983         }
01984     }
01993     void ResolveContinues(SQFuncState *funcstate, SQInteger ntoresolve, SQInteger targetpos)
01994     {
01995         while(ntoresolve > 0) {
01996             SQInteger pos = funcstate->_unresolvedcontinues.back();
01997             funcstate->_unresolvedcontinues.pop_back();
01998             //set the jmp instruction
01999             funcstate->SetInstructionParams(pos, 0, targetpos - pos, 0);
02000             ntoresolve--;
02001         }
02002     }
02003 private:
02004     SQInteger _token;
02005     SQFuncState *_fs;
02006     SQObjectPtr _sourcename;
02007     SQLexer _lex;
02008     bool _lineinfo;
02009     bool _raiseerror;
02010     SQInteger _debugline;
02011     SQInteger _debugop;
02012     SQExpState _es;
02013     SQScope _scope;
02014     SQChar _compilererror[MAX_COMPILER_ERROR_LEN];
02015     jmp_buf _errorjmp;
02016     SQVM *_vm;
02017 };
02018
02031 bool Compile(SQVM *vm, SQLEXREADFUNC rg, SQUserPointer up, const SQChar *sourcename, SQObjectPtr &out,
bool raiseerror, bool lineinfo)
02032 {
02033     SQCompiler p(vm, rg, up, sourcename, raiseerror, lineinfo);
02034     return p.Compile(out);
02035 }
02036
02037 #endif

```